



Cryptography Fundamentals

Prof. Catalin Boja, Ph.D.

IT&C Security Master

catalin.boja@ie.ase.ro



Course Added Value

- Assures you the full knowledge regarding meaning and usage of Computer Security concepts
- Covers Cryptography topics for [CEH Certified Ethical Hacking Certification](#), **CISSP: Certified Information Systems Security Professional** and **CISM: Certified Information Security Manager** and **CompTIA Security+**



Course Relation

Secure Application Programming

Implement in Java and C++ apps that use cryptographic solutions

Electronic Signature

More details on Electronic Signature
Key distribution
Protocols & more

Standards & Protocols

PKCSs
Key Distribution
Protocols

Audit & Legal

Legal stuff



Some Keywords

Hash

SHA-1

MD5

AES

MAC

ECB

CBC

Symmetric
Key

RSA

Public Key

Private Key

Encryption

Decryption

Digital
signature

DES

3DES



Other Keywords

- IETF RFC 2828 replaced by RFC 4949
- <https://tools.ietf.org/html/rfc2828>
- <https://tools.ietf.org/html/rfc4949>

Course objective



"I DON'T THINK YOU UNDERSTAND
THE CONCEPT OF CYBERSECURITY."



Course objective

theguardian

News | Sport | Comment | Culture | Business | Money | Life & style | ...

News > Technology > Adobe

Did your Adobe password leak? Now you and 150m others can check

Leak is 20 times worse than the company initially revealed, and could put huge numbers of peoples' online lives at risk

Alex Hern

Follow @alexhern Follow @guardiantech
theguardian.com, Thursday 7 November 2013 12.27 GMT

Jump to comments (25)



Adobe's HQ. The company leaked over 100m users' details. Photograph: PAUL SAKUMA/ASSOCIATED PRESS

Nearly 150 million people have been affected by a loss of customer data by Adobe, over 20 times more than the company admitted [in its initial statement last week](#).

As well as allowing the data to be stolen in the first place, Adobe made two other serious errors when storing the data. Firstly, it encrypted all the passwords with the same key; secondly, the encryption used a method (ECB mode) which renders the encrypted data insecure.

Every identical password also looks identical when encrypted. So if the database shows 1.9 million people whose password, when encrypted, reads "EQ7flpT7i/Q", then researchers know that they all have the same password.



Course objective

Giant Equifax data breach: 143 million people could be affected

by Sara Ashley O'Brien @saraashleyo

September 8, 2017: 9:23 AM ET



Equifax says a giant cybersecurity breach compromised the personal information of as many as 143 million Americans — almost half the country.

Recommend 107K

Social Surge - What's Trending

Japanese reporter died after clocking 159 hours of overtime

Trump hits a new low in attacks on the free press

NETFLIX
Netflix is raising its prices

Hot List

How GLAAD got Twitter to uncensor 'queer'

The eyes expose our lies. Now AI is noticing.

Facebook invests \$1 billion to build new



Course objective



Facebook just had its worst hack ever – and it could get worse

By Donie O'Sullivan, CNN Business

Updated 1322 GMT (2122 HKT) October 4, 2018





Course objective

- To give you the fundamentals for analyzing, assessing, using and implementing cryptographic solutions
- Covers Cryptography topics for **Certified Ethical Hacking Certification, CISSP: Certified Information Systems Security Professional** and **CISM: Certified Information Security Manager** and **CompTIA Security+**



References and recommended materials

1. Ion IVAN, Cristian TOMA – Informatics Security Handbook, 2nd Edition, Editura ASE, 2010
2. Cristian TOMA – Security in Software Distributed Systems, Editura ASE, 2008
3. Bruce Schneier – *Applied Cryptography*, Second Edition, Wiley, 1996
4. Niels Ferguson, Bruce Schneier – *Practical Cryptography*, John Wiley, 2003
5. Tom St Denis, Simon Johnson – *Cryptography for Developers*, Syngress, 2007 (<http://books.google.com>)
6. Alfred J. Menezes, Paul C. van Oorschot, Scott A. Vanstone – *Handbook of Applied Cryptography*, CRC Press, 1997 (<http://www.cacr.math.uwaterloo.ca/hac/>)
7. William Stallings – *Cryptography and Network Security Principles and Practices*, Fourth Edition, Prentice Hall, 2005
8. Cryptography and Cryptanalysis, MITOpenCourseware, <http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-875-cryptography-and-cryptanalysis-spring-2005/index.htm>
9. www.wikipedia.com / www.google.com



Cryptography Fundamentals

Section I – Cryptography basics

- Concepts
- Mathematical Background
- Prime numbers
- Random and Pseudorandom numbers

Section II –hash functions

- MD5
- SHA-1, SHA-2, SHA-3



Cryptography Fundamentals

Section III – Symmetric Algorithms

- Transposition ciphers
- Substitution ciphers
- OTP (One Time Pad) ciphers
- Complex ciphers (DES, AES - Rijndael)
- Encryption methods

Section IV – Asymmetric algorithms + cryptanalysis elements

- RSA



Concepts



Concepts

- **Cryptography** – **secret** writing science; the science of information security
- **Cryptanalysis** – science of “breaking” ciphertexts without knowing cipher key
- **Cryptology** – mathematic field that studies the mathematical fundamentals of cryptography



Concepts

- **Steganography:**
 - the art of **hiding** information;
 - the secret message is hidden in a public one (a image, sound file, text);
 - is NOT Cryptography
 - ...later covered by the Multimedia Security course (2nd year, 1st semester)



Cryptography

- Used to secure data in:
 - Networks: HTTPS, SSL/TLS, 802.11i WPA2 (Wi-Fi Protected Access), GSM, Bluetooth
 - Computers and mobile devices drives: TrueCrypt
 - DVD and Blu-ray disks: CSS (Content Scrambling System)
 - Software and Database applications: User authentication



Cryptography

Provides concepts for:

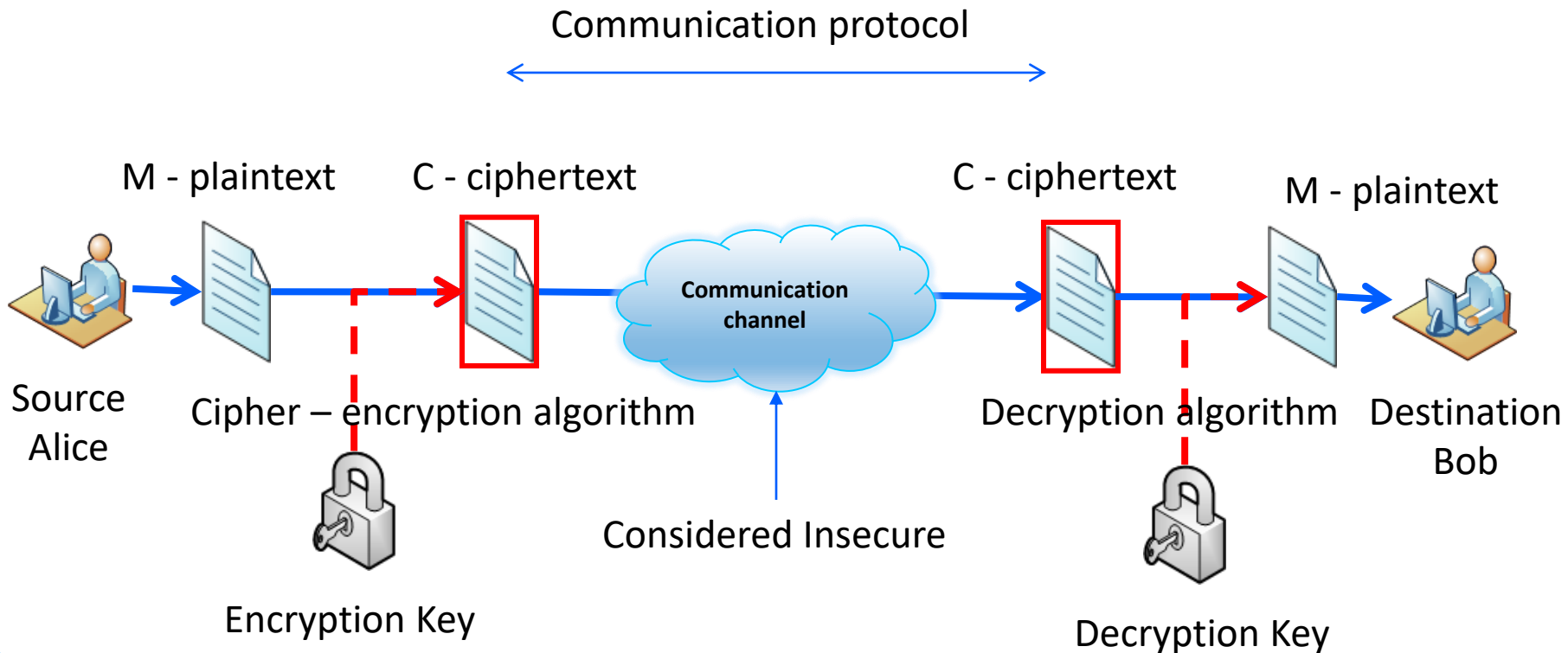
- Secret key establishment
- Secure communication
- Secure data
- Digital signatures
- Anonymous communication (Mix Net)
- Anonymous digital cash (crypto-currency)
- Electronic voting or auctions
- Protocols (like “Zero knowledge”)



Concepts



Key ingredients of a cryptographic system



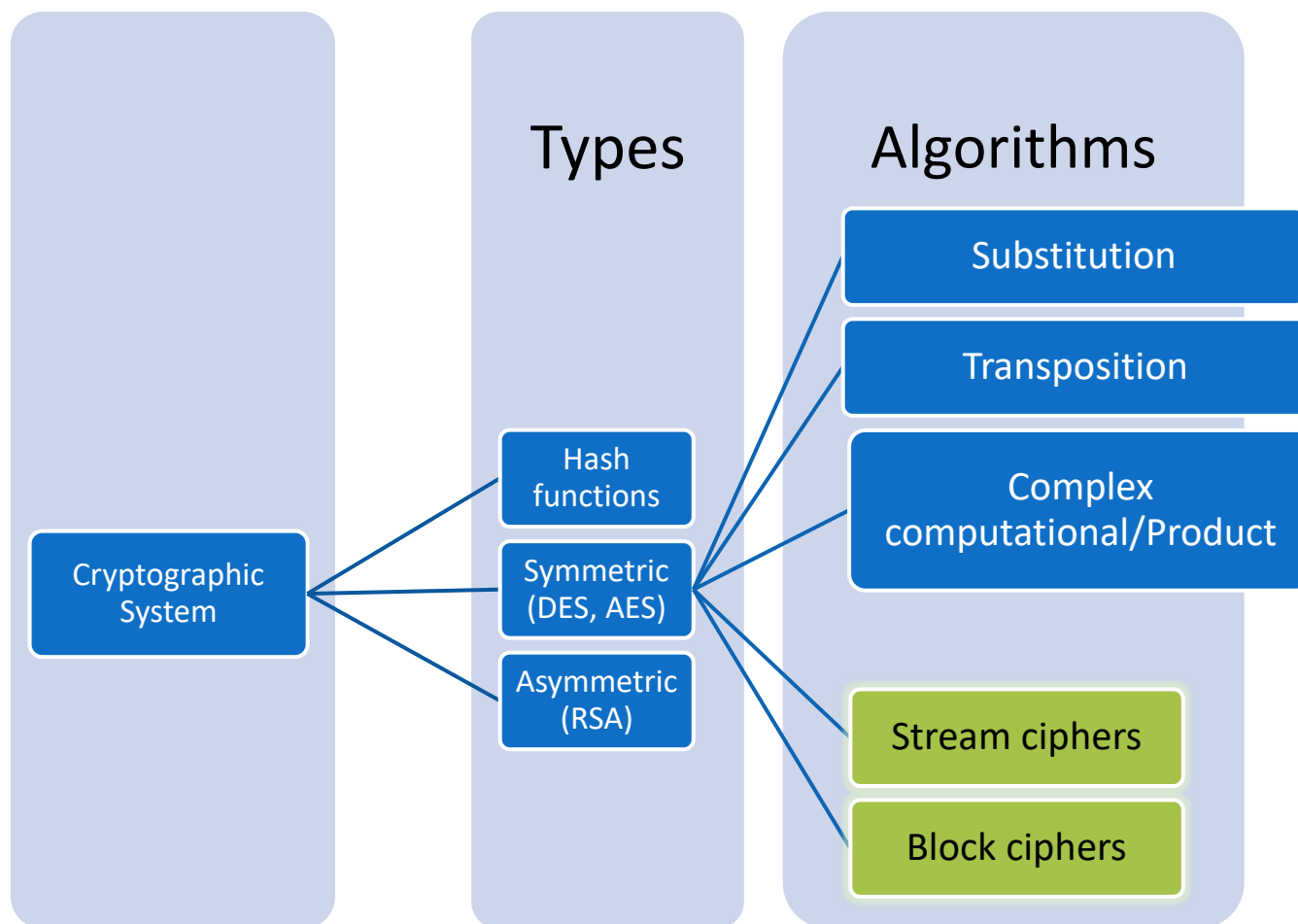


Concepts

- **unconditional security:** the cipher cannot be broken no matter how much computer power or time is available (one-time-pad);
- **computational security:** the cipher cannot be broken given limited computing resources (mostly time)



Cryptographic system

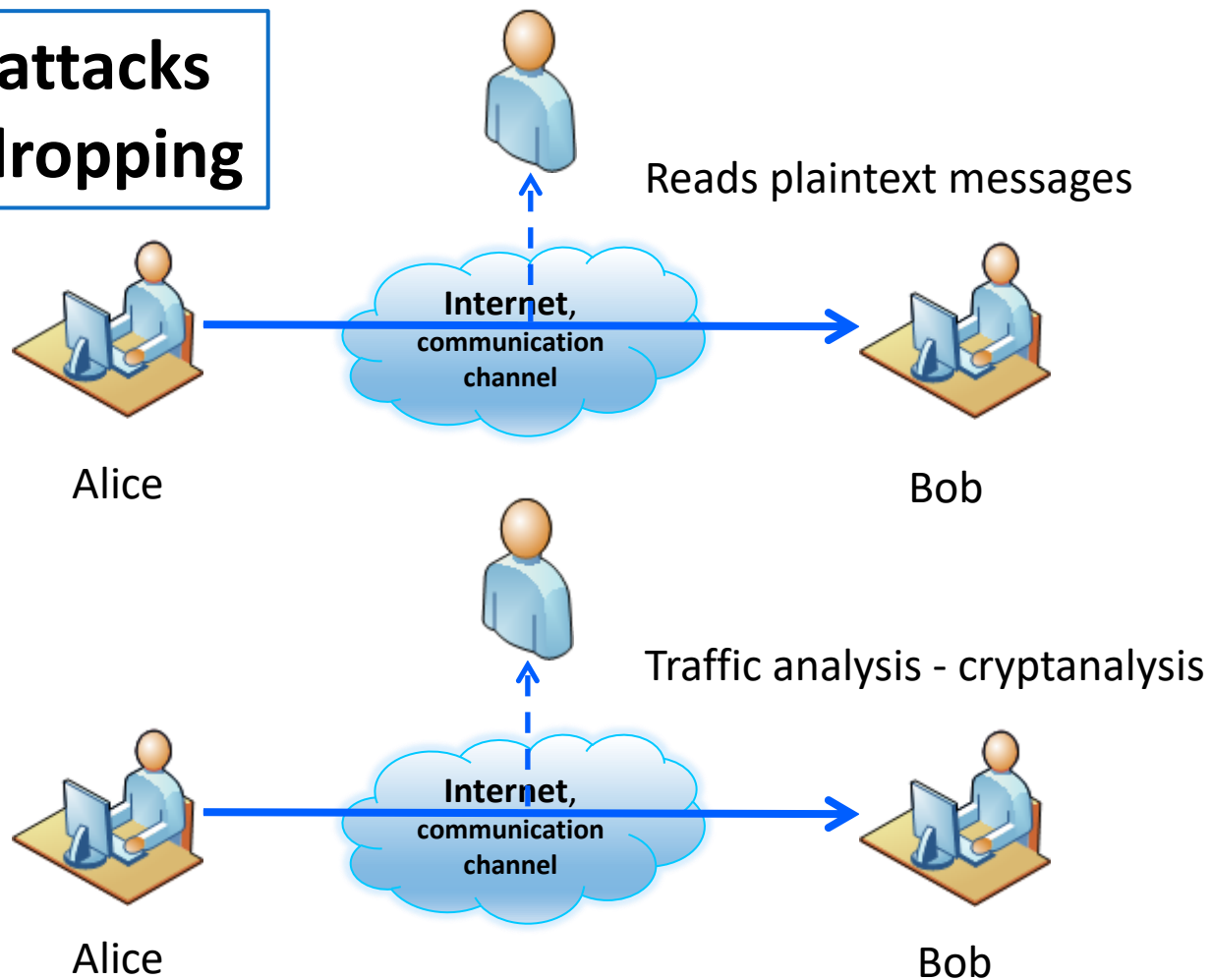




- www.ism.ase.ro

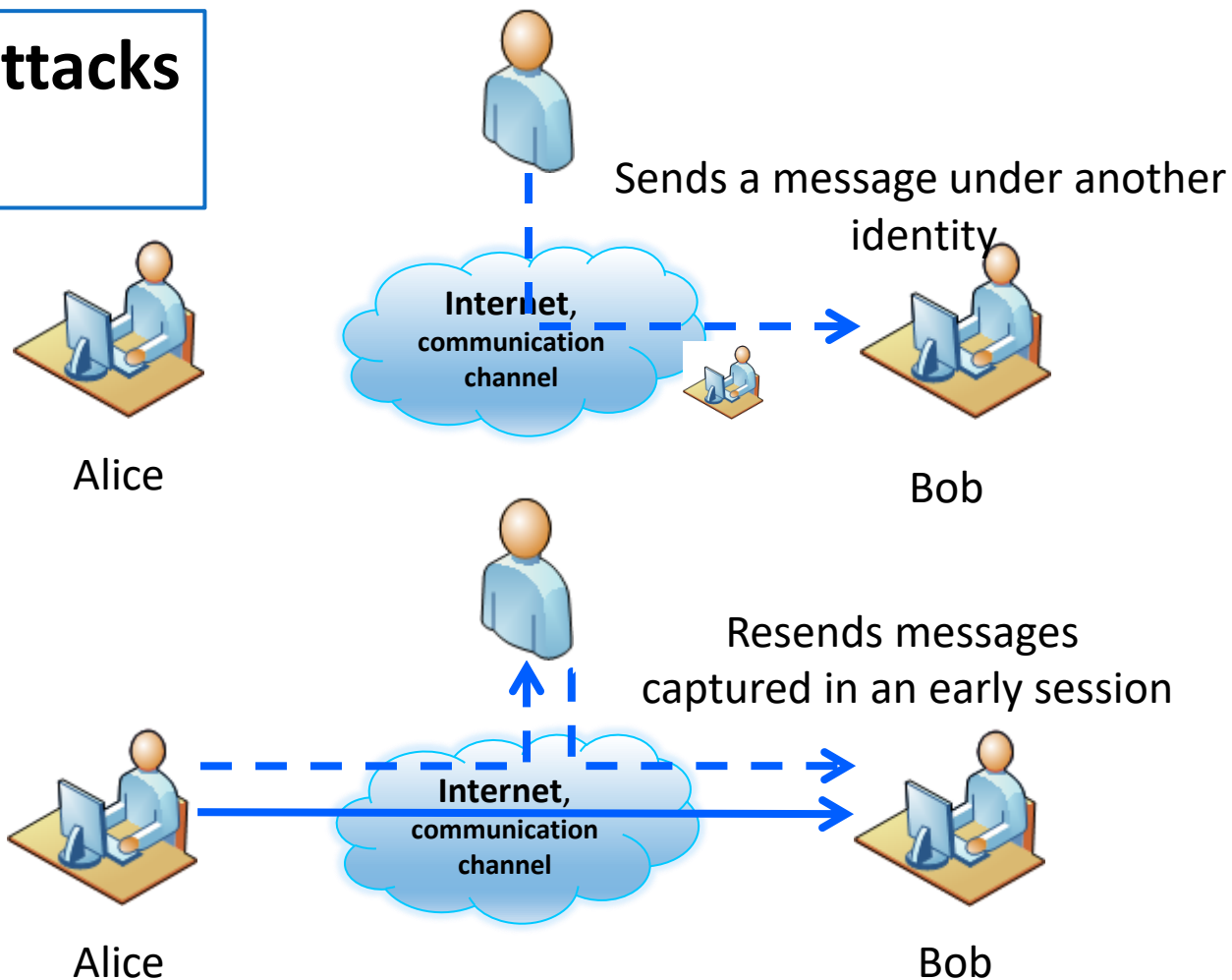
Vulnerabilities

Passive attacks - eavesdropping



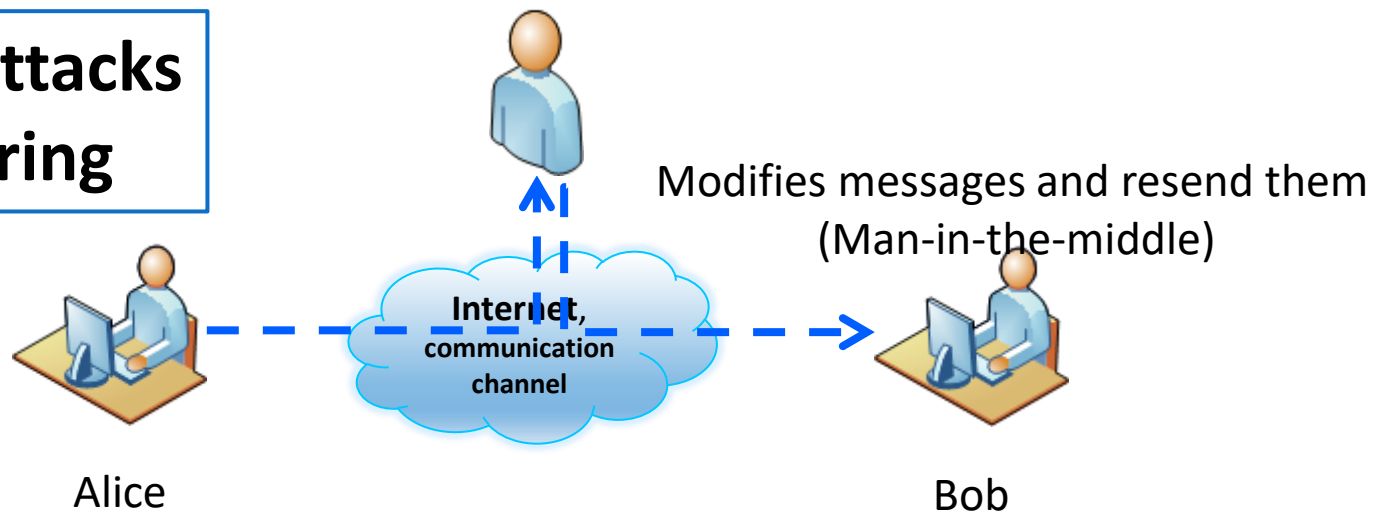
Vulnerabilities

Active attacks - replay



Vulnerabilities

Active attacks - tampering





Vulnerabilities

Attack type	Attacker knowledge
Ciphertext-only	-Encryption algorithm -Encrypted messages
Known-plaintext	- Encryption algorithm - Encrypted messages - Plaintext $\leftrightarrow$ ciphertext texts
Chosen-plaintext	- Encryption algorithm - Encrypted messages - Plaintext $\leftrightarrow$ ciphertext texts - Can choose the plaintext to be encrypted
Chosen-ciphertext	- Encryption algorithm - Can choose the ciphertext to be decrypted
Timing Attacks	- Measures the exact execution time of modular exponentiation/encryption operations
Rubber-hose	- Acquires information by bribery, threatens, blackmail, theft (many times is the cheapest solution)



Objectives

- To secure messages and transactions in software distributed systems
- Cryptographic systems characteristics:
 - Total or partial **confidentiality**
 - **Authentication**
 - **Data integrity**
 - **Nonrepudiation**
- Security Services:
 - X.800, <http://www.itu.int/rec/T-REC-X.800-199103-1>
 - RFC 2828, <http://www.ietf.org/rfc/rfc2828.txt>
 - RFC 4949, <https://tools.ietf.org/html/rfc4949>



Objectives - X.800

1. **Authentication:** *Peer entity authentication and Data origin authentication*
2. **Access Control**
3. **Data Confidentiality:** *Connection, Connectionless and Selective field confidentiality*
4. **Data Integrity**
 - with Recovery
 - without Recovery
5. **Nonrepudiation**
 - Origin
 - Destination



Objectives

- Data integrity assures data received is exactly as the one sent:
 - no modification
 - no insertion
 - no deletion
 - no replay
- Can provide
 - integrity with recovery (ex. [QR codes recovery](#))
 - Integrity without recovery (detection only)



Objectives – RFC 4949

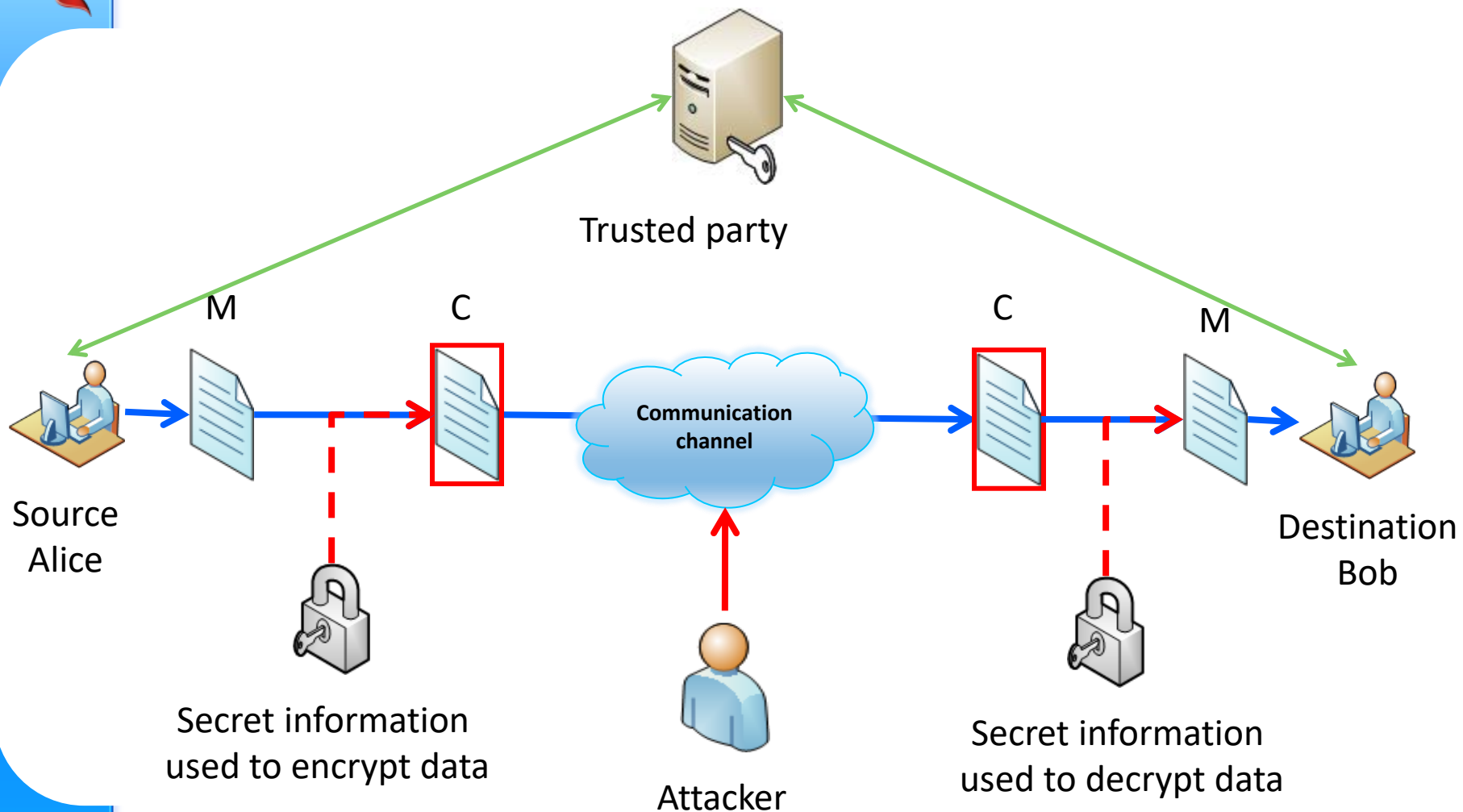
1. Access control service
2. Audit service
3. Authentication service
 - data origin authentication
 - peer entity authentication
4. Availability service
5. Data confidentiality service
6. System integrity service
7. Non-repudiation service



Objectives

Objectives	Properties
Confidentiality	Hides the message content Implemented by symmetric algorithms that generate ciphertexts <u>Does NOT</u> assures the integrity and accuracy of the content
Integrity	Guarantees the integrity and accuracy of the content Implemented by <i>one-way hash functions</i> that generate message digest values
Authentication	Ensures the communication parties identities It presumes that the communication channel is not safe Implemented by <i>Message Authentication Functions (MAC)</i> that generate message tag values.
Non-repudiation <i>Proof of Origin</i>	Guarantees the message source, the connection between the source and its sent message. Prevents situations in which the source denies it has sent the message Implemented by <i>public key digital</i> signatures that generates signature values

Security model





- Get any transmitted message throughout communication channel
- It is a network user (with rights)
- Opens communication channels with other users
- He can become the destination of a message
- He sends messages in the name of another user
- Has full control over the network



Dolev-Yao Security Model

What the attacker **CAN NOT** DO:

- He **CAN'T** guess a random number from a large enough set
- Without the secret key, he **CAN'T** get the plaintext and he **CAN'T** get a valid cipher (depends on the encryption algorithm)
- He **CAN'T** generate the private key related to a public key
- He **DOES'T** have physical access to the user machine



Security goals

- **unconditionally secure encryption:**
 - no matter how much ciphertext is available the plaintext can not be determined (only OTP)
- **computationally secure encryption:**
 - the cost of breaking the cipher $>$ the value of the encrypted information
 - the time required to break the cipher $>$ the useful lifetime of the information



Kerckhoff's Principle

- Proposed in 1883
 - The strength of the cryptosystem is given by the secrecy of the key
 - The algorithm is made public
 - After the algorithm has been analyzed and is proven it has no flaws, weak keys, backdoors, it is considered strong



MATHEMATICAL BACKGROUND



XOR logical function

- XOR function (exclusive or) – one of the most used function in cryptographic systems
- Available in programming languages like C, C++, Java and represented by the $\wedge$ operator
- Implements mod 2 addition

X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0



XOR logical function

- Has an essential role in OTP ciphers (one-time pad, stream ciphers) and AES (Advanced Encryption Standard)
- The sequence `rez = a xor c, b = rez xor c` transfers *a* value to *b*

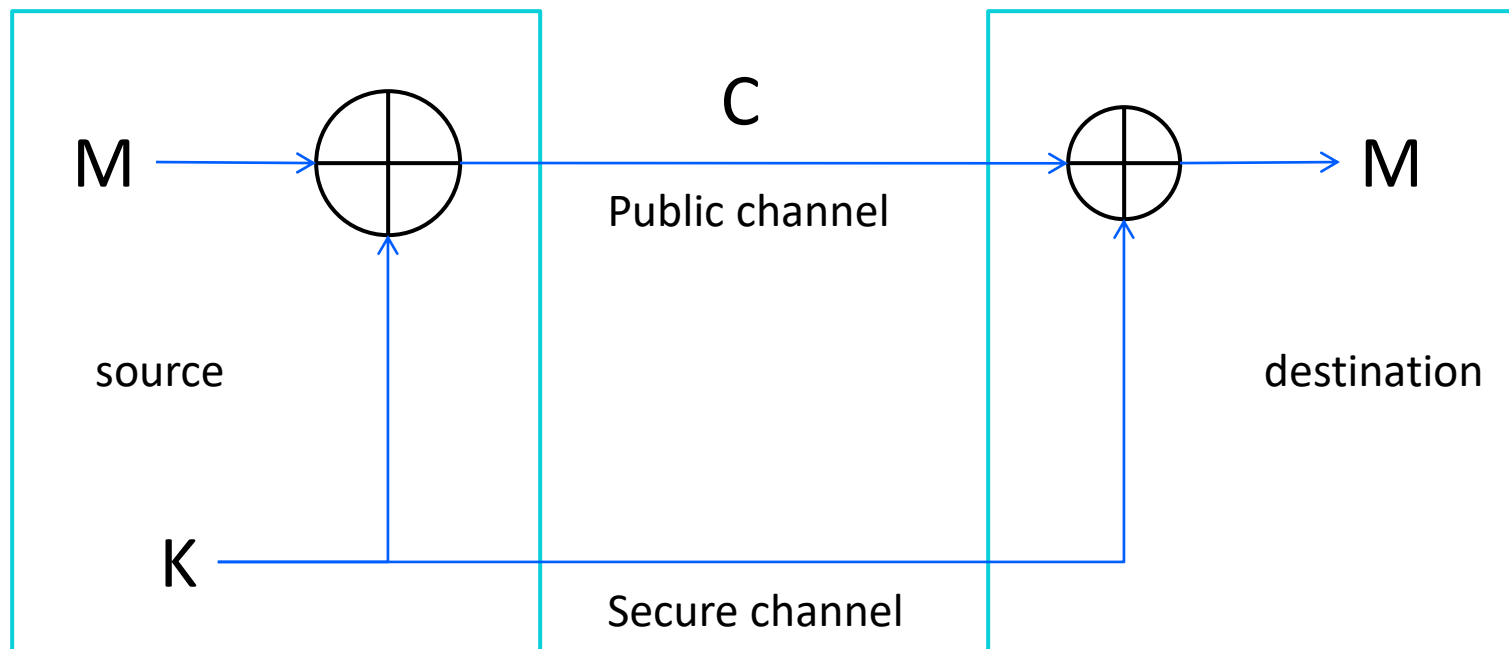
```
temp = a;  
a = b;  
b = temp;
```



```
a = a xor b;  
b = a xor b;  
a = a xor b;
```



XOR logical function



Cryptographic system based on XOR:

- the fastest
- the simplest
- the most secure one (!!! in some particular conditions: large messages with an equal size random generated key)



Modular Arithmetic

- define **modulo operator** " $a \bmod n$ " to be remainder when a is divided by n
- use the term **congruence** for: $a \equiv b \bmod n$
 - when divided by n , a & b have same remainder
 - eg. $25 \equiv 11 \bmod 7$
- b is called a **residue** of $a \bmod n$
 - because: $a = qn + b$
 - usually chose smallest positive remainder as residue
 - ie. $0 \leq b \leq n-1$
 - process is known as **modulo reduction**
 - eg. $-12 \bmod 7 = -5 \bmod 7 = 2 \bmod 7 = 9 \bmod 7$



Modular Arithmetic

- 'clock arithmetic'
- **uses a finite number of values;**
- **generates results in the same set**
- can do reduction at any point:
 - $a+b \bmod n = [a \bmod n + b \bmod n] \bmod n$
- can do modular arithmetic with any group of integers:
$$\mathbb{Z}_n = \{0, 1, \dots, n-1\}$$



Modular Arithmetic

+ 0 1 2 3 4 5 6 7

0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

**Modulo 8
Addition Example**



- $(a+b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$
- $(a-b) \bmod n = ((a \bmod n) - (b \bmod n)) \bmod n$
- $(a*b) \bmod n = ((a \bmod n) * (b \bmod n)) \bmod n$
- $(a*(b+c)) \bmod n = (((a*b) \bmod n) + ((a*c) \bmod n)) \bmod n$
- for a k bits modulus the intermediate result of any $+, -, *$ has a maximum of $2k$ bits
- $a^8 \bmod n =$
 - simplest solution: $(a*a*a*a*a*a*a*a) \bmod n$
 - addition chaining: $((a^2 \bmod n)^2 \bmod n)^2 \bmod n$



Greatest Common Divisor (GCD)

- a common problem in number theory
- $\text{GCD}(a,b)$ of a and b is the largest number that divides evenly into both a and b
 - $\text{GCD}(60,24) = 12$
- used to check **relatively prime numbers** (with no common factors, except 1):
 - $\text{GCD}(8,15) = 1$
 - 8 & 15 are relatively prime;



- www.ism.ase.ro



Groups

- A group is a set of group elements with a binary operation for combining any two elements to get a unique third element from the set [wiki].
- If $\#$ is the group operation and a, b are two group elements:
 - $a\#b = c$, a group element;
 - $a\#(b\#c) = (a\#b)\#c$, it is associative;
 - $a\#e = e\#a = a$, where e is the *identity element*
 - $a\#a^{-1} = a^{-1}\#a = e$, where a^{-1} is the inverse of a
- The group is **abelian** if $\#$ is commutative



Groups

- $\{Z_n, \text{integers mod } n\}$, is an often used group:
 - the operation is addition followed by remainder on division by n ;
 - the identity element is 0;
 - the inverse of a is $n-a$ (except for 0);
 - for n fixed, it is a finite group;
- Z_p integers mod p , where p is a prime number, is another favorite group;
- $GF(2^n)$ finite group (for AES, the operations of the 2^8 finite group)



- a group is cyclic if every element is a power of some fixed element:
 $\mathbf{b = a^k}$, where a and b are from the group
- $\mathbf{a}$ is the **generator** of the group;
- the **identity element** is $e = a^0$;



Ring

- a set of “numbers” in which we can do addition, subtraction and multiplication without leaving the set
- a set with two operations (addition and multiplication) which form:
 - an abelian group with addition operation;
 - and multiplication:
 - has closure
 - is associative
 - distributive over addition: $a(b+c) = ab + ac$



Field

- a set of numbers
- with two operations which form:
 - abelian group for addition
 - abelian group for multiplication (ignoring 0)
 - ring
- have hierarchy with more axioms/laws
 - group $\rightarrow$ ring $\rightarrow$ field
- Examples: real and complex numbers; NOT integers



Galois Fields

- finite fields play a key role in cryptography
- elements in a finite field **must** be a power of a prime p^n
- denoted $GF(p^n)$
- in particular often use the fields:
 - $GF(p)$ - is the set of integers $\{0, 1, \dots, p-1\}$ with arithmetic operations modulo prime p
 - $GF(2^n)$



Galois Fields

GF(7) Multiplication

×	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6
2	0	2	4	6	1	3	5
3	0	3	6	2	5	1	4
4	0	4	1	5	2	6	3
5	0	5	3	1	6	4	2
6	0	6	5	4	3	2	1



Polynomial Arithmetic

- can compute values using polynomials

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 = \sum a_i x^i$$

- add or subtract corresponding coefficients
- multiply all terms by each other

$$\text{For } f(x) = x^3 + x^2 + 2 \text{ and } g(x) = x^2 - x + 1$$

$$f(x) + g(x) = x^3 + 2x^2 - x + 3$$

$$f(x) - g(x) = x^3 + x + 1$$

$$f(x) \times g(x) = x^5 + 3x^2 - 2x + 2$$



Polynomial Arithmetic with Modulo Coefficients

- when computing value of each coefficient do calculation modulo some value
 - forms a polynomial ring
- could be modulo any prime
- but the most used is mod 2
 - ie all coefficients are 0 or 1
 - eg. let $f(x) = x^3 + x^2$ and $g(x) = x^2 + x + 1$
$$f(x) + g(x) = x^3 + x + 1$$
$$f(x) \times g(x) = x^5 + x^2$$



Polynomial Division

- can write any polynomial in the form:
 - $f(x) = q(x) g(x) + r(x)$
 - interpret $r(x)$ as being a remainder
 - $r(x) = f(x) \bmod g(x)$
- if have no remainder say $g(x)$ divides $f(x)$
- if $g(x)$ has no divisors other than itself & 1 say it is **irreducible** (or prime) polynomial
- arithmetic modulo an irreducible polynomial forms a field



Polynomial GCD

- can find greatest common divisor for polys
 - $c(x) = \text{GCD}(a(x), b(x))$ if $c(x)$ is the poly of greatest degree which divides both $a(x), b(x)$
- can adapt Euclid's Algorithm to find it:
EUCLID[$a(x), b(x)$]
 1. $A(x) = a(x); B(x) = b(x)$
 2. **if** $B(x) = 0$ **return** $A(x) = \text{gcd}[a(x), b(x)]$
 3. $R(x) = A(x) \bmod B(x)$
 4. $A(x) \leftarrow B(x)$
 5. $B(x) \leftarrow R(x)$
 6. **goto** 2



Modular Polynomial Arithmetic

- can compute in field $GF(2^n)$
 - polynomials with coefficients modulo 2
 - whose degree is less than n
 - hence must reduce modulo an irreducible poly of degree n (for multiplication only)
- form a finite field
- can always find an inverse
 - can extend Euclid's Inverse algorithm to find



Example GF(2³)

Table 4.6 Polynomial Arithmetic Modulo ($x^3 + x + 1$)

		000 0	001 1	010 x	011 $x + 1$	100 x^2	101 $x^2 + 1$	110 $x^2 + x$	111 $x^2 + x + 1$
000	0	0	1	x	$x + 1$	x^2	$x^2 + 1$	$x^2 + x$	$x^2 + x + 1$
001	1	1	0	$x + 1$	x	$x^2 + 1$	x^2	$x^2 + x + 1$	$x^2 + x$
010	x	x	$x + 1$	0	1	$x^2 + x$	$x^2 + x + 1$	x^2	$x^2 + 1$
011	$x + 1$	$x + 1$	x	1	0	$x^2 + x + 1$	$x^2 + x$	$x^2 + 1$	x^2
100	x^2	x^2	$x^2 + 1$	$x^2 + x$	$x^2 + x + 1$	0	1	x	$x + 1$
101	$x^2 + 1$	$x^2 + 1$	x^2	$x^2 + x + 1$	$x^2 + x$	1	0	$x + 1$	x
110	$x^2 + x$	$x^2 + x$	$x^2 + x + 1$	x^2	$x^2 + 1$	x	$x + 1$	0	1
111	$x^2 + x + 1$	$x^2 + x + 1$	$x^2 + x$	$x^2 + 1$	x^2	$x + 1$	x	1	0

(a) Addition

		000 0	001 1	010 x	011 $x + 1$	100 x^2	101 $x^2 + 1$	110 $x^2 + x$	111 $x^2 + x + 1$
000	0	0	0	0	0	0	0	0	0
001	1	0	1	x	$x + 1$	x^2	$x^2 + 1$	$x^2 + x$	$x^2 + x + 1$
010	x	0	x	x^2	$x^2 + x$	$x + 1$	1	$x^2 + x + 1$	$x^2 + 1$
011	$x + 1$	0	$x + 1$	$x^2 + x$	$x^2 + 1$	$x^2 + x + 1$	x^2	1	x
100	x^2	0	x^2	$x + 1$	$x^2 + x + 1$	$x^2 + x$	x	$x^2 + 1$	1
101	$x^2 + 1$	0	$x^2 + 1$	1	x^2	x	$x^2 + x + 1$	$x + 1$	$x^2 + x$
110	$x^2 + x$	0	$x^2 + x$	$x^2 + x + 1$	1	$x^2 + 1$	$x + 1$	x	x^2
111	$x^2 + x + 1$	0	$x^2 + x + 1$	$x^2 + 1$	x	1	$x^2 + x$	x^2	$x + 1$

(b) Multiplication



Polynomial Arithmetic with Modulo Coefficients

- since coefficients are 0 or 1, can represent any such polynomial as a bit string
- **addition becomes XOR of these bit strings**
- **multiplication is shift & XOR**
 - cf long-hand multiplication
- modulo reduction done by repeatedly substituting highest power with remainder of irreducible poly (also shift & XOR)



Polynomial Arithmetic with Modulo Coefficients

- in $GF(2^3)$ have (x^2+1) is 101_2 & (x^2+x+1) is 111_2
- so addition is
 - $(x^2+1) + (x^2+x+1) = x$
 - $101 \text{ XOR } 111 = 010_2$
- and multiplication is
 - $(x+1).(x^2+1) = x.(x^2+1) + 1.(x^2+1)$
 $= x^3+x+x^2+1 = x^3+x^2+x+1$
 - $011.101 = (101) \ll 1 \text{ XOR } (101) \ll 0 =$
 $1010 \text{ XOR } 101 = 1111_2$
- polynomial modulo reduction (get $q(x)$ & $r(x)$) is
 - $(x^3+x^2+x+1) \bmod (x^3+x+1) = 1.(x^3+x+1) + (x^2) = x^2$
 - $1111 \bmod 1011 = 1111 \text{ XOR } 1011 = 0100_2$



Theorems

- **Fermat Theorem:** if p is a prime and a is $0 < a < p$ then $a^{p-1} \bmod p = 1$; also $a^x \bmod p = a^{x \bmod (p-1)} \bmod p$ (because a to a power mod p always starts repeating after the power reaches $p-1$)
- **Euler Theorem:** If n is any positive integer and a is any positive integer, $a < n$, with no divisors in common with n , then

$$a^{\phi(n)} \bmod n = 1$$

- Where $\phi(n)$ (*Euler phi function*) is $n (1 - 1/p_1) \dots (1 - 1/p_m)$, with $p_1 \dots p_m$ prime numbers that divide n



Euclidean Algorithm

- an efficient way to find the $\text{GCD}(a,b)$;
- uses theorem that:
 - $\text{GCD}(a,b) = \text{GCD}(b, a \bmod b)$
- Euclidean Algorithm to compute $\text{GCD}(a,b)$ is:

`EUCLID(a,b)`

`1. A = a; B = b`

`2. if B = 0 return A = gcd(a, b)`

`3. R = A mod B`

`4. A = B`

`5. B = R`

`6. goto 2`



Extended Euclidean Algorithm

EXTENDED EUCLID(m, b)

1. $(A1, A2, A3) = (1, 0, m);$
 $(B1, B2, B3) = (0, 1, b)$
2. **if** $B3 = 0$
return $A3 = \gcd(m, b);$ no inverse
3. **if** $B3 = 1$
return $B3 = \gcd(m, b); B2 = b^{-1} \bmod m$
4. $Q = A3 \text{ div } B3$
5. $(T1, T2, T3) = (A1 - Q B1, A2 - Q B2, A3 - Q B3)$
6. $(A1, A2, A3) = (B1, B2, B3)$
7. $(B1, B2, B3) = (T1, T2, T3)$
8. **goto** 2



Inverse of 550 in $GF(1759)$

Q	A1	A2	A3	B1	B2	B3
—	1	0	1759	0	1	550
3	0	1	550	1	-3	109
5	1	-3	109	-5	16	5
21	-5	16	5	106	-339	4
1	106	-339	4	-111	355	1

Source: [7]



Fast integer exponentiation

- $x^y = x * x * x * \dots * x$ for y times
 - takes a long time for big numbers
- It's faster with the repeating squaring algorithm
- Any number can be written as a sum of power of 2 based values
- $Y = 53 = (110101)^2 = 32 + 16 + 4 + 1$
-



Prime numbers

“The problem of distinguishing prime numbers from composite numbers and of resolving the latter into their prime factors is known to be one of the most important and useful in arithmetic.”

Carl Friedrich Gauss (1805)

- Test primes
- Factor a composite number in primes



Prime numbers

- Large random prime integers are important components of a cryptographic system
- Test that verifies if a number is probably prime
 - *Simple Pseudo-prime Test*; are used to increase the algorithm efficiency; the probability to get a correct result is so high that risks are accepted



Prime numbers

- there are around 10151 primes, 512 bits in length or less[3]
- test a prime number:
 - checks $n\%i$ with $i = 2..n-1$; with $i = 2..\text{sqrt}(n)$
 - Solovay-Strassen
 - Fermat
 - Rabin-Miller



- calculate b , where b is the number of times 2 divides $p - 1$
- calculate m , such that $p = 1 + 2^b \cdot m$.
- (1) Choose a random number, a , such that $a < p$.
- (2) Set $j = 0$ and set $z = a^m \bmod p$.
- (3) If $z = 1$, or if $z = p - 1$, then p passes the test and may be prime.
- (4) If $j > 0$ and $z = 1$, then p is not prime.
- (5) Set $j = j + 1$. If $j < b$ and $z \neq p - 1$, set $z = z^2 \bmod p$ and go back to step(4). If $z = p - 1$, then p passes the test and may be prime.
- (6) If $j = b$ and $z \neq p - 1$, then p is not prime.



- Test (based on **Fermat theorem**): If a very large random integer p (100 decimal digits or more) is not divisible by a small prime a (2 **or** 3) and if $a^{p-1} \bmod p = 1$, then the number is prime except for a small probability, that can be ignored
- there are non-primes numbers, *Carmichael numbers*, that satisfy Fermat's theorem for all values of a even though they are not prime ($561 = 3 * 11 * 17$), but these numbers become very rare in the larger range, such as 1024-bit numbers



Prime numbers

- there are 20,138,200 Carmichael numbers between 1 and 10^{21} (approximately one in 50 trillion ($5 \cdot 10^{13}$) numbers)
(http://en.wikipedia.org/wiki/Carmichael_number)



Factoring composite numbers

- The best known algorithm: Number Field Sieve (NFS) factorization of large integers (http://en.wikipedia.org/wiki/General_number_field_sieve)
- Current world record: RSA-768 (232 digits) – 2 years on hindered of machines
- Factoring a 1024 bit integer: estimated about 1000 times harder (Dan Boneh, 2012)



Integer factorization problem

For a positive integer n get the factorization $n = p_1^{e_1} p_2^{e_2} \dots p_k^{e_k}$ where p_i are prime values and $e_i \geq 1$.

- Cryptographic algorithms based on this problem:
 - RSA public key encryption
 - RSA signature
 - Rabin public key encryption



Entropy

- the entropy of X represents a mathematical measurement of the amount of information obtained by analyzing X.
- is the uncertainty regarding the result before analyzing X;
- it represent [Claude Shannon] the number of bits needed to give the shortest binary representation of the message

- Measured by

$$\sum_{i=1}^n p_i \log_2 \left(\frac{1}{p_i} \right)$$



Entropy

What's the entropy of your 8 case-insensitive alpha (a-z) chars password ?



Entropy

Scenario	Available Characters	Required Password Length for 56-Bit Key	Required Password Length for 128-Bit Key
Numeric PIN	10 (0–9)	17	40
Case-insensitive alpha	26 (A–Z or a–z)	12	28
Case-sensitive alpha	52 (A–Z and a–z)	10	23
Case-sensitive alpha and numeric	62 (A–Z, a–z, and 0–9)	10	22
Case-sensitive alpha, numeric, and punctuation	93 (A–Z, a–z, 0–9, and punctuation)	9	20



Complexity

- Algorithm complexity is measured by:
 - Input length
 - Processing time
- Complexity classes
 - constant, $f(n) = 1$;
 - linear, $f(n) = n$;
 - logarithmic, $f(n) = \log_2 n$;
 - square, $f(n) = n^2$;
 - cubic, $f(n) = n^3$
 - polynomial, $f(n) = n^c$, cu $c > 1$;
 - exponential, $f(n) = 2^n$ or $f(n) = a^n$, cu $a > 1$.
 - factorial, $f(n) = n!$



Complexity

	Complexity
Direct access search	$O(1)$
Sequential search	$O(n)$
Binary search	$O(\log_2 n)$
Search in hash tables	$O(GU_{\text{hash}})$
Search in binary balanced search trees (AVL, Red & Black)	$O(\log_2 n)$
Search in B trees	$1 + \log_N((n+1)/2)$, where N is the B tree order
Sequential search in files	$O(n)$
Direct access search in files	$O(1)$
Search in indexed files	$O(\log_2 n)$ for an index of binary balanced search trees type
Search in reverse files	$O(n)$

90



Complexity

- Example – students distribution in dormitories

P vs NP

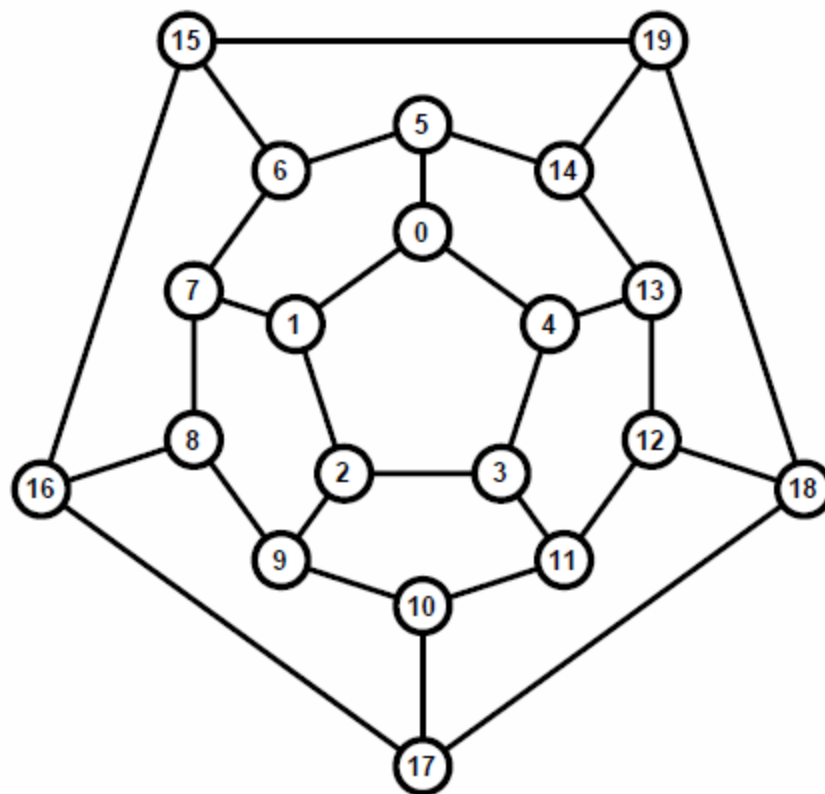
Is **P = NP**?

Is **NP = co-NP**?

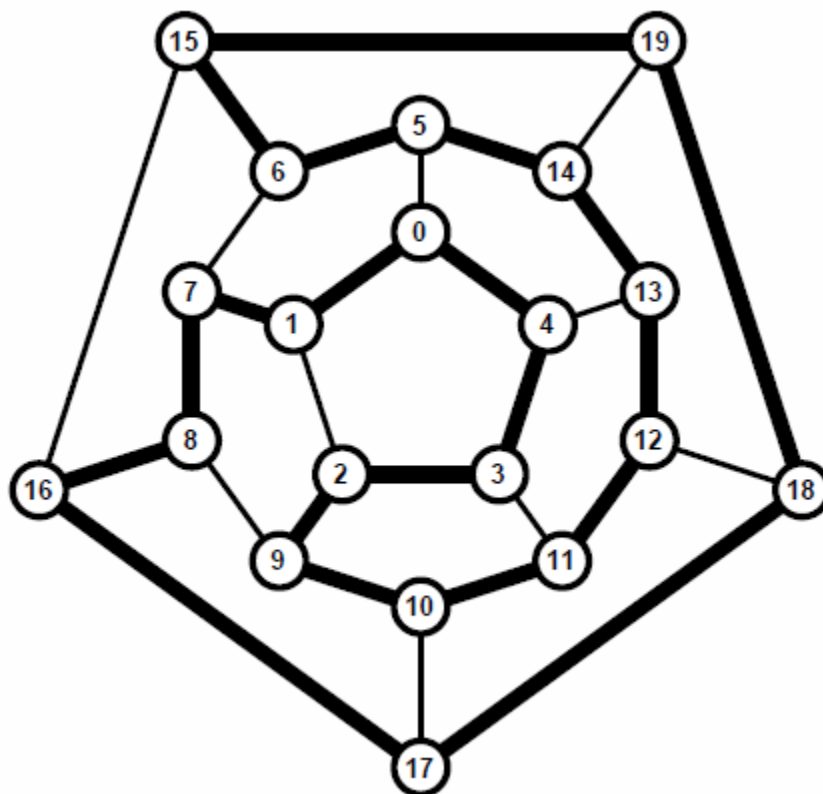
- One of the unsolved math theories
http://en.wikipedia.org/wiki/Millennium_Prize_Problems
- Over 3000 NP identified problems
http://en.wikipedia.org/wiki/List_of_NP-complete_problems



Complexity



Complexity





- 94



Complex theories

- From a mathematic viewpoint, the strength of a cryptographic algorithm = problem complexity
- A problem is considered simple if it can be solved (or a large part of solutions) in a polynomial time
- Are defined based on mathematic problems with unknown real complexity
- In well defined conditions (input data carefully selected) the solution is almost impossible to be determined



- The integer factorization problem
 - The RSA problem – *RSA inversion*
 - The knapsack problem – subset sum problem
 - The quadratic residuosity problem
 - Computing square roots in $\mathbb{Z}_n$
 - The discrete logarithm problem
 - The generalized discrete logarithm problem
 - The Diffie-Hellman problem
 - The generalized Diffie-Hellman problem
- * for a mathematical analysis consult [6]



- Cryptographic algorithms based on this problem:
 - RSA public key encryption
 - RSA signature
 - Rabin public key encryption



RSA problem – *RSA inversion*

Being given:

- a positive integer number, n that is the product of two prime numbers, p and q
- a positive integer number, e and $\gcd(e, (p-1)(q-1)) = 1$
- an integer c

find an integer m such that $m^e \equiv c \pmod{n}$

The conditions imposed by n and e guarantees the uniqueness of the solution $m \in \{0, 1, \dots, n-1\}$ for each integer $c \in \{0, 1, \dots, n-1\}$

- Cryptographic algorithms based on this problem:
 - RSA public key encryption
 - RSA signature



- Cryptographic algorithms based on this problem:
 - Goldwasser-Micali public key encryption
 - Blum-Blum-Shub pseudo-random number generator



Square roots modulo n

- Given a composite integer n , with unknown prime factors, and $a \in Q_n$ (the set of quadratic residues modulo n), find a square root of a modulo n ; that is, an integer x such that $x^2 \equiv a \pmod{n}$



Discrete logarithm problem

- given a prime p , a generator α of $(\mathbb{Z}_p)^*$ group, and an element $\beta \in (\mathbb{Z}_p)^*$, find the integer x , $0 \leq x \leq p - 2$, such that $\alpha^x \equiv \beta \pmod{p}$.
- Cryptographic algorithms based on this problem:
 - Diffie-Hellman *key agreement* protocol
 - ElGamal encryption
 - ElGamal electronic signature



Diffie-Hellman problem

- Given a prime number p , a generator α for $(\mathbb{Z}_p)^*$ group, and elements $\alpha^a \bmod p$ and $\alpha^b \bmod p$, find $\alpha^{ab} \bmod p$.
- Cryptographic algorithms based on this problem:
 - Diffie-Hellman *key agreement* protocol
 - ElGamal encryption



Knapsack problem – subset sum problem

- Given a set of positive integers $\{a_1, a_2, \dots, a_n\}$ and a positive integer value S , determine whether or not there is a subset of values a_j that sums to S
- Cryptographic algorithms based on this problem:
- The first Merkle-Hellman public key encryption scheme



Knapsack problem – subset sum problem

Possible solution 1:

- INPUT: a set of positive values $\{a_1, a_2, \dots, a_n\}$ and the integer $s > 0$.
- OUTPUT: $x_i \in \{0, 1\}$, $1 \leq i \leq n$, such that $\sum_{i=1}^n a_i x_i = s$, if x_i exists.

1. For each possible array $(x_1, x_2, \dots, x_n) \in (\mathbb{Z}_2)^n$:

- It is determined $l = \sum_{i=1}^n a_i x_i$
- If $l == s$ then one solution is $(x_1, x_2, \dots, x_n)$.

2. If all possible arrays are verified - > there is no solution

Solution complexity = $O(2^n)$ – inefficient



Knapsack problem – subset sum problem

Solution 2 - Meet-in-the-middle

- INPUT: a set of positive values $\{a_1, a_2, \dots, a_n\}$ and the integer $s > 0$.
 - OUTPUT: $x_i \in \{0, 1\}$, $1 \leq i \leq n$, such that $\sum_{i=1}^n a_i x_i = S$, if x_i exists.
1. It is determined $t = n/2$
 2. There are defined the arrays $(\sum_{i=1}^t a_i x_i, (x_1, x_2, \dots, x_t))$ sorted by the partial sum value; these arrays define a matrix
 3. For each array $(x_{t+1}, x_{t+2}, \dots, x_n) \in (Z_2)^{n-t}$:
 - it is computed $l_i = S - S_i$ and it is verified if l_i is in the previous defined matrix, with $S_i = \sum_{i=t+1}^n a_i x_i$
 - If l_i is in the matrix then the solution is found
 4. If all possible arrays are verified - > there is no solution

Solution complexity = $O(2^{n/2})$ – inefficient



Large (Big) numbers

- numbers with over 10 digits – mostly 100

Ex [3]:

- Age of Universe: 2^{34}
- Numbers of atoms in the planet: $2^{170} \leftrightarrow 10^{51}$



Large (Big) numbers

Problems for big numbers ($n = 1024/2048$ bit numbers):

- **How you store them ?**
- **How you process them with arithmetic operations?**



Large (Big) numbers

- Stored as fixed length blocks (with additional padding)
- Addition and subtraction (complexity $O(n)$)
- Multiplication
 - the basic approach – complexity $O(n^2)$
 - Karatsuba (1960) – complexity $O(n^{1.585})$
- Division with remainder – complexity $O(n^2)$



Random numbers

- number selected from a known set of numbers in such a way that each number in the set has the same probability of occurrence.
- a number obtained by chance.
- one of a sequence of numbers considered appropriate for satisfying certain statistical tests or believed to be free from conditions that might bias the result of a calculation.

[Federal Standard 1037C]



Random numbers

- many uses of **random numbers** in cryptography:
 - nonces in authentication protocols to prevent replay;
 - session keys;
 - public key generation;
 - keystream for a one-time pad
- in all cases its critical that these values be:
 - statistically random, uniform distribution, independent
 - unpredictability of future values from previous values



Random numbers

- Used to generate cipher keys
- Seeds for RSA and ElGamal algorithm routines
- *Salt* values for “changing” encryption keys
- Initialization arrays for different encryption modes (ex. ECB)



Random numbers

- Are generated by “physical sources that generate random events”, events that can’t be predicted
- The **rand()**, **random()** functions from high-level programming languages are generating statistic random numbers (NOT for cryptography)

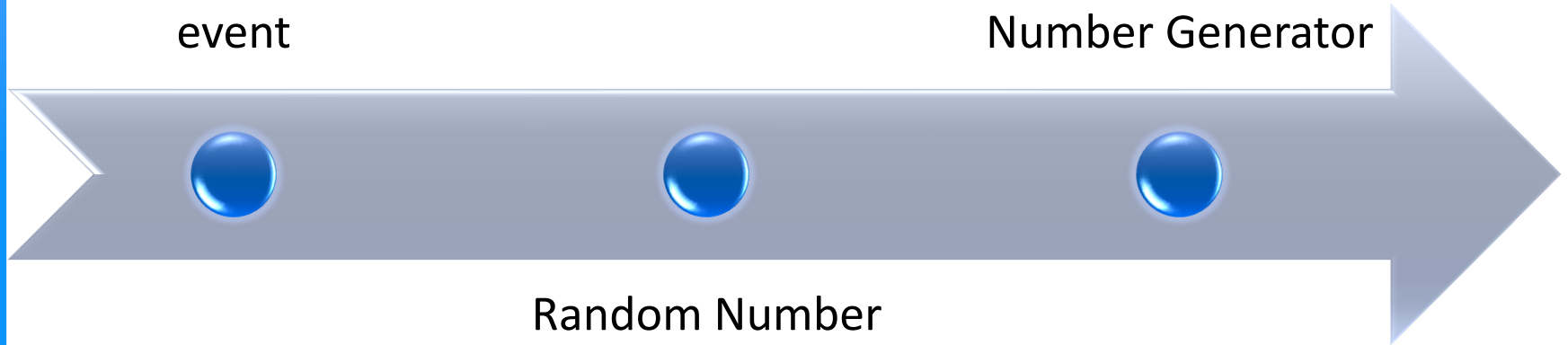


Random numbers

Random
hardware/software
event

Pseudo-Random
Number Generator

Random Number
Generator





Random Number Generators

DILBERT By SCOTT ADAMS



<https://www.random.org/analysis/>



Random numbers

- **Random Number Generator RNG** - produce a sequence of zero and one bits that may be combined into sub-sequences or blocks of random numbers
 - deterministic – based on an algorithm (**Pseudo Random Number Generator – PRNG**)
 - Nondeterministic (**True Random Number Generator – TRNG**)



Random numbers

- **Random Bit Generator RBG** - a device or algorithm that outputs a sequence of binary bits that appears to be statistically independent and unbiased. An RBG is either a Deterministic RNG (DRBG) or a Non-deterministic RBG (NRBG) [NIST Special Publication 800-90]
- **Pseudo-Random Number Generator PRNG:** a RNG that produces a sequence of values based on a seed and a current state. Given the same seed, it will always output the same sequence of values



- Source
 - Electronic noise of semiconductor devices
 - The least significant bits of an audio channel
 - Intervals between interrupts of hardware devices
 - Logging pressed keys in an interval or recording cursor position
- Processing
 - The event is “distilled” by a cryptographic hash function to increase the dependence between bits



- **Statistically random** - will pass various statistical tests of randomness [FIPS 800-22]
- **Cryptographically secure**— knowing generated random data, an attacker will not be able to predict the rest
- **Security strength of n bits** - the amount of work (of operations) required to break the system is 2^n



Random Numbers Generators - RNG

- the measure of randomness = *entropy*
- for a sequence of 16 bytes that are completely random (and *unbiased*) -> 128 bits of entropy
-> the *security strength* of the value is 128 bits
-> the amount of work required to break the security is 2^{128} operations
- what for 2 bytes ?



- RNG must generate sequence of values that are uniform distributed and independent (difficult to analyze)
- Key tests:
 - **bit count** [an even distribution is expected]
 - **word count**: counts the number of k-bit words (01010101.... – fails)
 - **gap space count**: the size of the gaps between the zero / one bits
 - **autocorrelation**: tries to determine if a subset of bits is related to another subset from the same string



Random Numbers Generators - RNG

autocorrelation:

- 1111 and 1110 -> correlated
- 1111 and 0000 -> correlated
- 1100 and 1010 -> perfectly uncorrelated
- $R(j) = \sum_n x_n \text{ XOR } x_{n-j}$
 - -> $n/2$ for uncorrelated streams
 - -> 0 or n for correlated streams



Random Numbers Generators

- RNG

NIST Guide to the Statistical Tests

- Frequency Test: Monobit
- Frequency Test: Block
- Runs Test
- Test for the Longest Runs of Ones in a Block
- Binary Matrix Rank Test
- Discrete Fourier Transform (Spectral Test)
- Non-Overlapping Template Matching Test
- Overlapping Template Matching Test
- Maurer's Universal Statistical Test
- Linear Complexity Test
- Serial Test
- Approximate Entropy Test
- Cumulative Sums Test
- Random Excursions Test
- Random Excursions Variant Test



Random Numbers Generators - RNG

- Hardware generators
 - <http://www.fdk.co.jp/cyber-e/pir/rpg100.htm>
 - <http://www.protego.se/products.htm>
- Environment based generators
 - <https://www.random.org/> - atmospheric noise



Pseudo-Random Number Generator - PRNG

One should not use a random method to generate random numbers. [Donald Knuth]

Anyone who uses software to produce random numbers is in a “state of sin”. [John von Neumann]



Pseudo-Random Number Generator - PRNG

- Can become the weakest link of the cryptographic system
- In cryptography you **DO NOT USE** rand functions from C, C++, Java programming languages because they are predictable (linear congruent functions based on an initial value - *seed*)
- Real world example- *How we Learned to Cheat in Online Poker: A Study in Software Security*, by Brad Arkin et. al.



Pseudo-Random Number Generator - PRNG

- rand function in C++ (MS VC C++ 7)

```
int __cdecl rand (void) {  
    return(((holdrand = holdrand * 214013L + 2  
        531011L) >> 16) & 0x7fff);  
}
```

- rand function in C (Kernighan & Ritchie C Standard)

```
unsigned long int next = 1;  
int rand(void) {  
    next = next * 1103515245 + 12345;  
    return (unsigned int) (next/65536) % 32768;  
}
```



Pseudo-Random Number Generator - PRNG

```
' VBScript
'prints 73 22 29 92 19 89 43 29 99 95.
Randomize 4269
For i = 0 to 9
  r = Int(100 * Rnd) + 1
WScript.echo(r)
Next
```

```
//C/C++
//prints 52 4 26 66 26 62 2 76 67 66.
#include <stdlib.h>
void main() {
  srand(12366);
  for (int i = 0; i < 10; i++) {
    int i = rand() % 100;
    printf("%d ", i);
  }
}
```

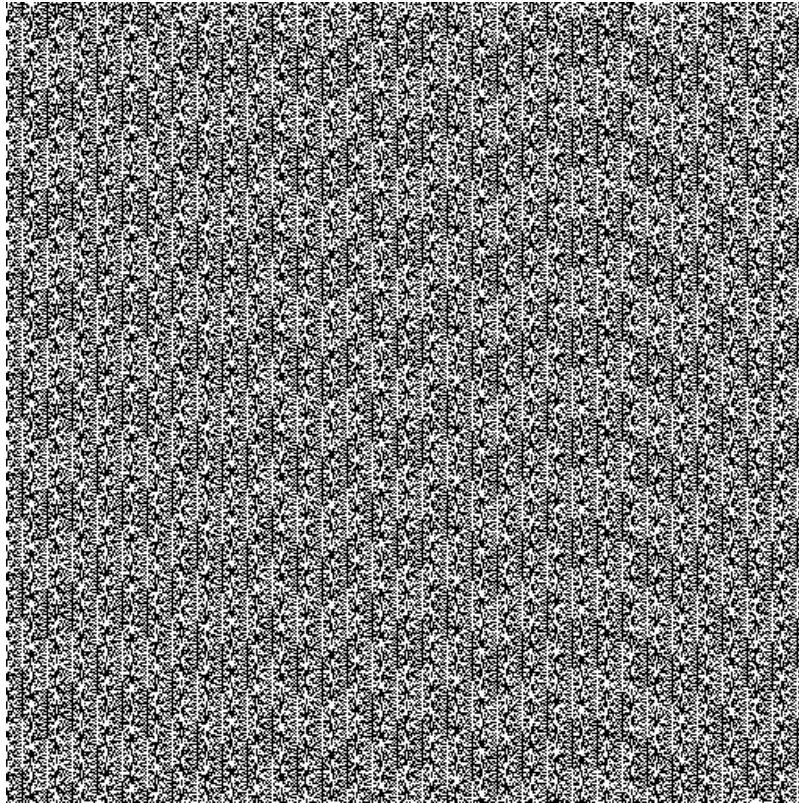


Pseudo-Random Number Generator - PRNG

```
// C#  
// prints 39 89 31 94 33 94 80 52 64 31  
using System;  
class RandTest {  
    static void Main() {  
        Random rnd = new Random(1234);  
        for (int i = 0; i < 10; i++) {  
            Console.WriteLine(rnd.Next(100));  
        }  
    }  
}
```



Pseudo-Random Number Generator - PRNG



- PHP rand() on Microsoft Windows
- <https://www.random.org/analysis/>



Pseudo-Random Number Generator - PRNG

- Uses a large number of bits (seed value) that contains the random event
- **Yarrow** – most known PRNG
 - Defined by Bruce Schneier and John Kelsey in Counterpane Labs
 - <http://www.schneier.com/yarrow.html>



- www.ism.ase.ro



Pseudo-Random Number Generator - PRNG

Characteristics:

- Simple and fast
- Must generate variable length numbers that does not repeat (maximizing the period is better because it is impossible to make it going to infinity)
- Must generate independent values
- Must generate uniform distributed numbers



Pseudo-Random Number Generator - PRNG

m – module, with $m > 0$
 a – multiplier, with $0 \leq a < m$
 c – increment, with $0 \leq c < m$
 X_0 – initial value, with $0 \leq X_0 < m$

Magic numbers

$$X_{n+1} = (aX_n + c) \bmod m$$

Recurrence relation

Linear congruential generator

$\{X_n\}$

Maximal period generator



Pseudo-Random Number Generator - PRNG

- Linear congruence method

m	a
2^{31}	65539
$2^{31}-1$	16807
$2^{31}-249$	40692
$2^{31}-1$	48271
$2^{31}-1$	62089911
2^{32}	69069
2^{48}	31167285
2^{64}	6364136223846793005

[Knuth]



Pseudo-Random Number Generator - PRNG

- Linear congruence method

Overflow at	a	c	m
2^{31}	8121	28411	134456
2^{31}	4561	51349	243000
2^{31}	7141	54773	259200
2^{32}	9301	49297	233280
2^{32}	4096	150889	714025
2^{33}	2416	374441	1771875
2^{34}	17221	107839	510300

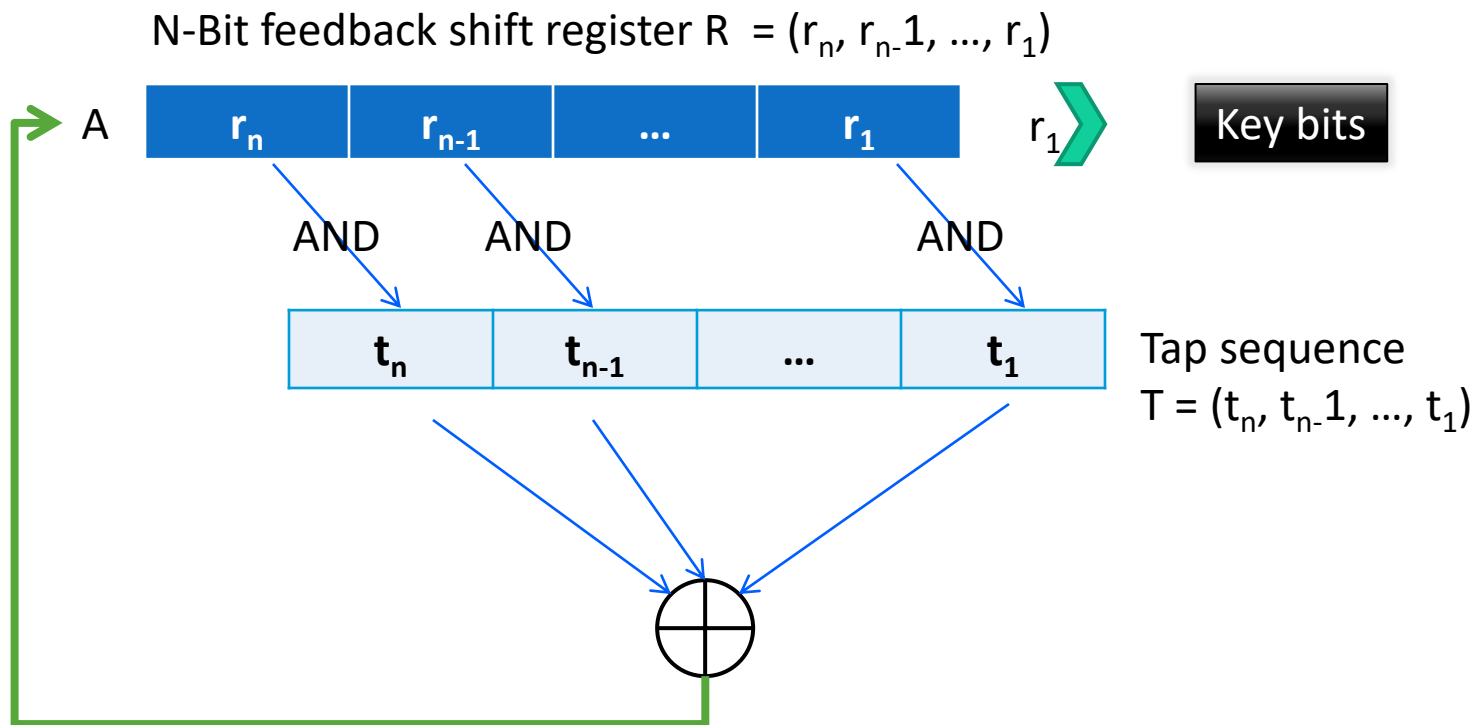
[3]



- NOT used anymore in cryptography
- broken in 1977 by Jim Reeds
- combining linear congruential generators gives better results (ex. $2^{31} - 85$ with $2^{31} - 249$)



Pseudo-Random Number Generator - PRNG



$$r'_n = t_1 * r_1 + \dots + t_n * r_n$$

Linear feedback shift register
generator - LFSR



Pseudo-Random Number Generator - PRNG

LFSR characteristics:

- simple feedback sequence
- an n -bit LFSR can have $2^n - 1$ internal states (depends on the tap sequence – maximal period LFSR) – the polynomial formed by tap sequence plus constant 1 must be a primitive polynomial mod 2 (ex. $x^{10} + x^3 + 1$)
- stream ciphers have been built based on LFSR (ex. A5 for GSM) because they can be easily implemented in hardware



Pseudo-Random Number Generator - PRNG

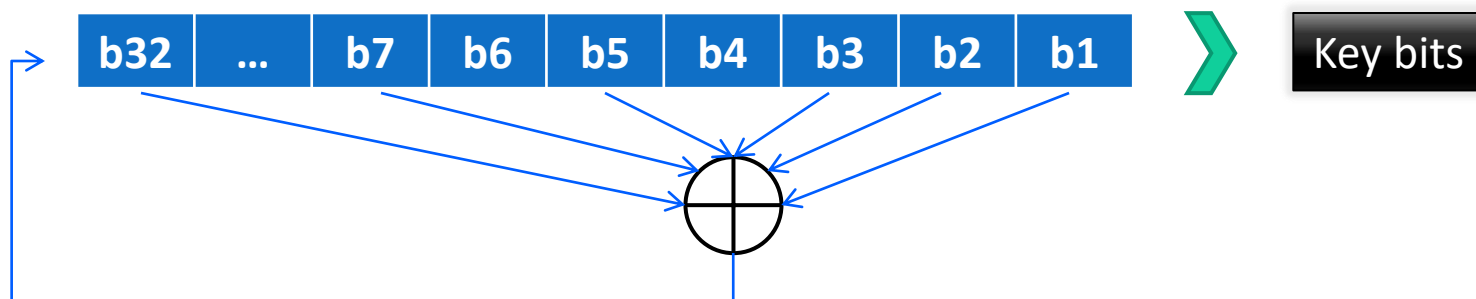
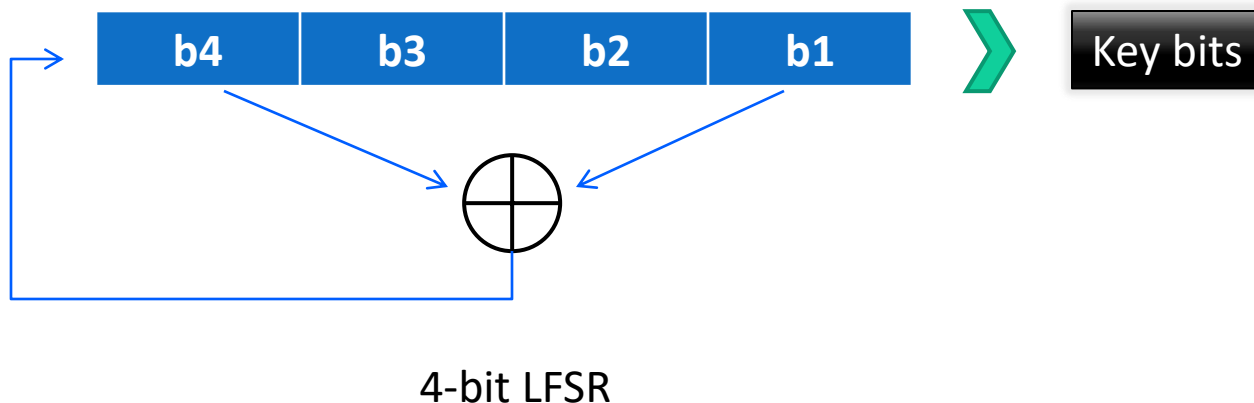
LFSR characteristics:

- competent pseudo-random-sequence generators
- *Berlekamp-Massey* algorithm can determine the feedback function from only 2^n output bits



Pseudo-Random Number Generator - PRNG

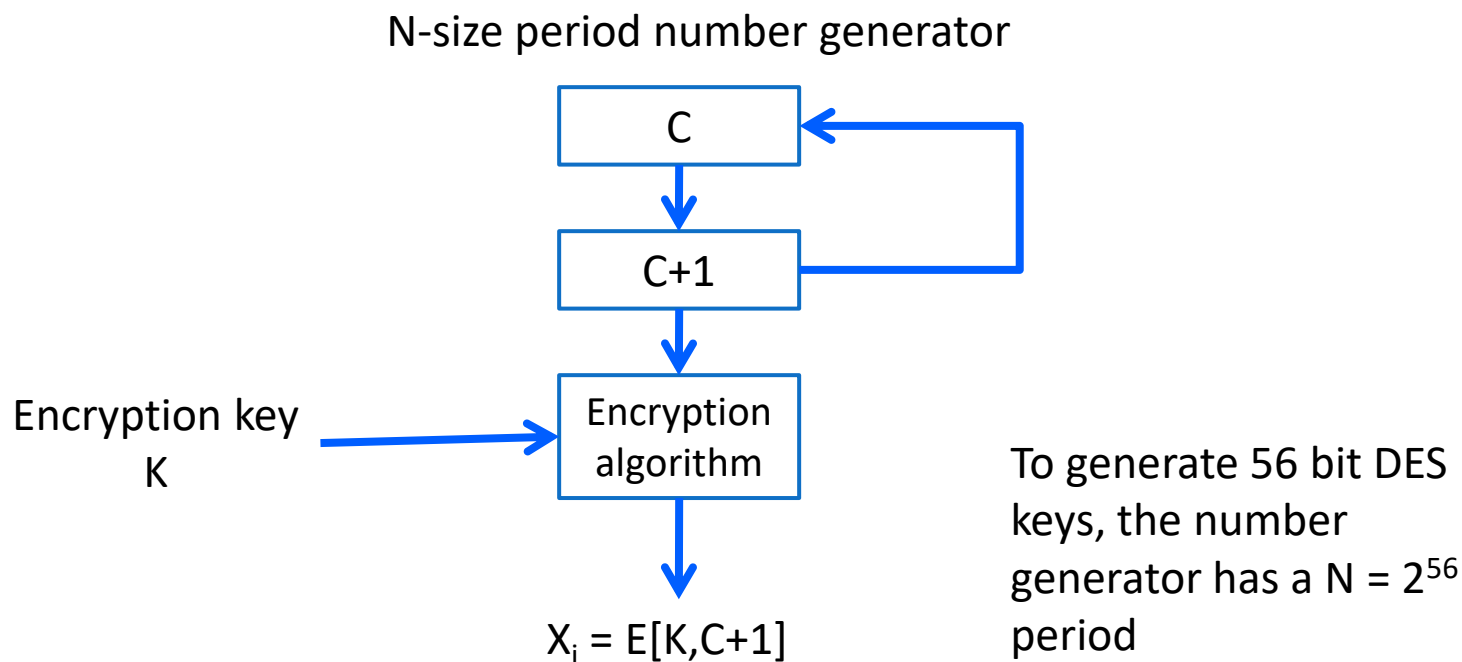
- Examples of LFSR





Pseudo-Random Number Generator - PRNG

- Some PRNGs use cryptographic methods – repeated encryption of an input





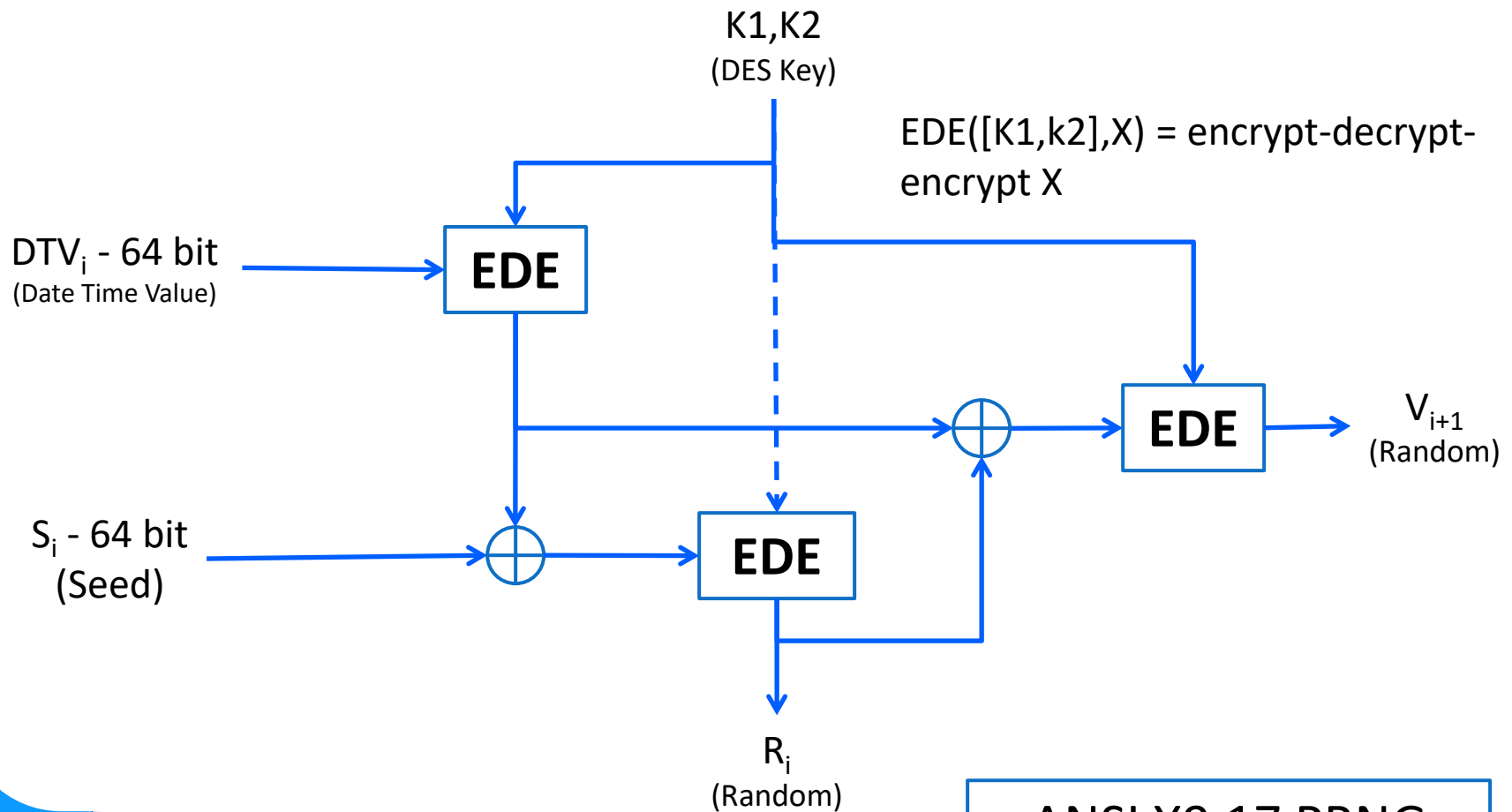
Pseudo-Random Number Generator - PRNG

ANSI X9.17

- One of the safest PRNG that uses encryption (triple DES - EDE)
- It starts with 2 initial pseudo-random values: 64 bit value of current date and time, random generated 64 bit seed value
- Uses 3 encryption modules that use triple-DES 56 bit key
- It generates $2 * 64$ bit values: a pseudo-random number and a seed value



Pseudo-Random Number Generator - PRNG



ANSI X9.17 PRNG



Pseudo-Random Number Generator - PRNG

Blum Blum Shub Generator – BBS

- One of the most used generators
- Highly secure – it uses the factorization problem
- Generates pseudo-random values of any length
- cryptographically secure pseudorandom bit generator (CSPRBG) – it pass the *next-bit* test



Pseudo-Random Number Generator - PRNG

Blum Blum Shub Generator – BBS

- 2 prime and large numbers are generated, p and q such that $p \equiv q \equiv 3 \pmod{4}$
- It is computed $n = p * q$
- It is selected a random seed number s that is relatively prime to n ($\gcd(s, n) = 1$)
- Each bit is determined by

$$X_0 = s^2 \pmod{n}$$

for $i = 1$ to n

$$X_i = (X_{i-1})^2 \pmod{n}$$

$$B_i = X_i \pmod{2} \quad \text{-- random bit}$$



Pseudo-Random Number Generator - PRNG

Blum Blum Shub Generator – BBS

- At each iteration, the least significant bit is selected to generate the random value

$n = 192649$

$p = 383$

$q = 503$

$s = 101355$

i	X_i	B_i
0	20749	
1	143135	1
2	177671	1
3	97048	0
4	89992	0
5	174051	1
6	80649	1
7	45663	1
8	69442	0
9	186894	0
10	177046	0

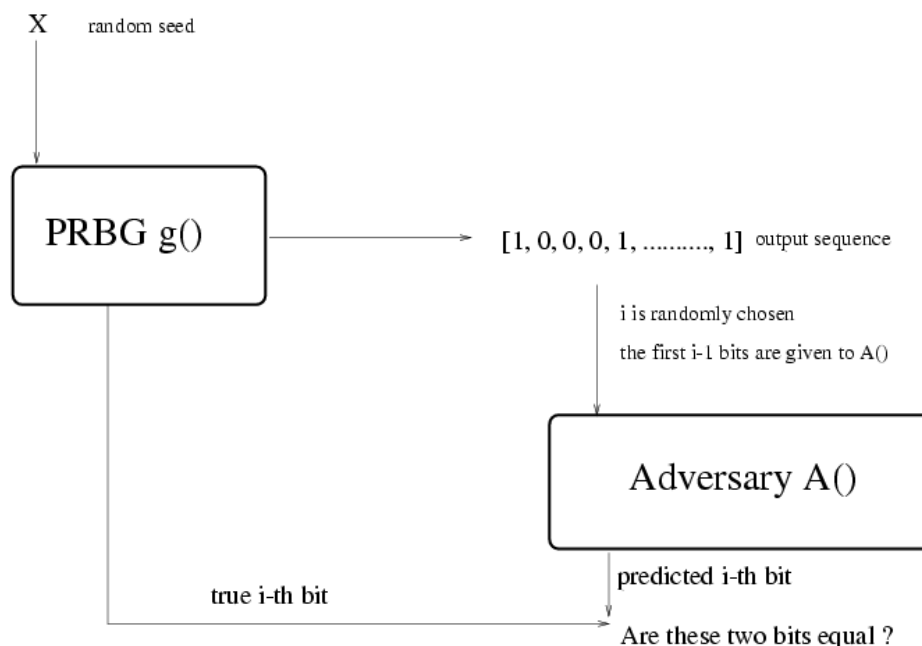
i	X_i	B_i
11	137922	0
12	123175	1
13	8630	0
14	114386	0
15	14863	1
16	133015	1
17	106065	1
18	45870	0
19	137171	1
20	48060	0



Pseudo-Random Number Generator - PRNG

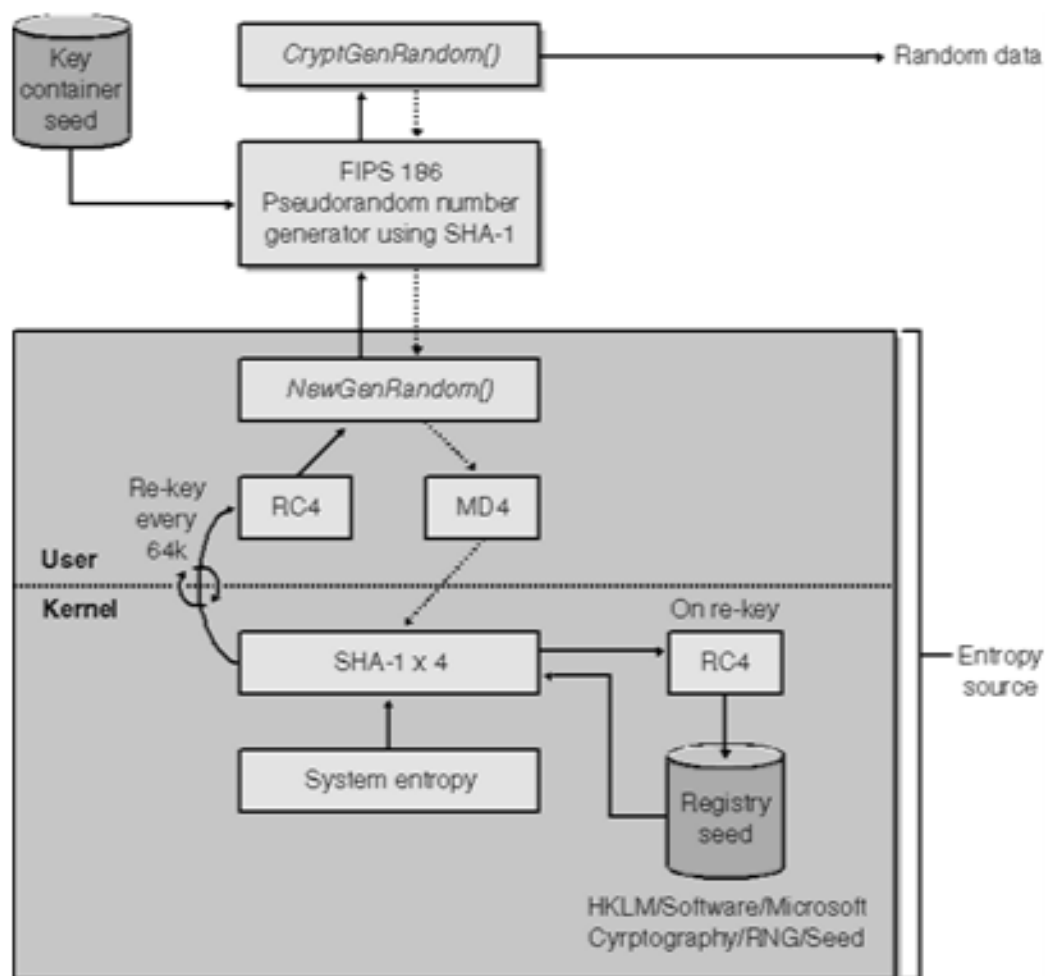
Blum Blum Shub Generator – BBS

- Statistical experiment for the next-bit test





Pseudo-Random Number Generator - PRNG



[Microsoft]



Protocols

- a series of steps, involving 2 or more parties, designed to accomplish a task [3]
- types of protocols:
 - **arbitrated** – with a trusted third party
 - **adjudicated** – 2 lower-level subprotocols (one nonarbitrated and one arbitrated)
 - **self-enforcing**



Protocols

- Secret key establishment
- Elections
- Auctions
- Secure multi-party computation



Conditions defined by Alice and Bob:

- There is a “**magic**” function f
 - it is easy to determine $f(x)$, but is impossible to determine x , knowing $f(x)$
 - it is impossible to find 2 values, x and y with $x \neq y$ such that $f(x) = f(y)$
- It is defined the correlation head = x if even,
tail = x if odd



- Alice chose a large random number x and tells Bob the value of $f(x)$
- Bob says if x is odd (tail) or even (head) - he flips the coin and tells the result
- Alice tells Bob the value of x
- Bob computes $f(x)$ and checks if he has won or lost.



Hash functions

- a function $H(M)$ used to compute a fixed value h (hash) unique for the variable-length message
- has a role in authenticating the message content
- **DOES NOT** hide the content
- used in digital signature procedure



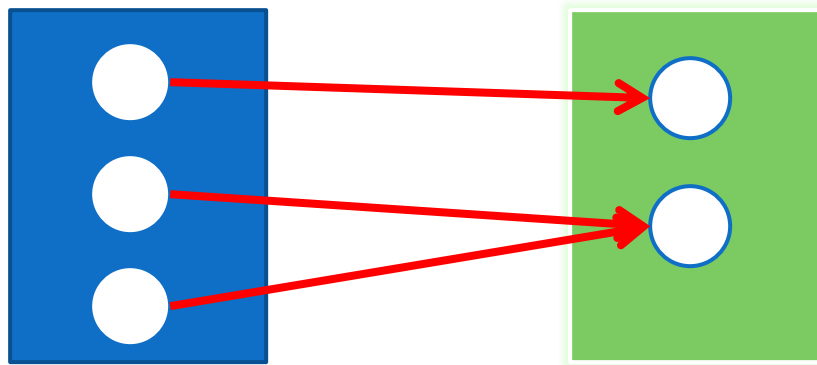
- for M (variable-length plaintext), it is easy to compute h (fixed length)
- for h , is IMPOSSIBLE to determine M
- for M , it is very difficult to find M' such as $H(M) = h(M')$
- **collision-free** = the difficulty in finding M'
- modifying a single bit of M , the h value is totally different



Hash functions

Collision-free:

- A collision for the hash function $h : D \rightarrow \{0, 1\}^n$ it is the pair $x_1, x_2 \in D$ such that $h(x_1) = h(x_2)$ but with $x_1 \neq x_2$.
- Because $|D| > 2^n$



Collisions exist,
but are difficult
to find them



Hash functions

Function	Hash length
SHA-1 (Secure Hash Algorithm or SHS – Secure Hash Standard)	160 bit
SHA-256	256 bit
RIPEMD-160	160 bit
MD5 (Message Digest Algorithm)	128 bit
Tiger	128 bit
MD2, MD4	128 bit



Sponge function

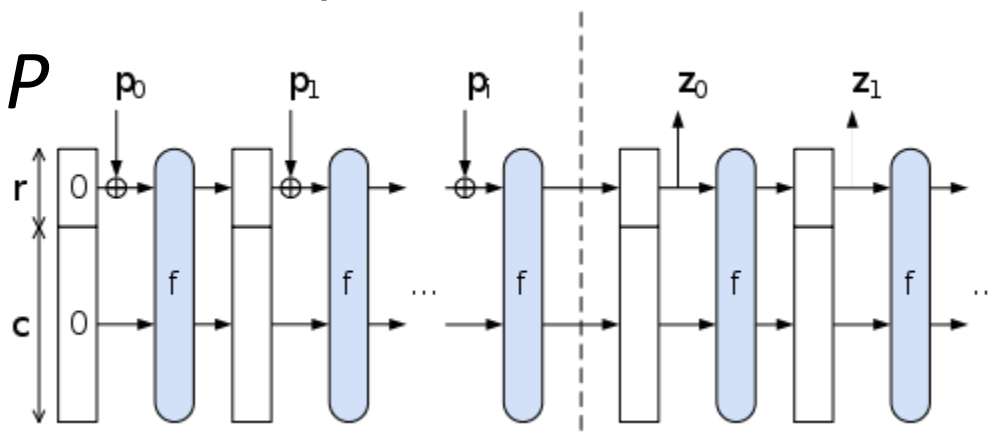
- a class of algorithms with finite internal state that take an input bit stream of any length and produce an output bit stream of any desired length



Sponge function

A sponge function is built from three components:[\[2\]](#)

- a state memory, S , containing b bits,
- a function, f , of fixed length that permutes or transforms the state memory
- a padding function P



http://en.wikipedia.org/wiki/Sponge_function



Sponge function

- The state memory is divided into two sections, R of size r bits and C of size $c = b - r$ bits. The parameter r is called the *bitrate* and c is the *capacity*.
- The padding function appends enough bits to the input string so that the length of the padded input is a whole multiple of the bitrate, r . The padded input can thus be broken into r -bit blocks.



- The state S is initialized to zero
- The input string is padded
- R is XORed with the first r -bit block of padded input
- S is replaced by $f(S)$
- R is XORed with the next r -bit block of padded input (if any)
- S is replaced by $f(S)$

160



Birthday paradox

- What is the probability to meet a random person on the street that has the same **birthday** (day not year) as you ?
 - assuming that all days of the year have the same probability, the chances of another person sharing your birthday is $1/365$ which is 0.27%. Is low
- How many random people should you have in a room so the probability will be around 50% ?



Birthday paradox

- For N different possibilities of an event to happen, then you need square root of N random items for them to have a 50% chance of a collision
- For 365 different possibilities of birthdays, you need $\text{Sqrt}(365) = \sim 23$, randomly chosen people, for 50% chance of two people sharing same birthday
- there are needed 253 persons in a room to have > 50% probability that one of them shares your birthday



Birthday paradox

- Allows “*birthday attack*” on a hash function to find collisions
- In conclusion, a collision can be found in almost $\text{Sqrt}(2^n) \approx 2^{n/2}$ tries.



Birthday paradox

- the “*birthday attack*” allows attackers to find faster (with a high probability of $\sim 50\%$) two random messages, M and M' , such that $H(M) = H(M')$
- as a result, hashes have half of their digest size in strength: MD5 needs 2^{64} effort to find collisions



- Proposed by Ronald Rivest from MIT and developed by RSA Data Security company
- generates a 128 bit hash
- defined based on MD4
- has 5 important stages
- In 2005 researches have announced that can find collisions for the hash function; now, the collisions can be generated in couple of hours



Properties of Hash functions

- **Deterministic** – same input = same output
- **Fast computation**
- **Pre-Image Resistance**
- Small changes in the input changes the entire hash value
- **Collision Resistant**
- **Puzzle Friendly**



Properties of Hash functions

Pre-Image Resistance

- given $H(A)$ it is infeasible to determine A , where A is the input and $H(A)$ is the output hash
- Is NOT impossible to determine A
 - by brute-force you can try all possible inputs
 - If the input range is small enough you can do it



Properties of Hash functions

Collision Resistant:

- for two different inputs A and B where $H(A)$ and $H(B)$ are their respective hashes, it is infeasible for $H(A)$ to be equal to $H(B)$
- Is NOT impossible
 - MD 5: collision resistance was broken after $\sim 2^{21}$ hashes.
 - SHA 1: collision resistance broke after $\sim 2^{61}$ hashes.



Properties of Hash functions

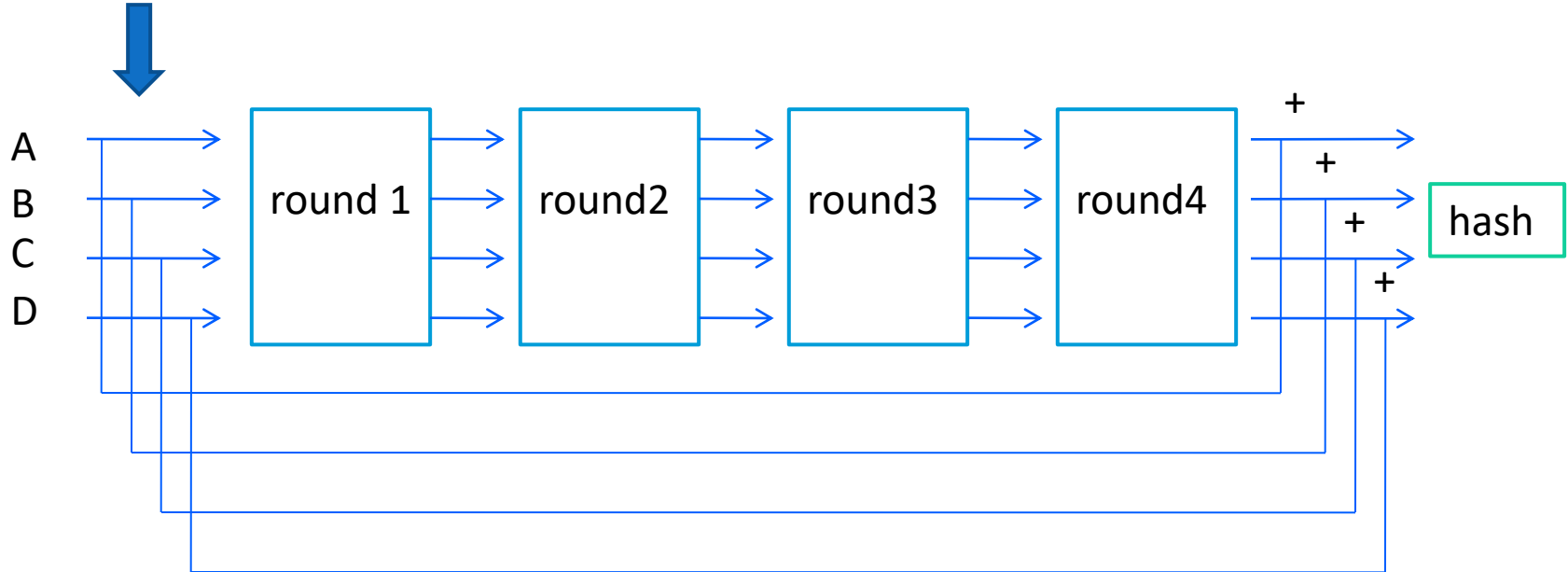
Puzzle Friendly

- For every output “Y”, if k is chosen from a distribution with high min-entropy it is infeasible to find an input x such that $H(k|x) = Y$
- high min-entropy - the distribution is hugely distributed so choosing a random value has a negligible probability (ex $[1..2^{256}]$)
- low min-entropy distribution – $[1..5]$



Hash functions– MD5

$M_1, M_2, \dots, M_n$ message



General scheme of MD5 algorithm



- The message M is extended to a length (measured in bits), L, that is congruent with 448 mod 512 ($L - 448 = K * 512$, with k – integer value)
- the padding is made with “1” followed by many “0” bits



Hash functions– MD5

Stage 2:

- there are added 64 bits that represent the length of the initial message
- the message is split in n blocks of 512 bits, $M_1, M_2, \dots, M_n$
- each block M_i has 16 words of 32 bits
- text dimension is now $= n * 16 * 32$;



- to generate the hash it is used a register MD that is 128 long (4 words of 32 bits each – A,B,C,D)

- 0



- Each block M_j (16 words* 32 bits) is processed in 4 rounds with the functions FF, GG, HH, II
- $MD_j = MD_{j-1} + II(M_j, HH(M_j, GG(M_j, FF(M_j, MD_{j-1}))))$
- each round has 16 steps



Hash functions– MD5

Stage 4:

- in each step of the round (it has 16):

$$A = B + ((A + F(B, C, D) + M_{jm} + t_i) \lll k)$$

- t_i – constant value, step dependent, equal with the first 32 bits of the value $abs(sin(j+1))$, $0 \leq j \leq 63$
- M_{jm} – the m^{th} 32 bit word from the M_j block
- F – nonlinear function that is modified in each round
- $\lll k$ – shifts to the left the word with k positions





Hash functions– MD5

Stage 4:

- the forms of the F function:
 - round 1 (FF): $F(B,C,D) = (B \text{ and } C) \text{ or } (B' \text{ and } D)$
 - round 2 (GG): $F(B,C,D) = (B \text{ and } D) \text{ or } (C \text{ and } D')$
 - round 3 (HH): $F(B,C,D) = (B \text{ xor } C \text{ xor } D)$
 - round 4 (II): $F(B,C,D) = C \text{ xor } (B \text{ or } D')$
 - B' represents the complement 2 value

Stage 5:

- the MD register value is the hash value (128 biti)



Hash functions - SHA-1

- Algorithm proposed by NIST
- SHA-1 Collision Search Graz - <http://www.iaik.tugraz.at>
- Based on this version, the second generation has been developed: SHA-256, SHA-512
- There is a NIST contest for a new solution that will become SHA – 3(will end in 2010) - http://ehash.iaik.tugraz.at/wiki/The_SHA-3_Zoo



SHA-1 and SHA-2

Algorithm and variant		Output size (bits)	Internal state size (bits)	Block size (bits)	Max message size (bits)	Rounds	Security (bits)	Example Performance (MiB/s) ^[26]
MD5 (as reference)		128	128 (4×32)	512	$2^{64} - 1$	64	<64 (collisions found)	335
SHA-0		160	160 (5×32)	512	$2^{64} - 1$	80	<80 (collisions found)	-
SHA-1		160	160 (5×32)	512	$2^{64} - 1$	80	<80 (theoretical attack ^[27] in 2^{61})	192
SHA-2	SHA-224 SHA-256	224 256	256 (8×32)	512	$2^{64} - 1$	64	112 128	139
	SHA-384 SHA-512 SHA-512/224 SHA-512/256	384 512 224 256	512 (8×64)	1024	$2^{128} - 1$	80	192 256 112 128	154

<http://en.wikipedia.org/wiki/SHA-3>



SHA-3

- a subset of the cryptographic primitive family **Keccak**
- On October 2, 2012, Keccak was selected as the winner of the [NIST hash function competition](#)
- is not meant to replace [SHA-2](#), as no significant attack on SHA-2 has been demonstrated
- may make it useful for so-called “embedded” or smart devices
- SHA-3 uses the [sponge construction](#)



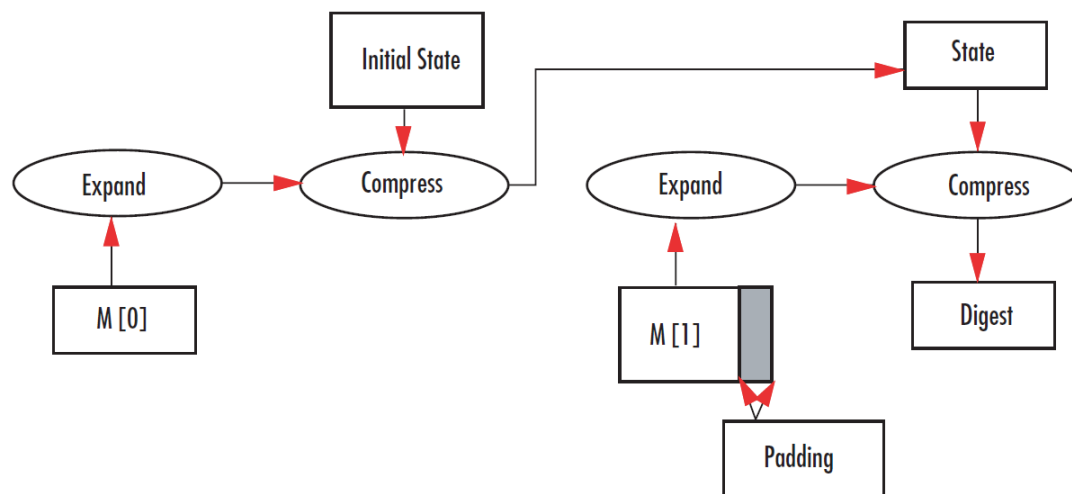
SHA-3

Algorithm and variant		Output size (bits)	Internal state size (bits)	Block size (bits)	Max message size (bits)	Rounds	Operations	Security (bits)	Example Performance (MiB/s) ^[26]
SHA-3	<i>SHA3-224</i>	224	1600 (5×5×64)	1152	∞	24	and, xor, not, rot	112	
	<i>SHA3-256</i>	256		1088				128	
	<i>SHA3-384</i>	384		832				192	
	<i>SHA3-512</i>	512		576				256	
	<i>SHAKE128</i>	<i>d</i> (arbitrary)		1344				min(<i>d</i> /2, 128)	
	<i>SHAKE256</i>	<i>d</i> (arbitrary)		1088				min(<i>d</i> /2, 256)	

<http://en.wikipedia.org/wiki/SHA-3>



Hash functions - SHA



Hash of a Two-Block Message in SHA hash functions [5]



- EuroCrypt 2005 - "How to break MD5 and other hash functions", Xiaoyun Wang et. Al.
- two different Win32 executable with different functionality but equal MD5 hash values, <http://www.win.tue.nl/hashclash/SoftIntCodeSign/>
- Generates 2 random messages with only 3 different bits that have same hash, <http://www.cs.colorado.edu/~jrblack/>



Hash functions

Role:

- Allow storing passwords and hiding its value
- Use by PRNG to generate random numbers (hide the initial seed value)
- Used to validate files integrity or verify files versions (ex Git is using sha-256)
- Used to detect modifications – intrusion detection software
- Extend/Reduce secret values to a predefined size



Hash functions

Are NOT used to:

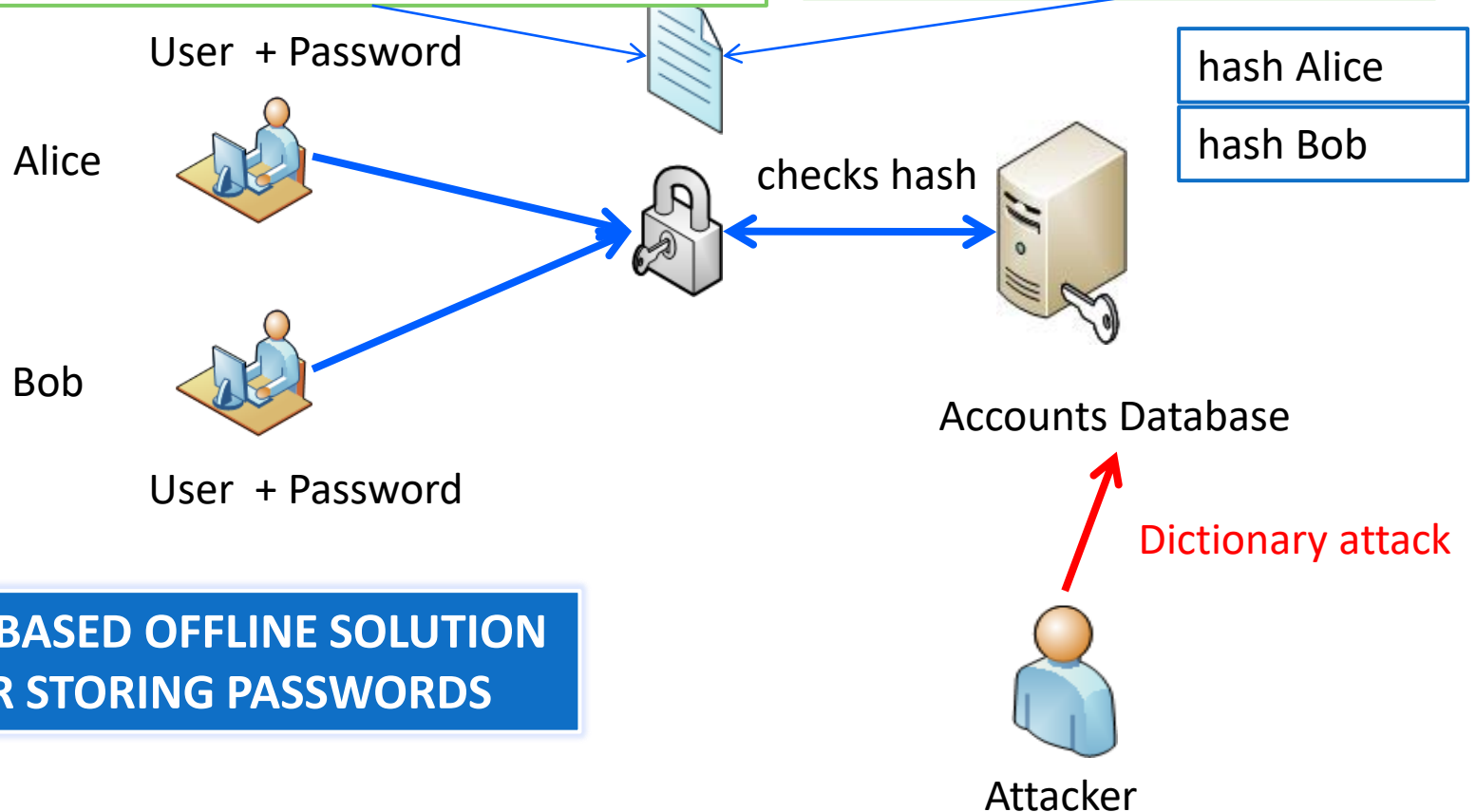
- store passwords without applying a salt value
- replace Message Authentication Code
- generate cryptographic algorithms
- double the size of the message digest by concatenating two message digests of slightly different messages
- concatenate two message digests from different hashes



Hash functions

Use a salt:
 $\text{Hash User} = \text{hash}(\text{Salt User} + \text{Password})$

Rehash:
 $\text{Hash User} = \text{hash}(\dots\text{hash}(\text{User}))$





- Predefined or unique values, added to user passwords before they are hashed
- Are NOT part of the secret
- Intended to increase attacker time needed to dictionary attack a user account
 - The Rainbow table must be regenerated
 - Add computational effort – adding the salt to the password may require some operations that will increase processing time



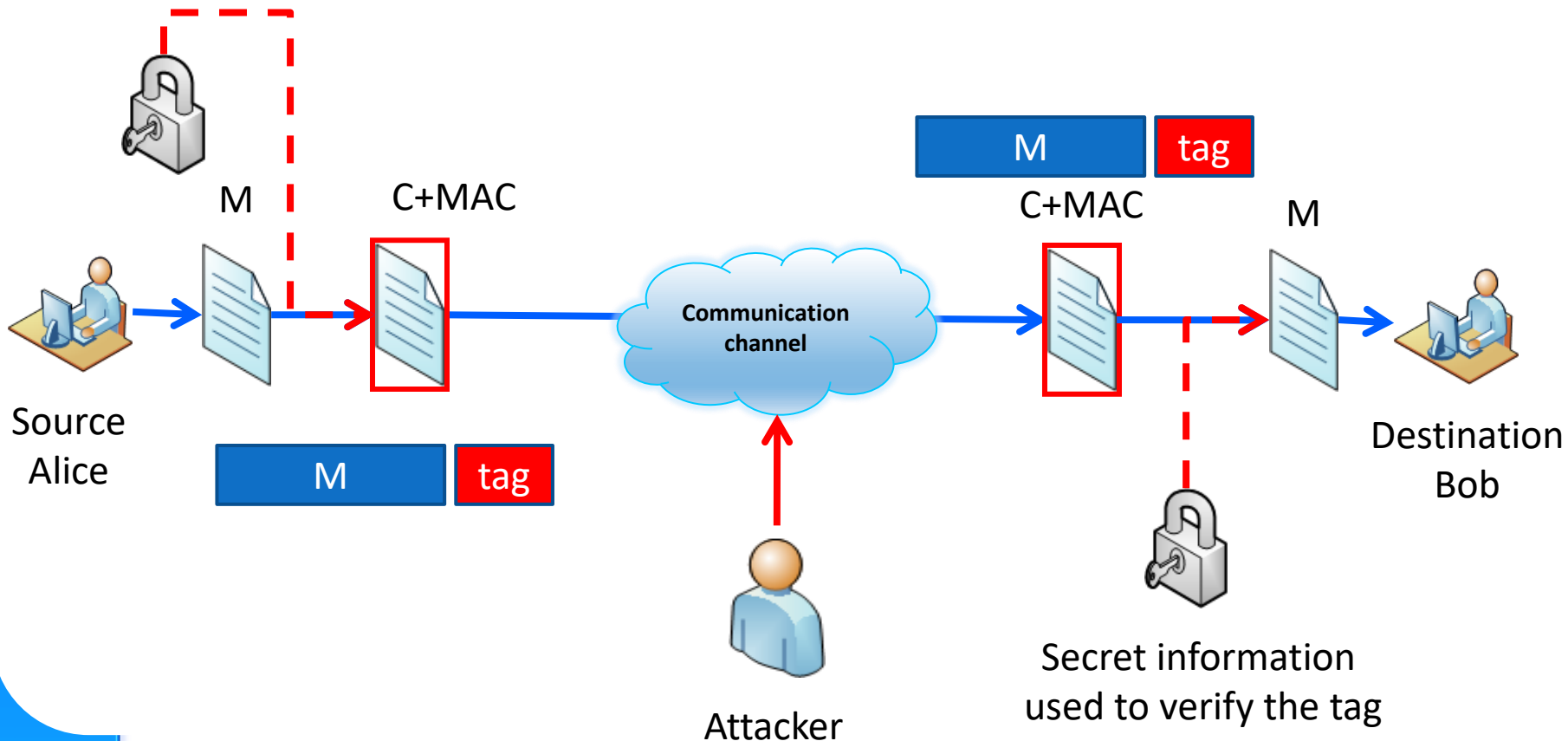
Message Authentication Codes MAC

- is a key-dependent one-way hash function
- a one-way hash function generates a MAC if the hash is encrypted with a symmetric algorithm
- used **to authenticate** files between users
- used to provide **data integrity**
- **NOT used** for securing data



Message Authentication Codes MAC

Secret information
used to generate the MAC tag





- The MAC tag is generated by a function $S(K, M)$ based on the secret key and the message hash
- Bob verifies the integrity using a verification function $V(K, M, \text{tag})$ by re-computing the tag
- Without the use of the secret key, K , the attacker can temper the message and re-compute the tag
- The tag is a fixed length value, long enough in order to avoid guessing it



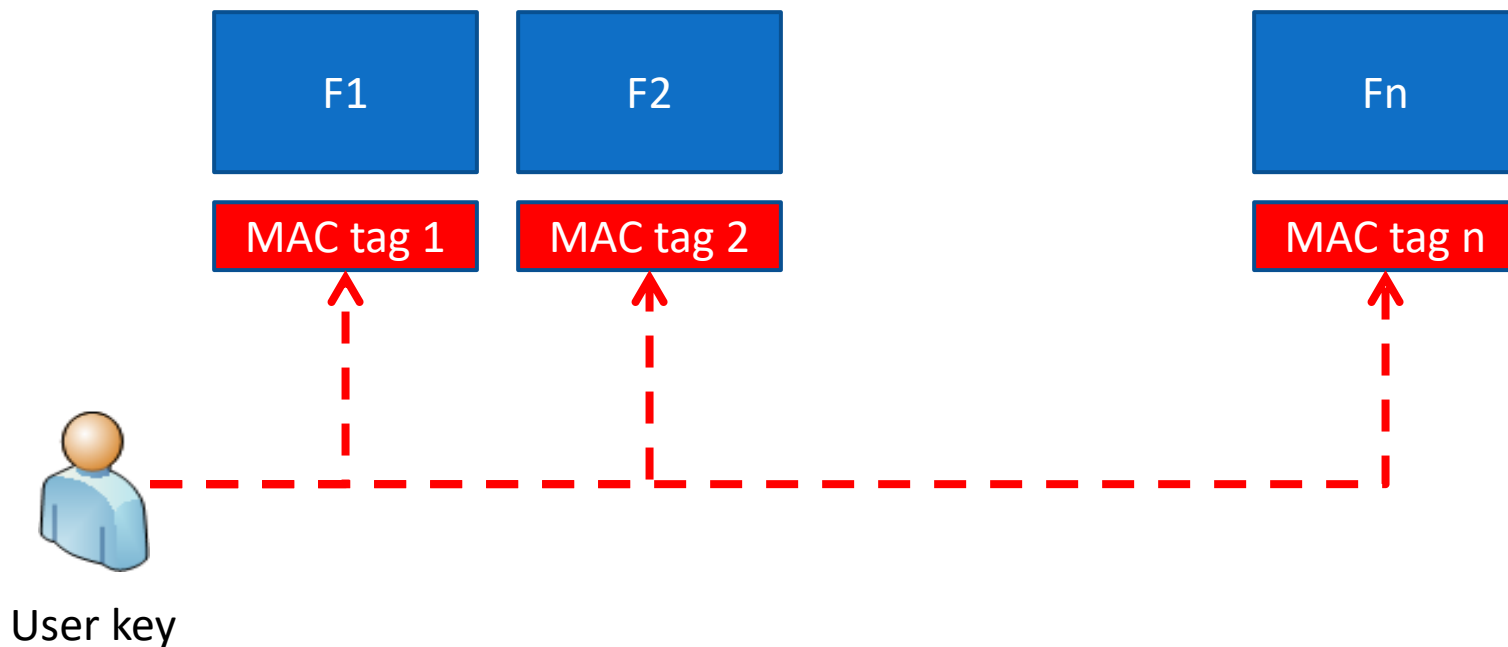
- Chosen message attack – the attacker gets the tag for some messages
- Existential forgery – to generate some new valid message, tag pairs

- Attacker cannot generate a valid tag
- Given (M1,tag) attacker CANNOT generate (M,tag') for tag $\neq$ tag'



Message Authentication Codes MAC

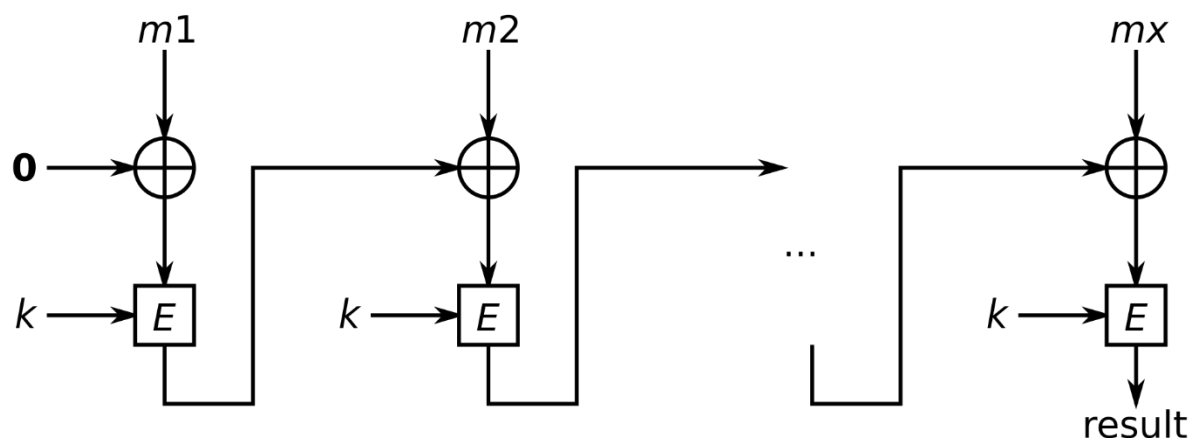
Protecting system files:





CBC-MAC

- CBC-MAC (banking – ANSI X9.9, X9.19, FIPS 186-3)



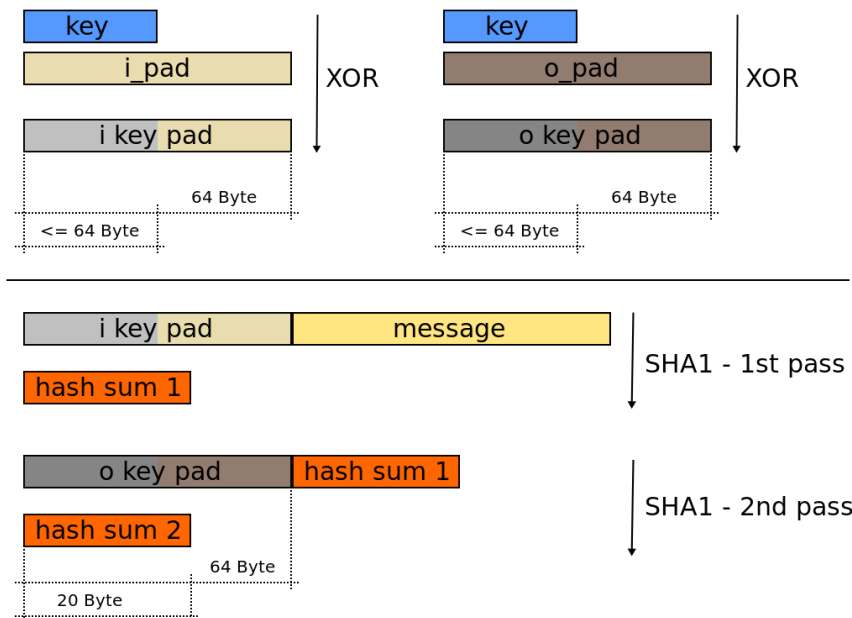
<https://en.wikipedia.org/wiki/CBC-MAC>



- www.ism.ase.ro



HMAC – RFC 2104



http://en.wikipedia.org/wiki/Hash-based_message_authentication_code

where

H is a cryptographic hash function,

K is a secret key **padded** to the right with extra zeros to the input block size of the hash function, or the hash of the original key if it's longer than that block size,

m is the message to be authenticated,

| denotes **concatenation**, $\oplus$ denotes **exclusive or** (XOR),

opad is the outer padding (0x5c5c5c...5c5c, one-block-long **hexadecimal** constant),

and **ipad** is the inner padding (0x363636...3636, one-block-long **hexadecimal** constant).



- $MAC = H(key \parallel message)$ - with most hash functions, it is easy to append data to the message without knowing the key and obtain another valid MAC (length-extension attack).
- $MAC = H(message \parallel key)$, an attacker who can find a collision in the (unkeyed) hash function has a collision in the MAC (as two messages m_1 and m_2 yielding the same hash will provide the same start condition to the hash function before the appended key is hashed, hence the final hash will be the same).
- $MAC = H(key \parallel message \parallel key)$ is better, but not

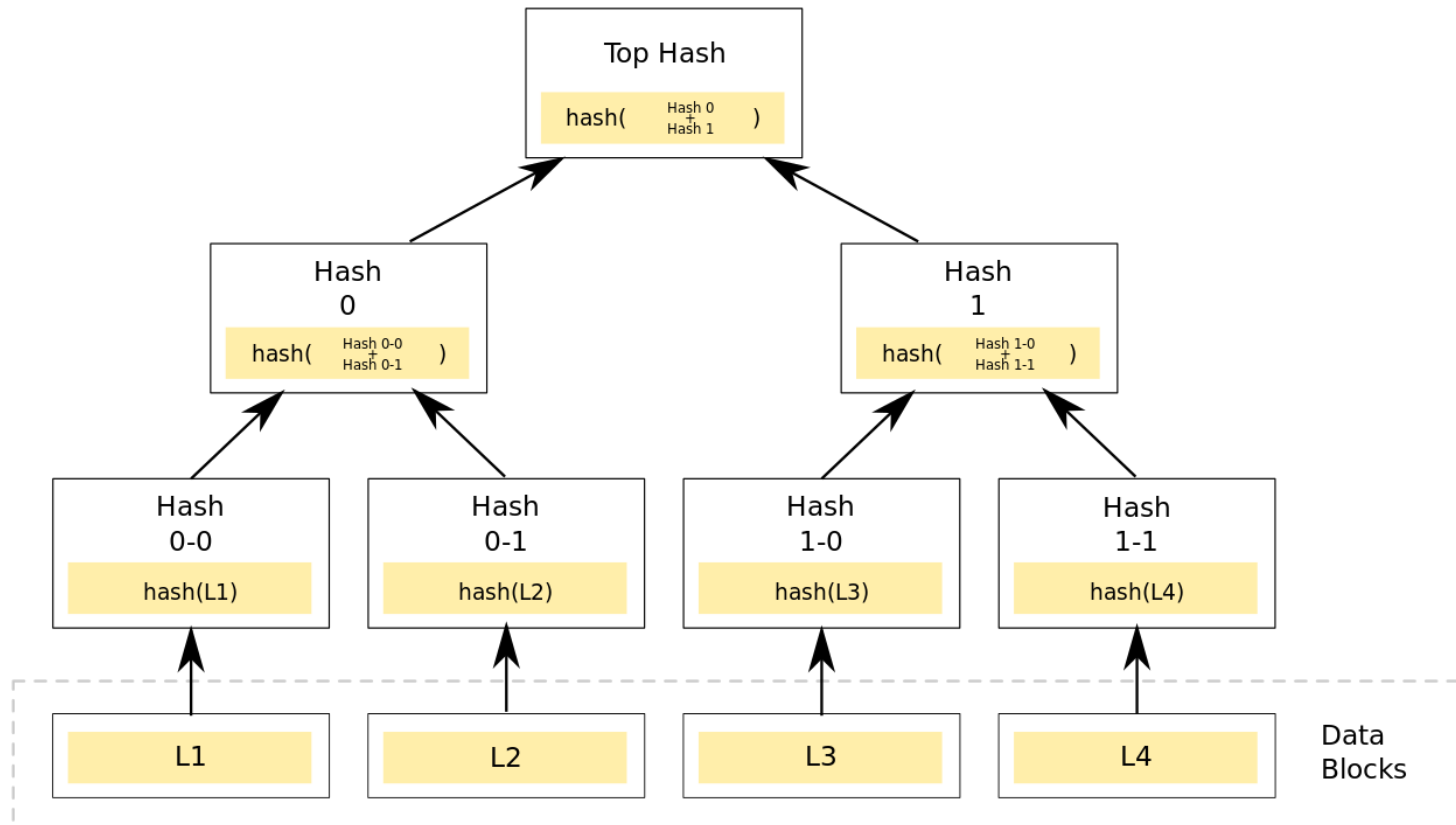


Merkle Trees

- *a hash tree or Merkle tree is a tree in which every leaf node is labelled with the cryptographic hash of a data block, and every non-leaf node is labelled with the cryptographic hash of the labels of its child nodes [https://en.wikipedia.org/wiki/Merkle_tree]*



Merkle Trees



https://en.wikipedia.org/wiki/Merkle_tree



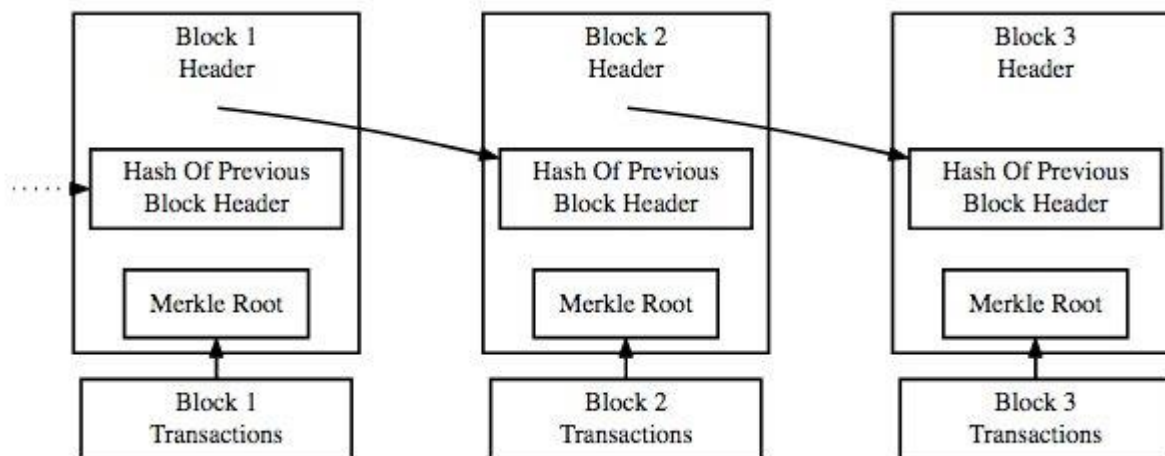
Merkle Trees

- Used to verify integrity of data
- NOT used for searching
- Are efficient to check data integrity because you don't need the entire tree to verify a block
- Used
 - P2P networks
 - blockchains



Hash for Blockchain

- Bitcoin uses SHA 256
- Ethereum uses Keccak-256



Block Chain Overview,

https://developer.bitcoin.org/devguide/block_chain.html



- Protect the chain for future changes: all the transactions and blocks are hashed and chained together
- Provides **proof-of-work**
- Used to add computational effort in the mining effort and to control the pace of generating new blocks



Blockchain Block Header

version	02000000
previous block hash (reversed)	17975b97c18ed1f7e255adf297599b55 330edab87803c8170100000000000000
Merkle root (reversed)	8a97295a2747b4f1a0b3948df3990344 c0e19fa6b2b92b3a19c8e6badc141787
timestamp	358b0553
bits	535f0119
nonce	48750833
transaction count	63
coinbase transaction	
transaction	
...	

Block hash

0000000000000000
e067a478024addfe
cdc93628978aa52d
91fabd4292982a50

<http://www.righto.com/2014/02/bitcoin-mining-hard-way-algorithms.html>



- Is impossible to modify transactions included in any block without modifying all subsequent blocks
- The *proof of work* takes advantage of the apparently random nature of cryptographic hashes: hashing random data produces an apparently random hash value and is very difficult to predict the output



Hash for Blockchain

1. The hash of the contents of the new block is computed
2. A nonce (random string) is appended to the hash
3. The new value is hashed
4. The new hash is compared to the difficulty level (a 64-character string value = 256 bits) and checked whether is less than that or not:
 1. If not, then the nonce is changed and the process repeats again
 2. If yes, then the block is added to the chain and the public ledger is updated



- www.ism.ase.ro



Hash for Blockchain

- [https://en.bitcoin.it/wiki/Block hashing algorithm](https://en.bitcoin.it/wiki/Block_hashing_algorithm)
- <https://en.bitcoin.it/wiki/Difficulty>
- <http://www.righto.com/2014/02/bitcoin-mining-hard-way-algorithms.html>



Symmetric cryptographic systems



- www.ism.ase.ro



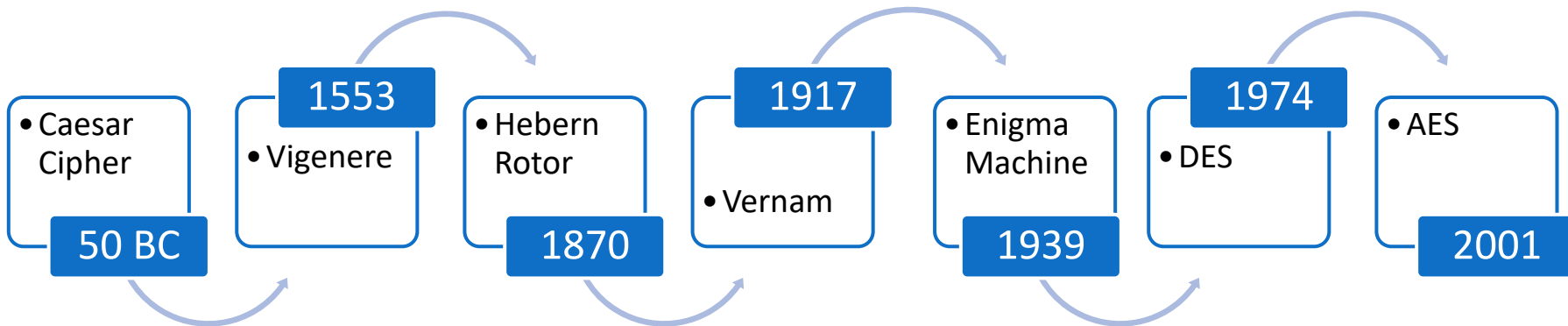
Symmetric cryptographic systems

Encryption algorithm is publicly
known !

DO NOT USE proprietary
solutions !



Symmetric cryptographic systems





Transposition ciphers

- change the position of the plaintext characters
- there are changed blocks of chars or the entire message
- the encryption key, $K = (d, f)$, where d represent the length of consecutive char blocks that will be enciphered accordingly to the permutation, f
- The alphabet of the clear text remains unchanged



Transposition ciphers

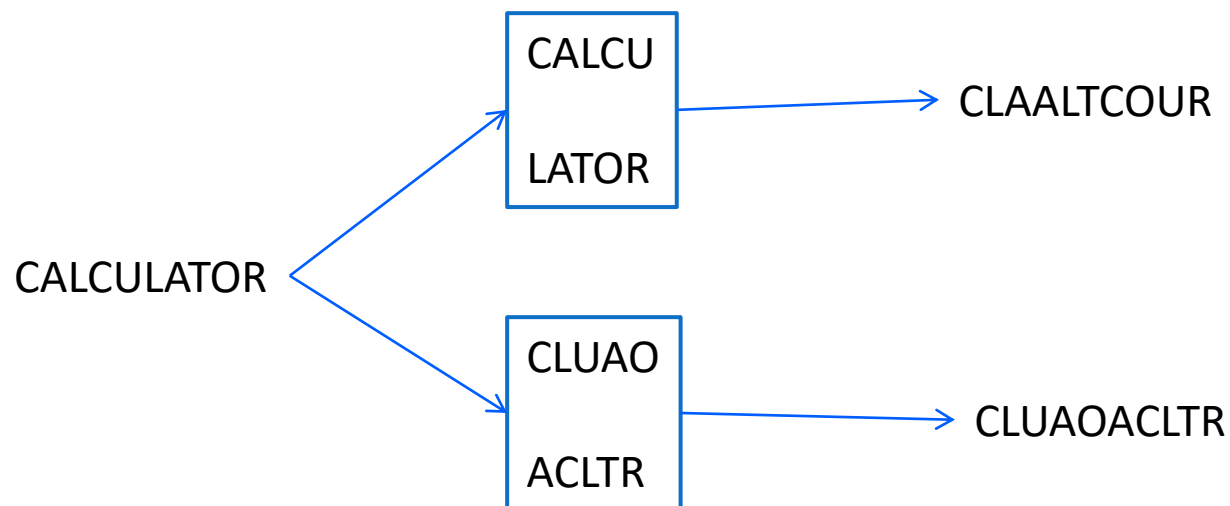
Classification

- By the number of transpositions
 - Mono-phase
 - Multi-phase
- By the target element
 - Monographic – for chars
 - Multigraphic – for groups of characters



Transposition ciphers

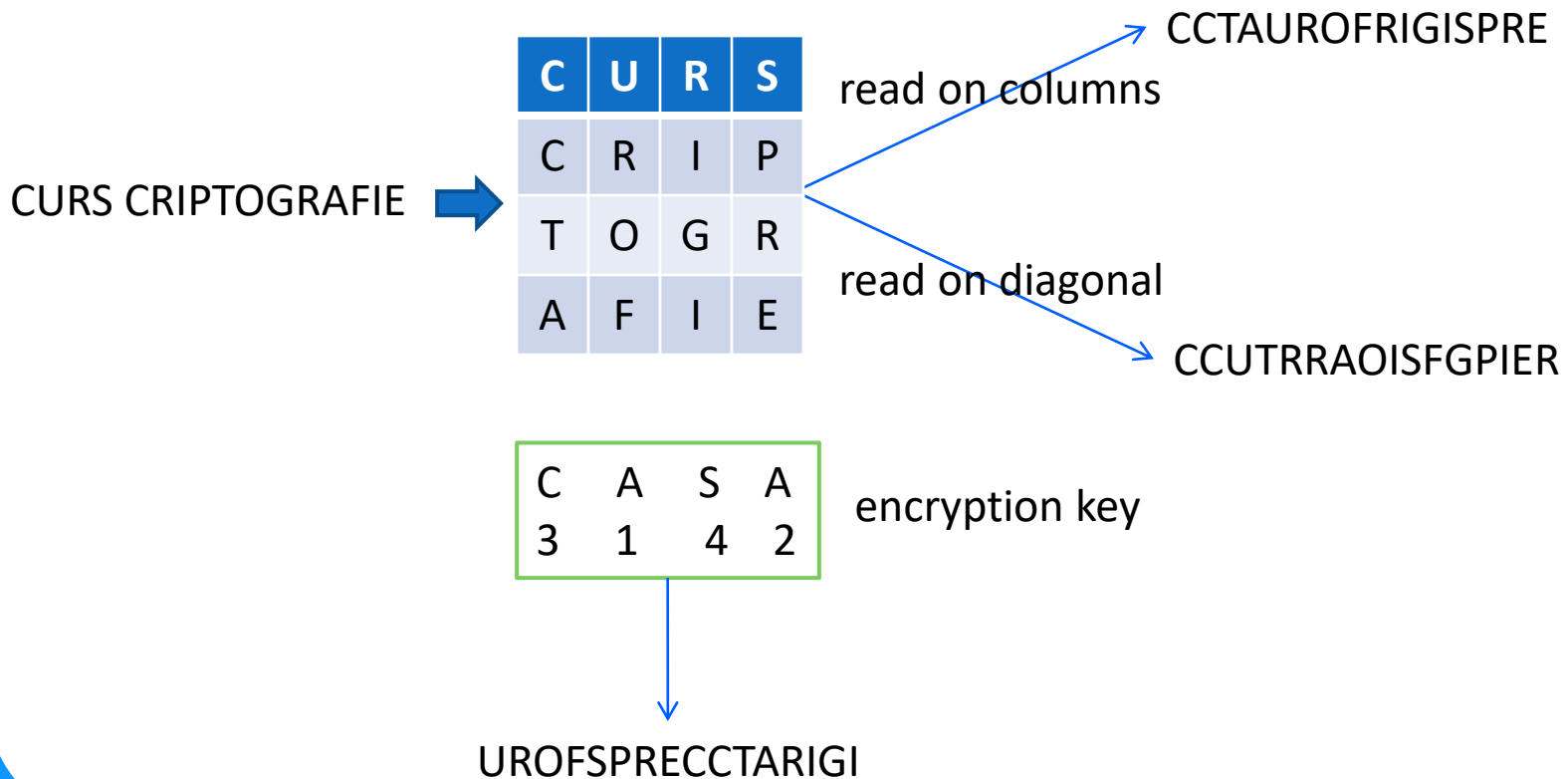
- Monographic





Transposition ciphers

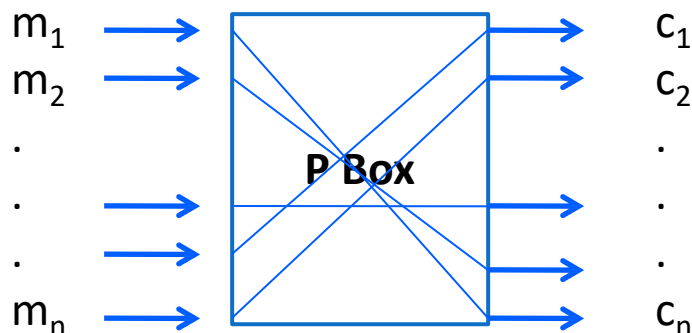
- Monographic





Transposition ciphers

- Easy to implement
- Vulnerable to statistic attacks (character frequency remains the same)
- **Transpositions are implemented by P boxes**





Substitution ciphers

- Replace each character from the plaintext alphabet, A , with one from the ciphers alphabet, C
- If $A = \{a_1, a_2, \dots, a_n\}$ and $C = \{f(a_1), f(a_2), \dots, f(a_n)\}$, $f: A \rightarrow C$ is the substitution function, the cipher key



- $$C = a * M + b \pmod{N}$$

- a – amplification factor / selection factor for $b = 0$
- b – shifting coefficient
- the pair (a,b) – the substitution key



Substitution ciphers

Caesar Cipher

- Mono-alphabetic substitution
- $A = \{A, B, C, \dots, X, Y, Z\} = C$
- $C(e_i) = e_i + 3 \pmod{26}$, with $e_i = \{0, 1, 2, \dots, 25\}$

A B C ... X Y Z

D E F ... A B C

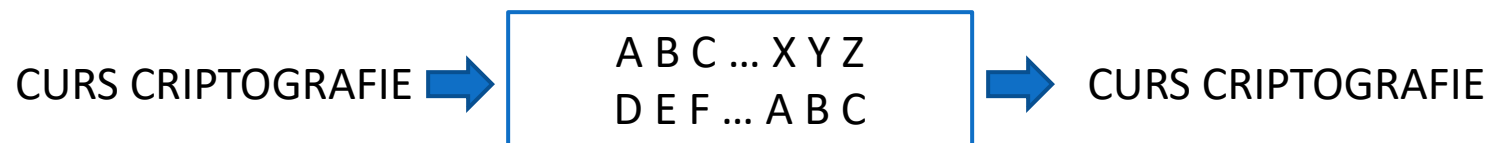
- a general function: $C(e_i) = (e_i + b_i) \pmod{26}$



Substitution ciphers

Caesar Cipher

- very vulnerable to attacks – mainly brute-force attacks
- the number of possible keys is 26





Substitution ciphers

Random substitution cipher

- increase protection
- the characters of the substitution alphabet are statistical independent
- the key is a set $\{(a_1, b_1), (a_2, b_2), \dots, (a_{26}, b_{26})\}$, where a_i, b_i has values in $\{0, 1, 2, \dots, 25\}$



Substitution ciphers

Substitution by mnemonic keys:

- the substitution rule is given by a literal key
- the mnemonic key generated by the literal key
- the number of correlations is bigger



Substitution ciphers

Substitution by mnemonic keys:

C H E I E
1 4 2 5 3

encryption key

A B C D E
F G H I J
K L M N O
P Q R S T
U V W X Y
Z

alphabet

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
A F K P U Z C H M R W E J O T Y B G L Q V D I N S X

P_1 permutation



Substitution ciphers

Substitution by stair-shaped table

S	E	C	U	R	I	T	A	T	E
7	3	2	10	6	5	8	1	9	4

encryption key

1								A	B	C
2			D	E	F	G	H	I	J	K
3		L	M	N	O	P	Q	R	S	T
4										U
5						V	W	X	Y	Z
6										

alphabet

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
 L D M E N F O G P V H Q W A I R X B J S Y C K T U Z

permutation P_1



Substitution ciphers

Homophonic substitution

- ciphers based on simple substitution are vulnerable to attacks that take into account characters frequency
- characters are replaced with symbols from $f(a)$, where $f:A \rightarrow 2^C$
- the frequency of the code symbols is almost constant
- the number of possible keys is $(26!)^n$



Substitution ciphers

Poly-alphabetic substitutions

$C_1, C_2, \dots, C_d$ – d cipher alphabets

$f_1, f_2, \dots, f_d$ – d substitution functions $f_{i=1..d}: A \rightarrow C_i$

$M = m_1 m_2 \dots m_d m_{d+1} \dots m_{2d} \dots$ – plaintext

$E_k(M) = f_1(m_1)f_2(m_2)\dots f_d(m_d)f_1(m_{d+1})\dots$ – cipher



Substitution ciphers

Poly-alphabetic substitutions

- use different simple substitutions
- increase the security by leveling the language statistic characteristics
- the number of possible keys is $(26!)^n$
- a symbol from the ciphertext can represent different symbols in the plaintext



Substitution ciphers

Poly-alphabetic substitutions – Vigenere cipher

- the key: $K = \{k_1, k_2, \dots, k_d\}$
- the substitution function $f_i(a) = (a + k_i) \pmod n$, where n – alphabet length
- another version is to use a binary alphabet – Vernam cipher



Substitution ciphers

Poly-alphabetic substitutions – Vigenere cipher

It is defined the equivalences $A = 0, B = 1, C = 2, \dots$

Plaintext: **SUBSTITUTIE POLIALFABETICA**

Key: **ACADEMIE**

$$S + A = 18 + 0 \pmod{26} = 18 \pmod{26} = 18 = S$$

$$U + C = 20 + 2 \pmod{26} = 22 \pmod{26} = 22 = W$$

$$B + A = 1 + 0 \pmod{26} = 1 \pmod{26} = 1 = B$$

Ciphertext: **SWBVXUBYTKE SSXQELHAEIFQGA**



Substitution ciphers

Polygram substitution:

- substitutes block of chars (polygrams) from the plaintext
- Hides the frequency of different characters
- the simplest form is for $n=2$ when the diagram m_1m_2 from the plaintext is substituted by the c_1c_2 diagram from the ciphertext



Substitution ciphers

- The correspondence between m_1m_2 and c_1c_2 diagrams is defined by a square table

A B C D E

A	QX	FN	LB	YE	HJ
B	AS	EZ	BN	RD	CO
C	PD	RA	MG	LU	OP



Polygram substitution :

- **PLAYFAIR cipher**- in the first line of the square it is placed a key word; the rest of the lines are completed with alphabet chars, without repeating them
- **algebraic encryption method** – linear transformation based on: $f(M) = P * M^T$ where P is a square matrix with $n \times n$ lines and columns, and M is a column vector with n elements from the plaintext



Playfair Cipher

- not even the large number of keys in a monoalphabetic cipher provides security
- improves security by encrypting multiple letters;
- invented by Charles Wheatstone in 1854, but named after his friend Baron Playfair



Playfair Key Matrix

- a 5X5 matrix of letters based on a keyword
- fill in letters of keyword (sans duplicates)
- fill rest of matrix with other letters
- eg. using the keyword **MONARCHY**

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z



Encrypting and Decrypting

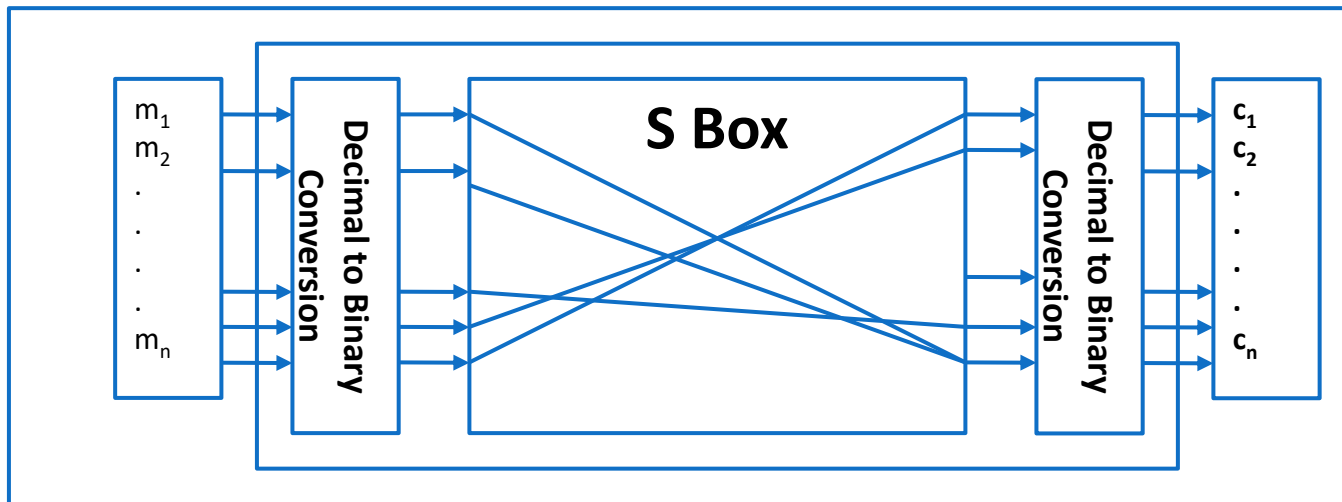
- plaintext is encrypted two letters at a time
 1. if a pair is a repeated letter, insert filler like 'X'
 2. if both letters fall in the same row, replace each with letter to right (wrapping back to start from end)
 3. if both letters fall in the same column, replace each with the letter below it (again wrapping to top from bottom)
 4. otherwise each letter is replaced by the letter in the same row and in the column of the other letter of the pair



- www.ism.ase.ro

Substitution ciphers

Simple S Box





Substitution ciphers

- Vernam (1917)
- Fish
- Enigma
(http://en.wikipedia.org/wiki/Enigma_machine)
- Vigenere
- Hill



OTP ciphers (One Time Pad)

- Each bit/character from the plaintext is encrypted by a modular addition (XOR) with a bit/character from a secret random key
- secure till our days (!!! if properly used = random key as large or greater than the plaintext)
- patented by Vernam (1917)



- the key length is the message length
- the key becomes a message that must be transmitted





Rotor Machines

- before modern ciphers, rotor machines were most common complex ciphers in use
- widely used in WW2
 - German Enigma, Allied Hagelin, Japanese Purple
- implemented a very complex, varying substitution cipher
- used a series of cylinders, each giving one substitution, which rotated and changed after each letter was encrypted
- with 3 cylinders have $26^3=17576$ alphabets



OTP cipher - Enigma

- Implements a poly-alphabetic Vigenere encryption
- Designed in 1918 in Germany
- The security of the cipher:
 - number of disks (rotors): for 3 $\rightarrow 26 \cdot 26 \cdot 26 = 17.576$ possible solutions
 - 6 ways to transpose disks $\rightarrow 6 \cdot 17.576 = 105.456$ solutions
 - connectionn table with 10 pairs of chars $\rightarrow 90 \cdot 105.456 = 9.491.040$
- A machine cu n rotors does the encryption of a symbol in $2n + 1$ substitutions



OTP cipher - Enigma

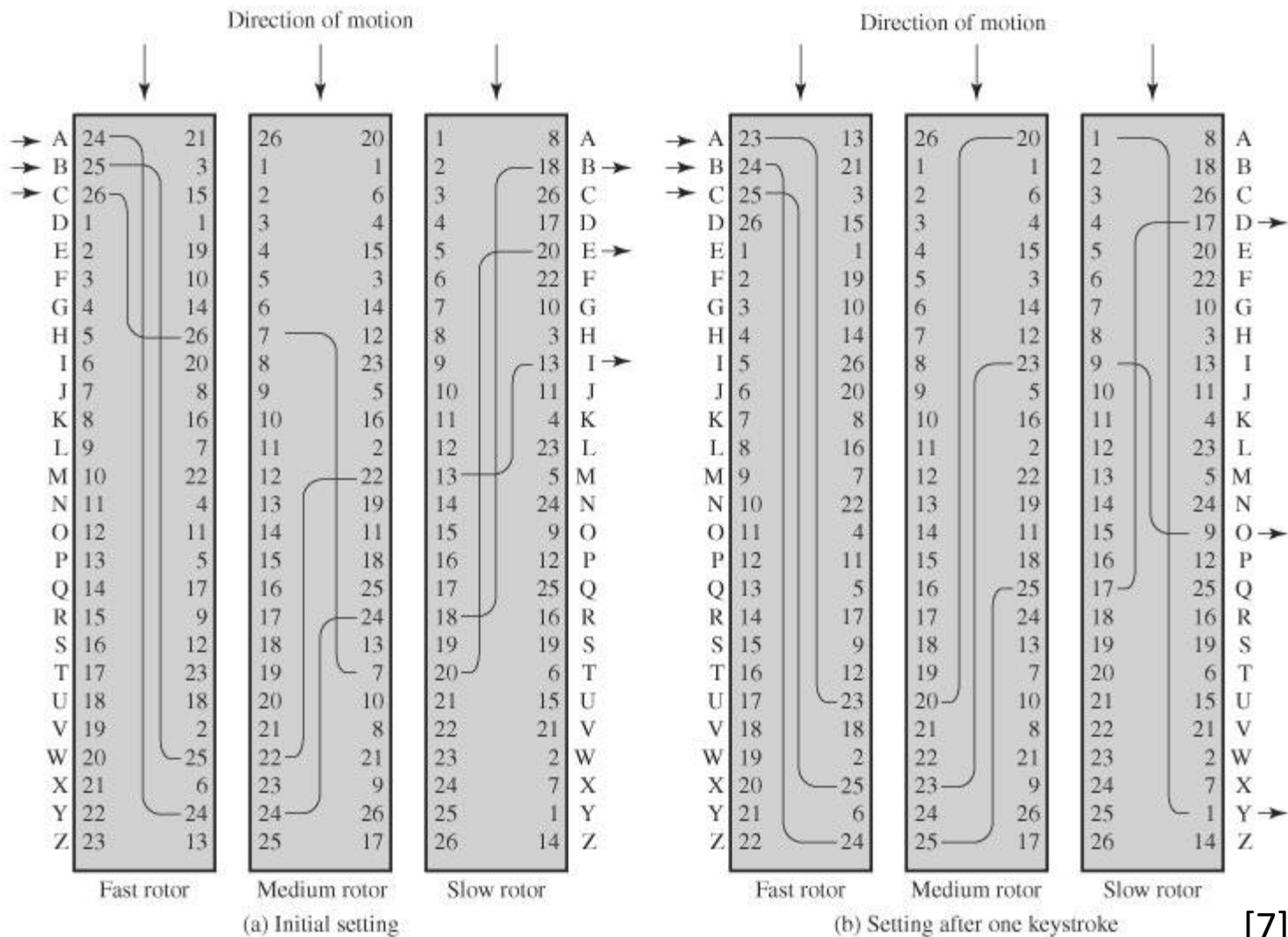
Components:

- Keyboard
- Electric Circuits
- Rotors with 26 symbols (3 -> 7)
- Reflector
- Connections table

Settings:

- The rotors order and their initial position
- The initialization of the symbols ring
- The initialization of connections

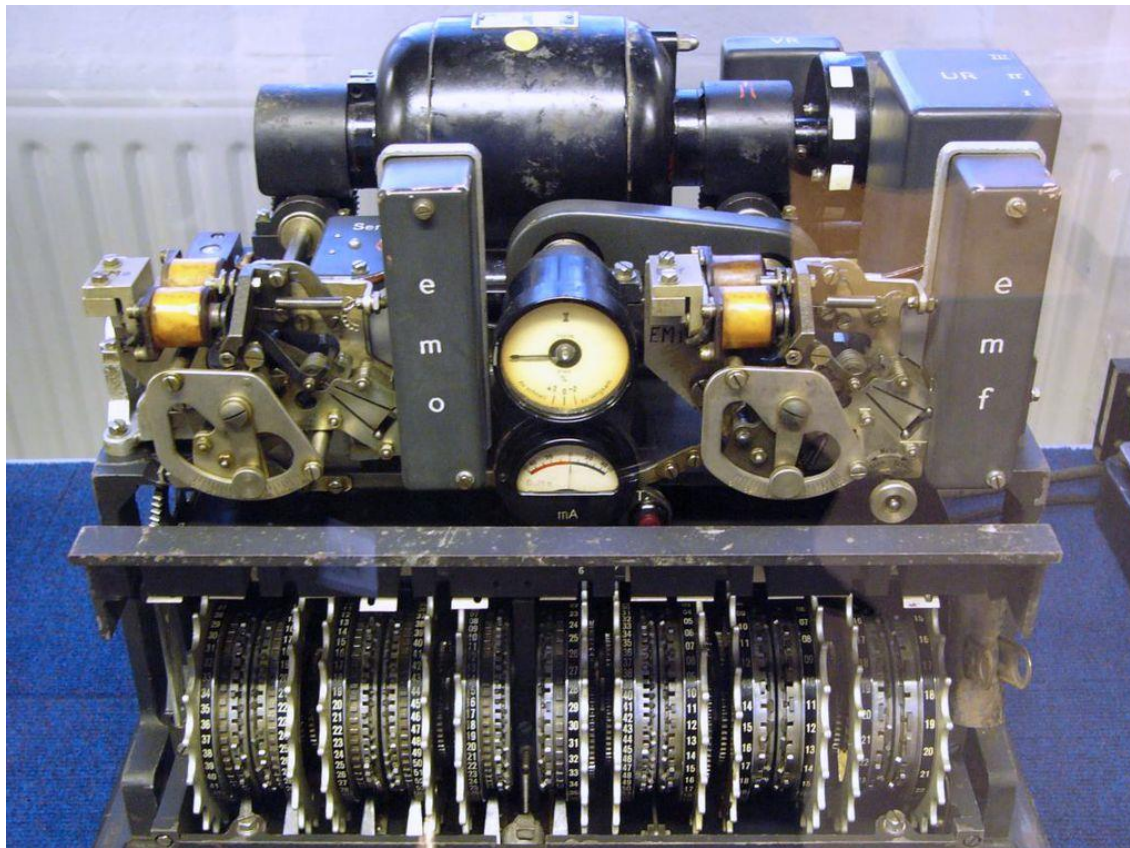
OTP cipher - Enigma



OTP cipher - Lorenz

- Declassified in 2002.
- Used from 1940 by the German Army.
- Used by high command and top generals.
- More advanced, complex, faster and more secure than Enigma.
- Bill Tutte broke Lorenz system in spring 1942 (without ever having seen the machine)

OTP cipher - Lorenz



https://en.wikipedia.org/wiki/Lorenz_cipher



Enigma vs Lorenz

Enigma	Lorenz
~100.000 made	~200 made
3 or 4 rotors	12 rotors and 501 pins
103.325.660.891.587.134.000.000 different possible settings	$6.546.781 \cdot 10^{144}$ different possible settings



Product ciphers

- A product/generated algorithm (also called product cipher) is a composition of t functions (ciphers) $f_1, f_2, \dots, f_t$, where each f_i can be a substitution or a permutation
- Are based on S-P boxes networks, resulting the cryptogram $C = E_k(M) = S_t P_{t-1} \dots S_2 P_1 S_1(M)$, each S_i being dependent of a k key, part of K cipher



Product Ciphers

- ciphers using substitutions or transpositions are not secure because of language characteristics
- hence consider using several ciphers in succession to make harder, but:
 - two substitutions make a more complex substitution
 - two transpositions make more complex transposition
 - **a substitution followed by a transposition makes a new much harder cipher**
- this is bridge from classical to modern ciphers



Product ciphers

Complex ciphers:

- **S-boxes** – maps entrances of n bits in exits of m bits (often $m=n$)
- **Feistel networks** - method of transforming a cryptographic function into a permutation or building bits blocks, used by de cipher, of simple functions
- **key scheduling** – the process of key expanding from N bits in $N*r$ bits
- **Logical operations** on bits groups(bit slice operations) - AND, OR, XOR, NOT

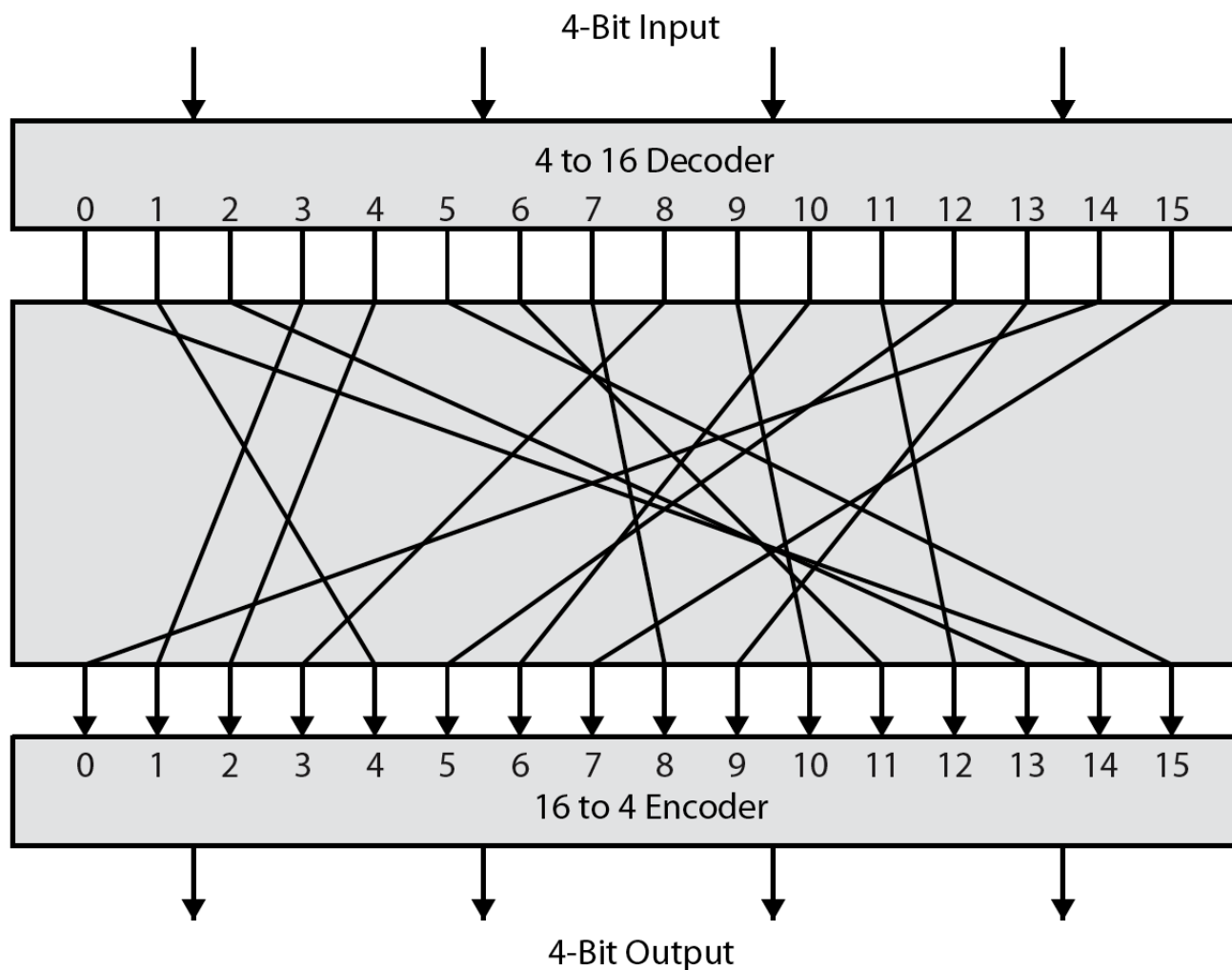


Feistel Network

- most symmetric block ciphers are based on a **Feistel Cipher Structure**
- needed since must be able to **decrypt** ciphertext to recover messages efficiently
- block ciphers look like an extremely large substitution
- would need table of 2^{64} entries for a 64-bit block
- instead create from smaller building blocks
- using idea of a product cipher



Feistel Ideal Block Cipher



[Source 7]



- Claude Shannon introduced idea of substitution-permutation (S-P) networks in 1949 paper
- form basis of modern block ciphers
- S-P nets are based on the two primitive cryptographic operations seen before:
 - *substitution* (S-box)
 - *permutation* (P-box)
- provide *confusion* & *diffusion* of message & key



Confusion and Diffusion

- cipher needs to completely obscure statistical properties of original message
- a one-time pad does this
- more practically Shannon suggested combining S & P elements to obtain:
- **diffusion** – dissipates statistical structure of plaintext over bulk of ciphertext
- **confusion** – makes relationship between ciphertext and key as complex as possible

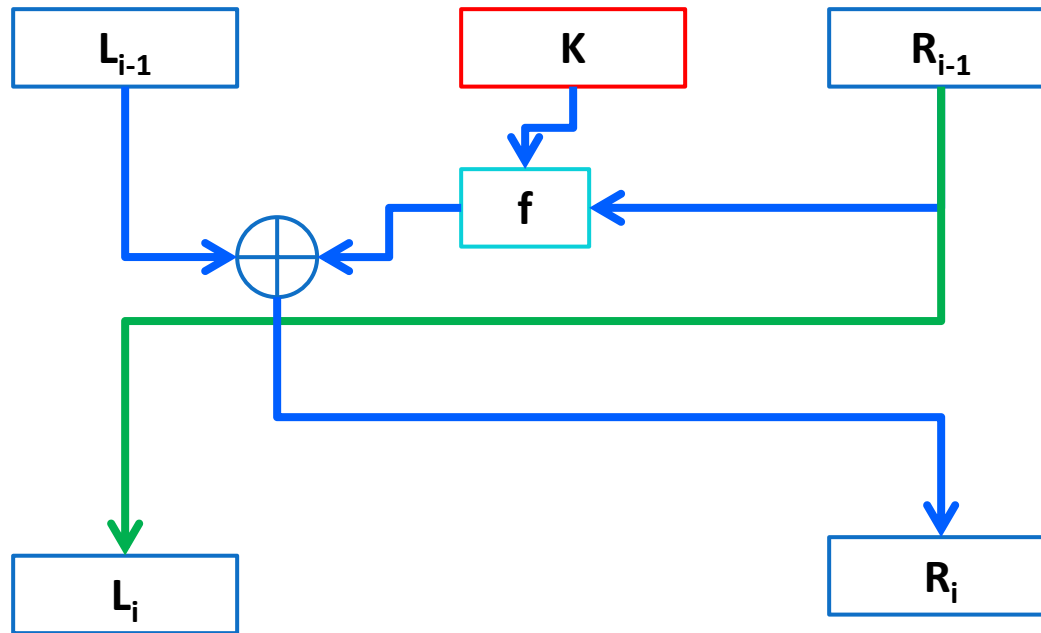


Feistel Network

- Defined by Horst Feistel in the '60
- Used for the 1st time in Lucifer algorithm (IBM)
- The network takes a function f , $f:\{0,1\}^{n/2} \times \{0,1\}^N \rightarrow \{0,1\}^{n/2}$ and generates a reversible function $ff:\{0,1\}^n \rightarrow \{0,1\}^n$
 - $n/2$ is the lengths in bits of each L and R block
 - N is the number of bits of the key used by f function.
- Function ff is often called round function.
- If a round function depends of N key bits, then a cipher using Feistel networks with r rounds (r round functions meaning r ff functions) need $N \cdot r$ key bits.
- For designing f functions are typically used S boxes
- implements Shannon's S-P net concept

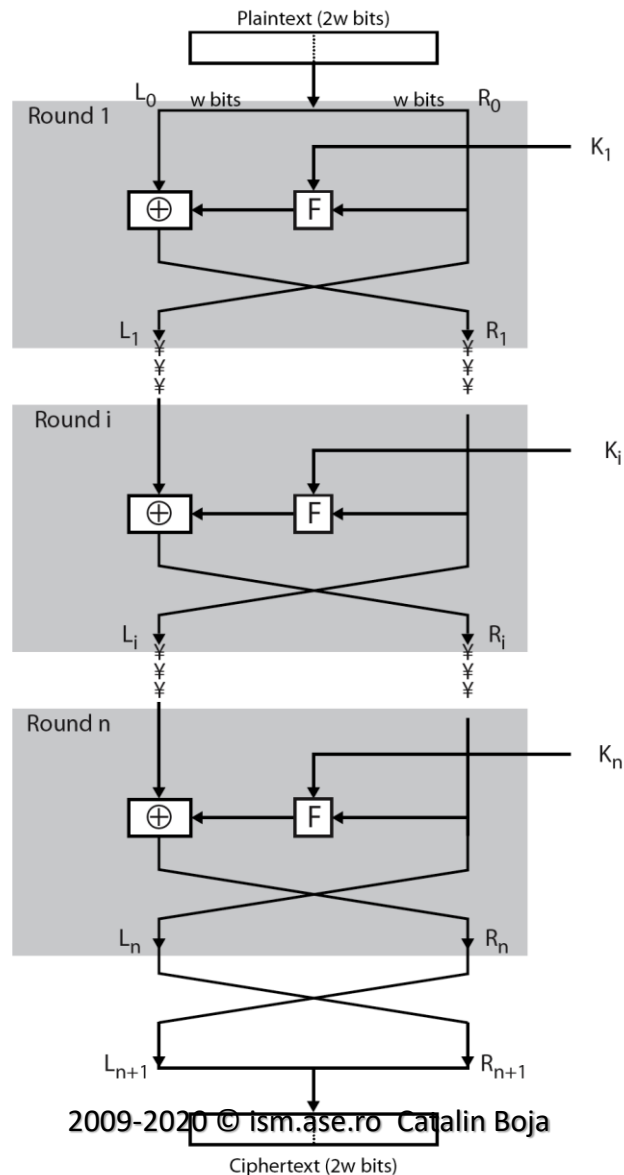


Feistel Network



Feistel network

Feistel Cipher Structure



[Source 7]

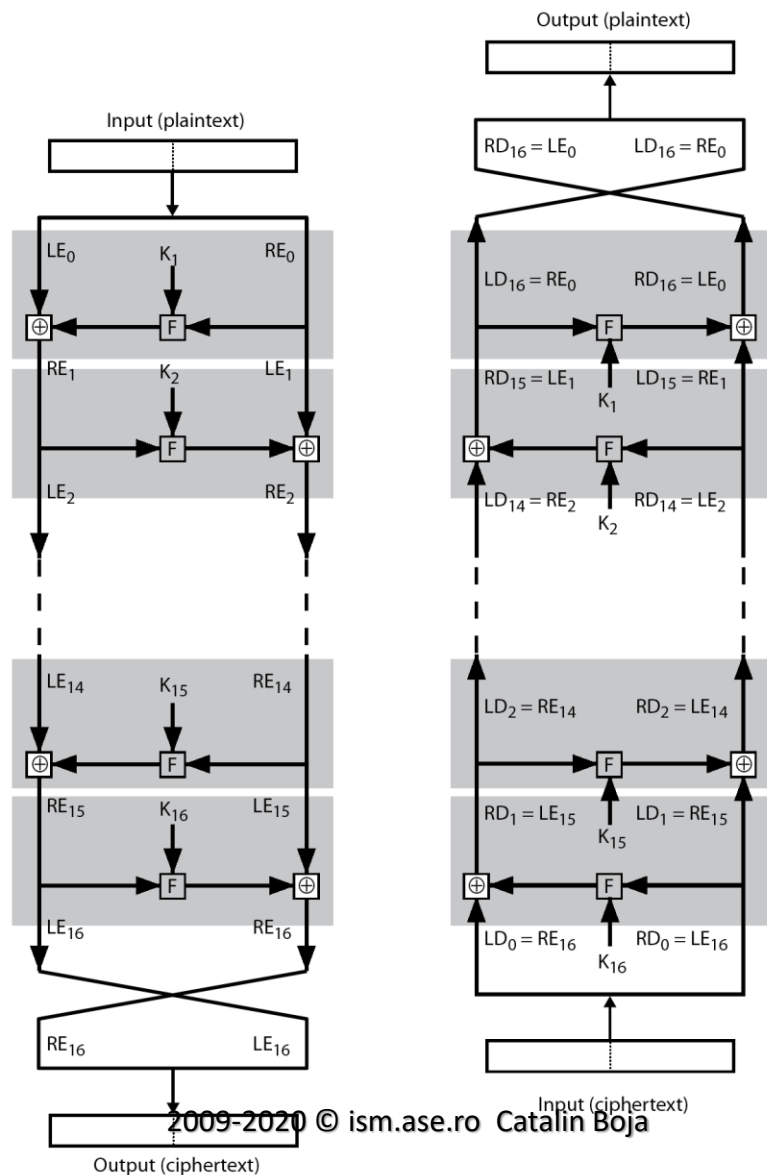


Feistel Cipher Design Elements

- block size
- key size
- number of rounds
- subkey generation algorithm
- round function
- fast software en/decryption
- ease of analysis



Feistel Cipher Decryption





Cipher types



Cipher types

- Cipher types:
 - **block ciphering**
 - stream ciphering
- Define ways to use symmetrical algorithms (the algorithm used does no matter)
- Are simple because the security is the attribute of ciphering and not of the way the ciphering scheme is done



Cipher types

Block ciphering

- Operates on blocks of plaintext and ciphertext – usually of 64, 128 bits and larger
- Most known block ciphering types: ECB, CBC, PCBC, OFB, CFB
- The same plaintext block will be always encrypt to the same ciphertext block, using the same key



Cipher types

Stream ciphering

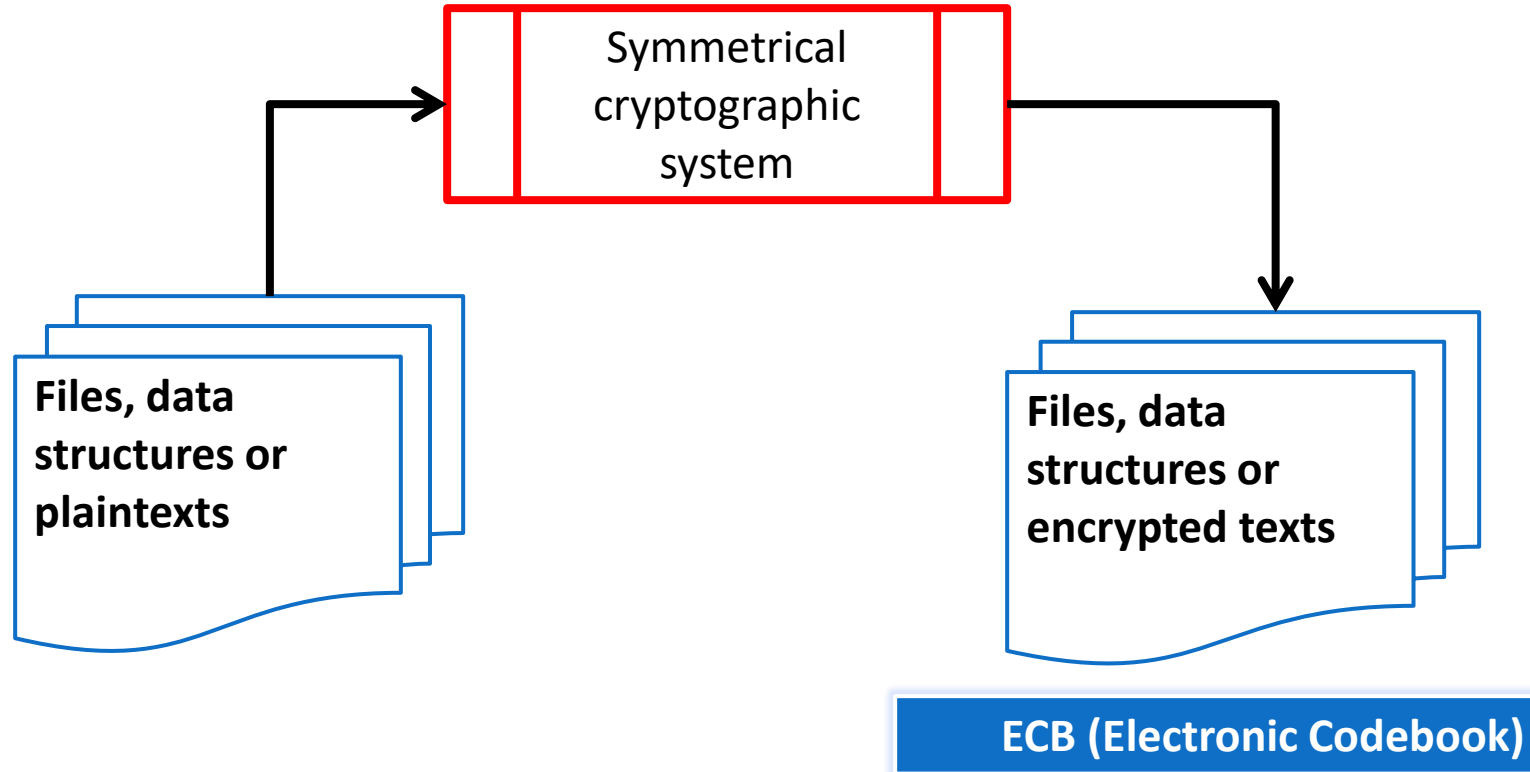
- Operates on streams of plaintext and ciphertext one bit or byte large, but sometimes even on 32-bit data
- Modes:
 - sequential cipher
 - self-synchronizing sequential cipher
 - feedback cipher
 - synchronous sequential cipher
 - output-feedback sequential cipher
 - counter cipher.
- The same plaintext will be encrypted to a different bit or byte in case of repeated encryptions



- Same block of plaintext encrypts into the same block of ciphertext every time is encrypted
- It is theoretically possible to create a code book of plaintexts and corresponding ciphertexts (not feasible because every key need its own code book)
- Each plaintext block is encrypted independently.
- Very vulnerable to block replay



Block ciphering - ECB





Block ciphering

Plaintext block

B_i
128 bits

Files, data
structures or
plaintexts

ECB

Symmetrical
cryptographic
system

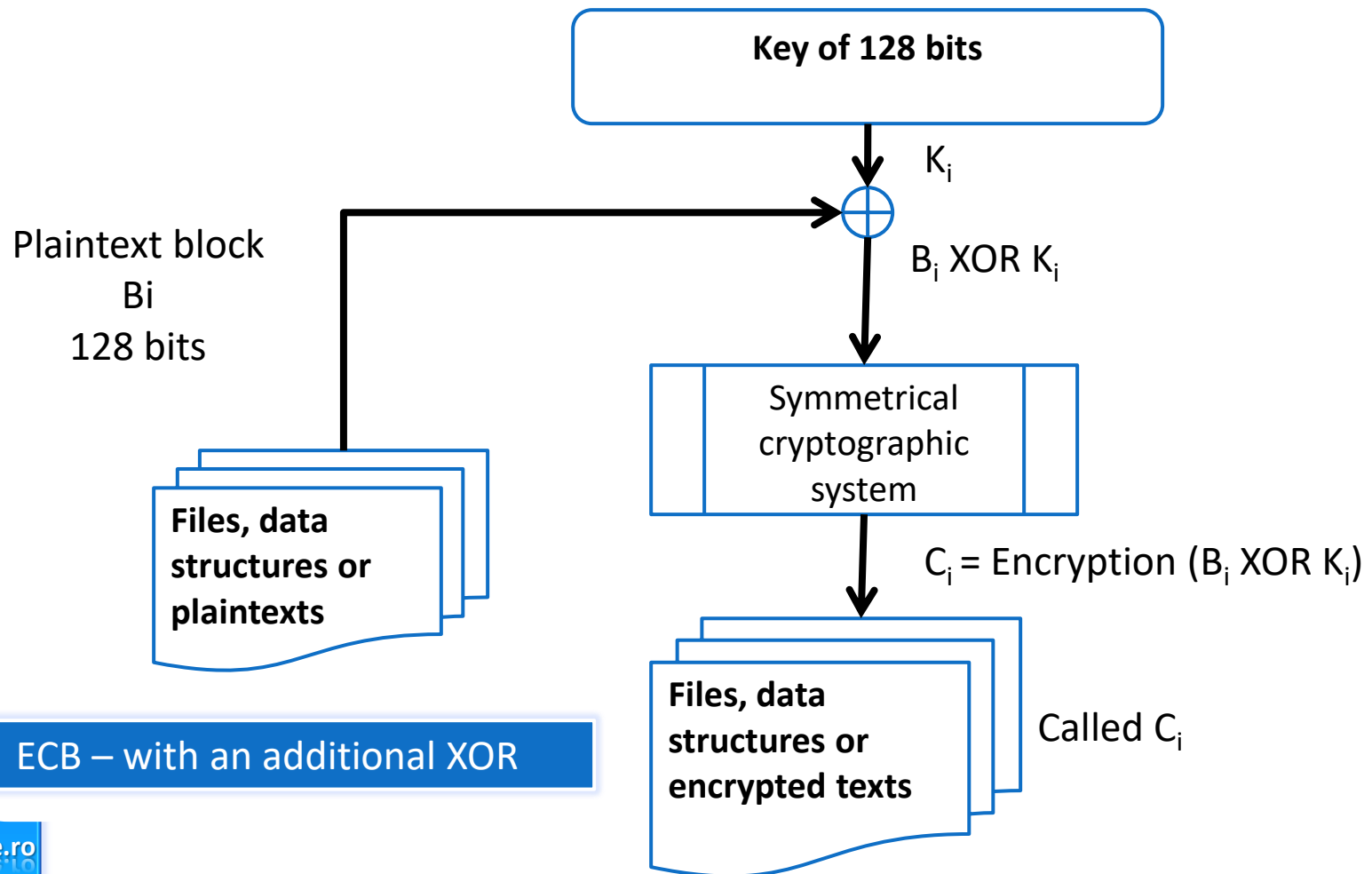
$C_i = \text{Encryption}(B_i \text{ XOR } K_i)$

Files, data
structures or
encrypted texts

Called C_i



Block ciphering



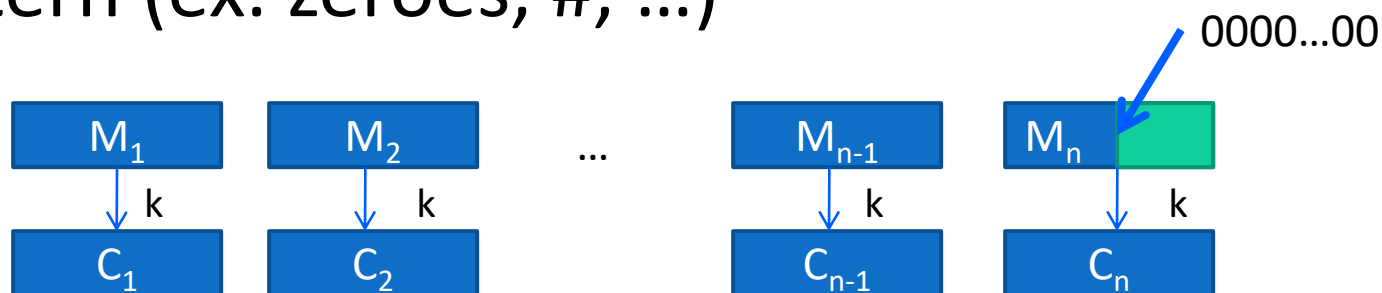


[3]



Block ciphering - Padding

- allows for processing of messages that are not evenly divisible into blocks of defined size (ex 64 bits)
- the last block could be shorter than required size
- the last block is padded by adding regular pattern (ex. zeroes, #, ...)





- ```
... | 1101 0100 0010 0010 1000 0000 |
```

- ```
... | 1101 0100 0010 0010 0000 0000 |
```

is not reversible

<https://en.wikipedia.org/wiki/Padding> (cryptography)



Block ciphering - Padding

- **ANSI X.923** – use needed 0 value bytes and the last byte defines the number of padded bytes

```
... | CC CC CC CC CC CC CC CC | CC CC  
00 00 00 00 00 06 |
```

- **ISO 10126** – use random bytes and the last byte defines the number of padded bytes

```
... | CC CC CC CC CC CC CC CC | CC CC  
16 4F 5D A4 11 06 |
```

[https://en.wikipedia.org/wiki/Padding_\(cryptography\)](https://en.wikipedia.org/wiki/Padding_(cryptography))



Block ciphering - Padding

- **PKCS#7** - [RFC 5652](https://tools.ietf.org/html/rfc5652) - value of each added byte is the number of bytes that are added

```
... | CC CC CC CC CC CC CC CC | CC CC  
06 06 06 06 06 06 |
```

the number of bytes added will depend on the block boundary to which the message needs to be extended; previous example is for 8 bytes (64 bit) blocks

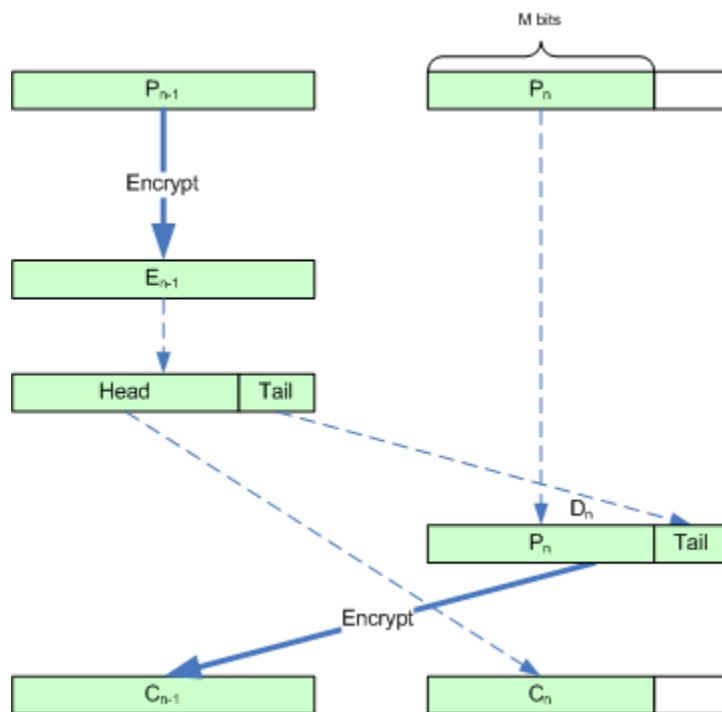
- **PKCS#5** – same as PKCS7 but only for 64 bit block ciphers

[https://en.wikipedia.org/wiki/Padding_\(cryptography\)](https://en.wikipedia.org/wiki/Padding_(cryptography))



Block ciphering - Padding

- Ciphertext stealing (CTS) is an alternative to padding [http://en.wikipedia.org/wiki/Ciphertext_stealing]

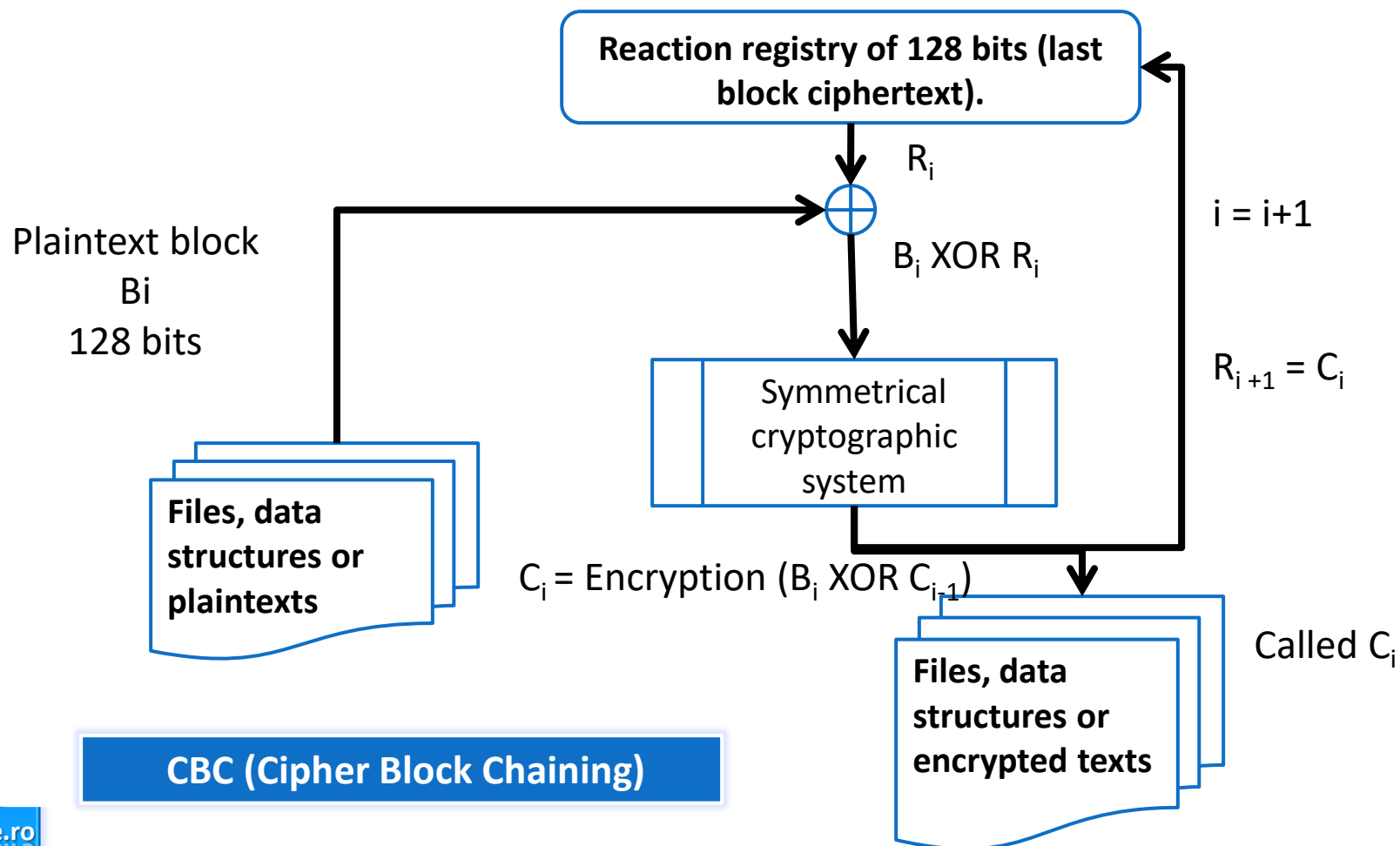




- Adds a feedback mechanism to a block cipher
- The result of the encryption of previous block are fed back into the encryption of current block
- Each ciphertext block is dependent not just on the plaintext block that generated it but on all the previous plaintexts blocks.



Block ciphering





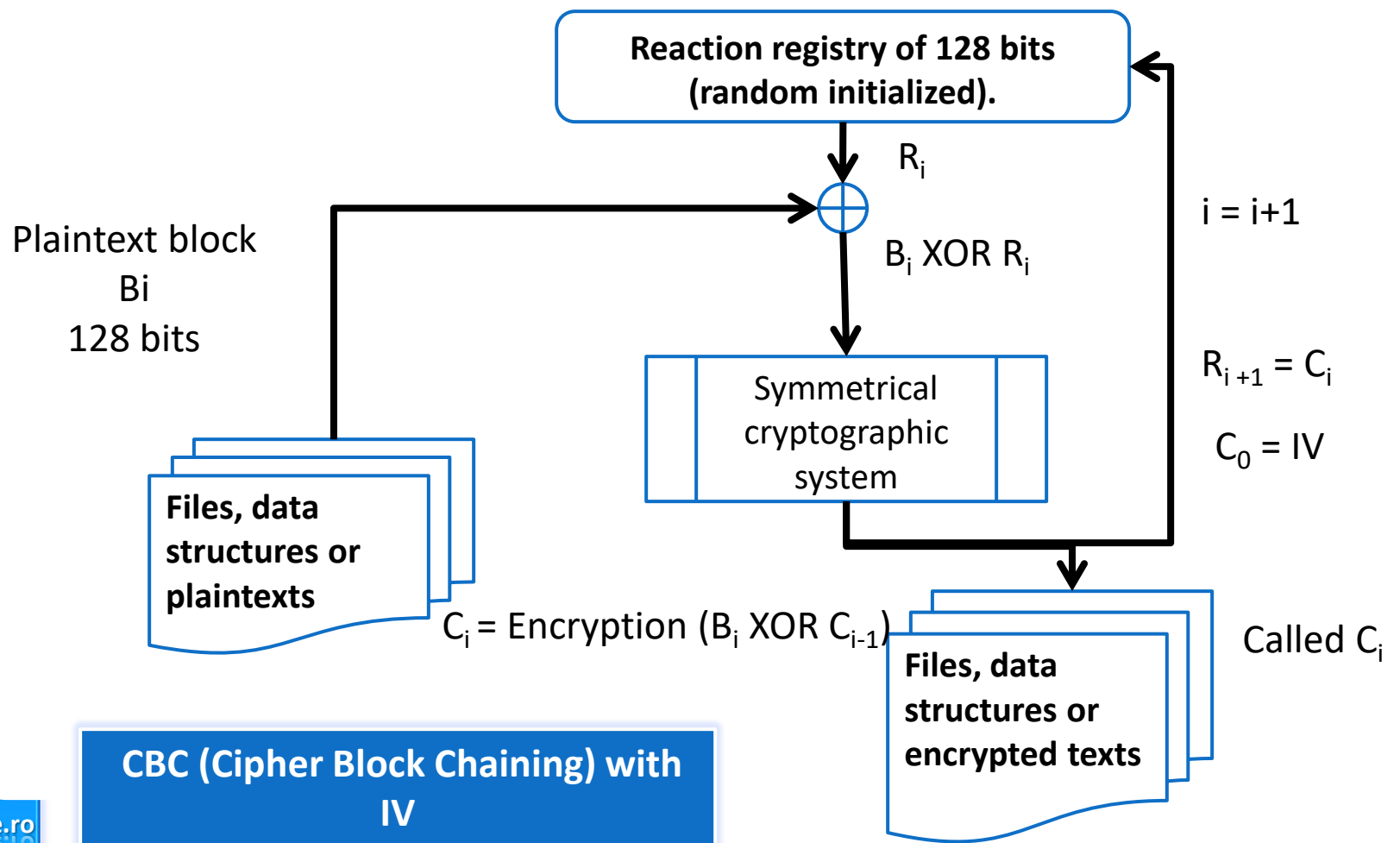
Block ciphering

Cipher Block Chaining - CBC

- Two identical messages will still encrypt to the same ciphertext
- Two messages that begin the same will encrypt in the same way up to the first difference
- Increase security with a IV – initialization vector



Block ciphering





[3]



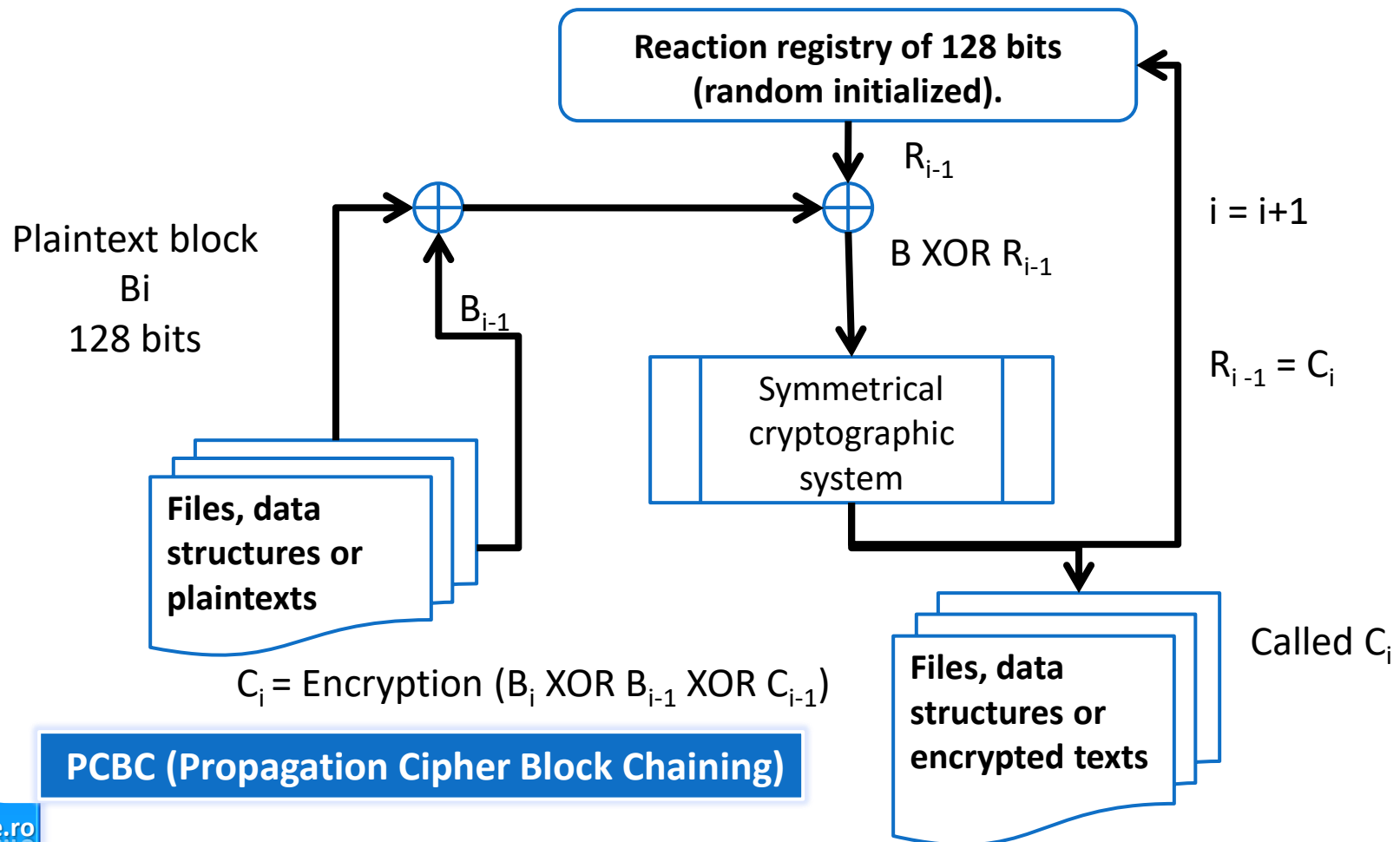
Block ciphering

Propagation Cipher Block Chaining - PCBC

- similar to CBC mode
- Both previous plaintext block and ciphertext blocks are XORed with the current plaintext block before encryption (or after decryption)
- PCBC was used in Kerberos version 4 to perform both encryption and data integrity checking in one pass.



Block ciphering

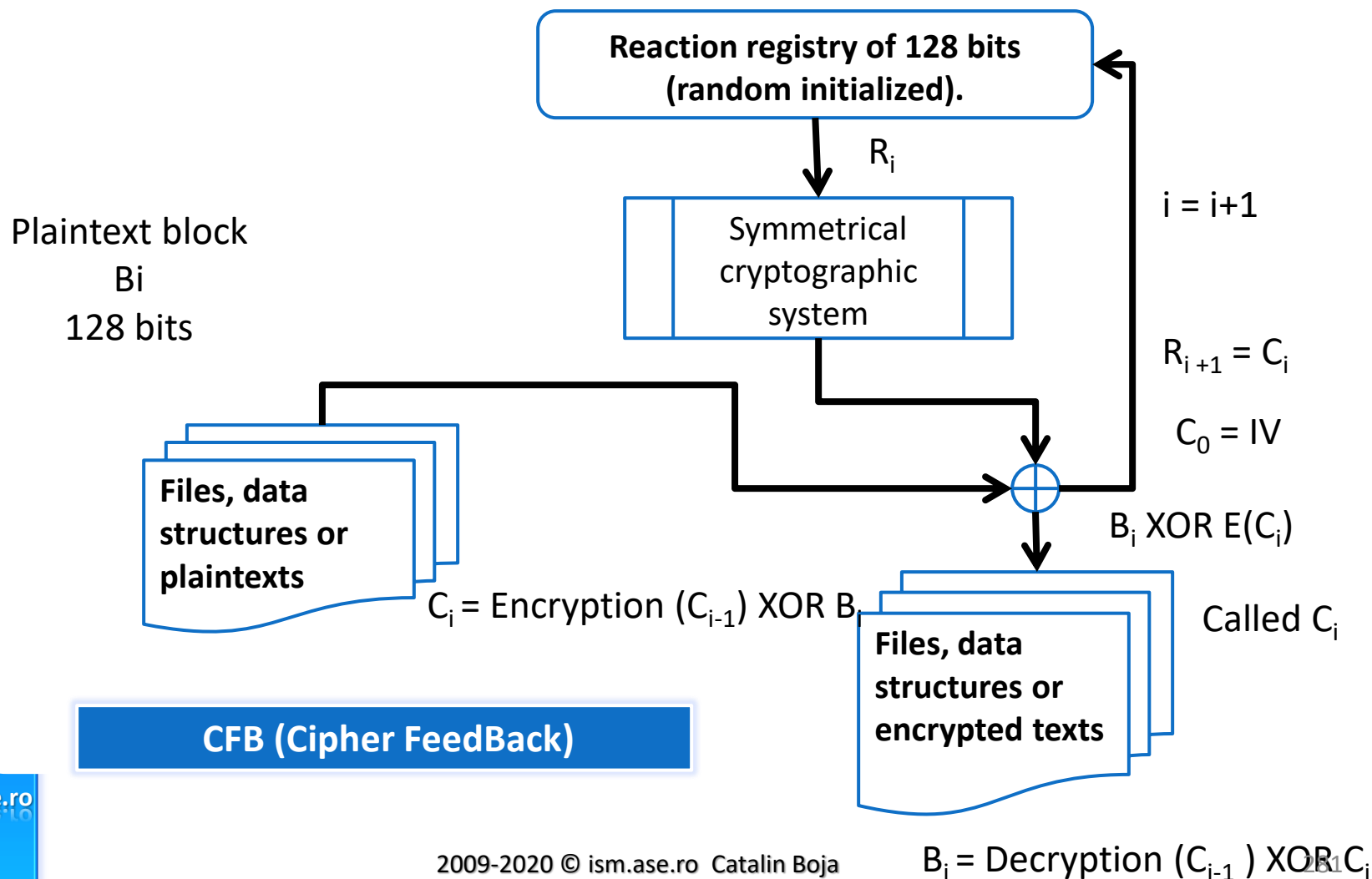


PCBC (Propagation Cipher Block Chaining)

$$B_i = \text{Decryption}(C_i) \text{ XOR } B_{i-1} \text{ XOR } C_{i-1}$$



Block ciphering





[3]



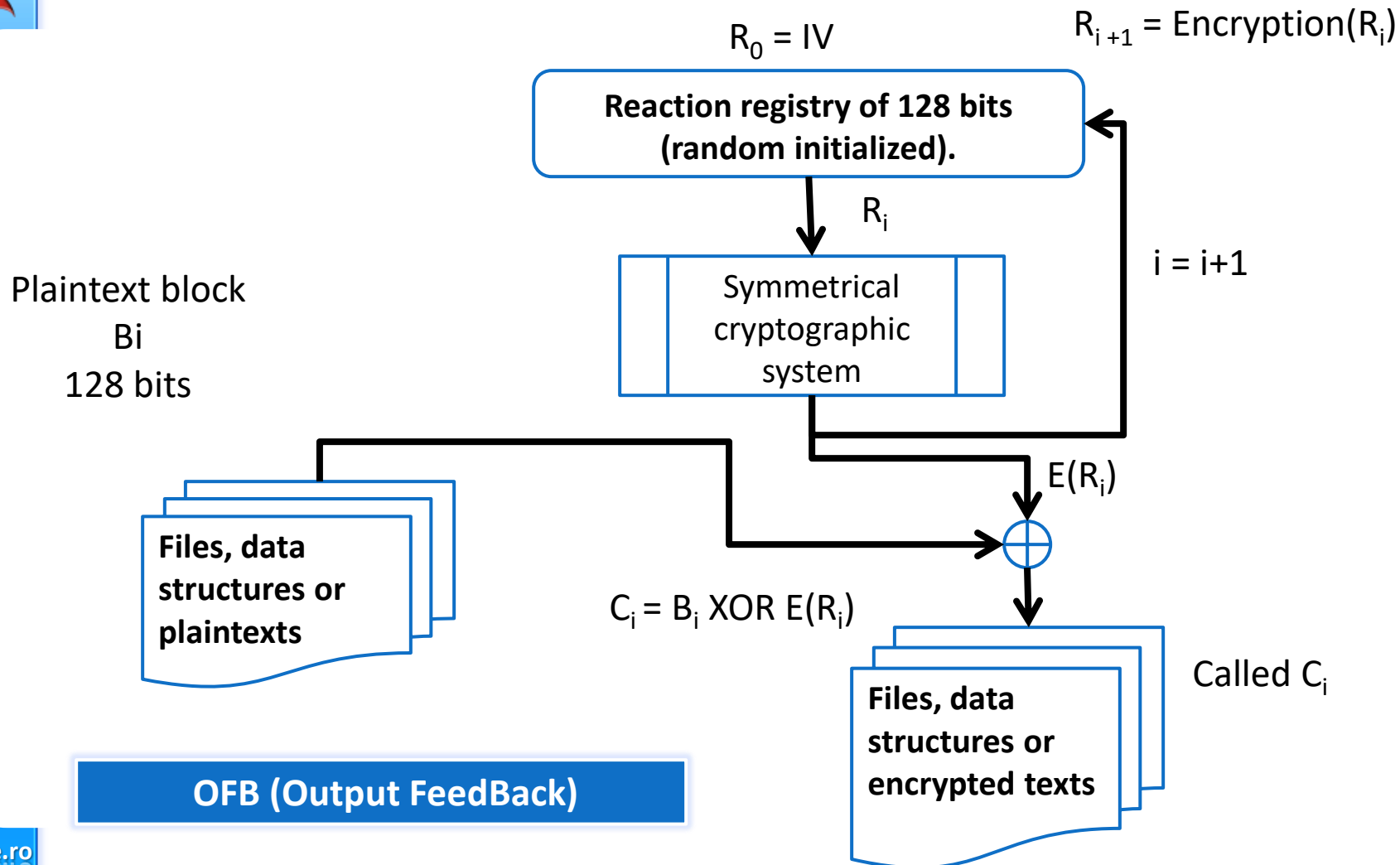
Block ciphering - CBC

	Advantages	Disadvantages
Security	• More than one message can be encrypted with the same key (with different IV). • Plaintext patterns are concealed. • Input to the block cipher is randomized. • Ciphertext is hard to manipulate;	• blocks can be removed from the beginning and end of the message, bits of the first block can be changed
Efficiency	• Speed is the same as the block cipher. • Decryption is parallelizable	• No preprocessing is possible. • Ciphertext is the same size as the plaintext, not counting IV. • Encryption is not parallelizable.
Fault-tolerance		A ciphertext error affects the corresponding bit of plaintext and the next block.

[3]



Block ciphering



Decryption: $B_i = C_i \text{ XOR } R_i$

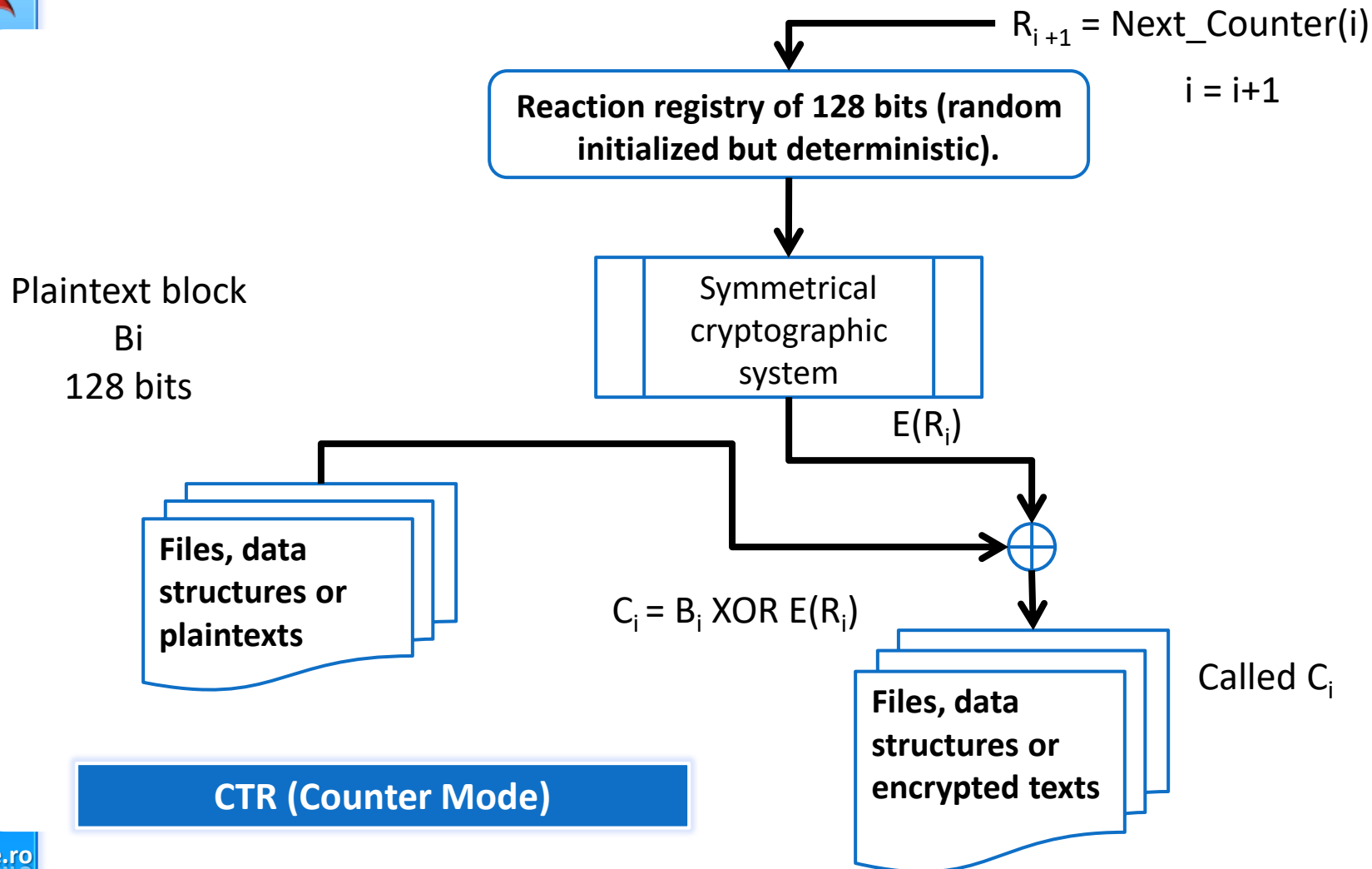


Advantages and Limitations of OFB

- bit errors do not propagate
- more vulnerable to message stream modification
- a variation of a Vernam cipher
 - hence must **never** reuse the same sequence (key+IV)
- sender & receiver must remain in sync
- originally specified with m-bit feedback
- subsequent research has shown that only **full block feedback** (ie CFB-64 or CFB-128) should ever be used



Block ciphering



Decryption: $B_i = C_i \text{ XOR } R_i$



Advantages and Limitations of CTR

- efficiency
 - can do parallel encryptions in h/w or s/w
 - can preprocess in advance of need
 - good for bursty high speed links
- random access to encrypted data blocks
- provable security (good as other modes)
- but must ensure never reuse key/counter values, otherwise could break (cf OFB)



Block ciphering

- **cipher feedback (CFB)** mode, transforms a block cipher into a self-synchronizing stream cipher
- **output feedback (OFB)** mode makes a block cipher into a synchronous stream cipher
- **counter mode (CTR)** turns a block cipher into a stream cipher



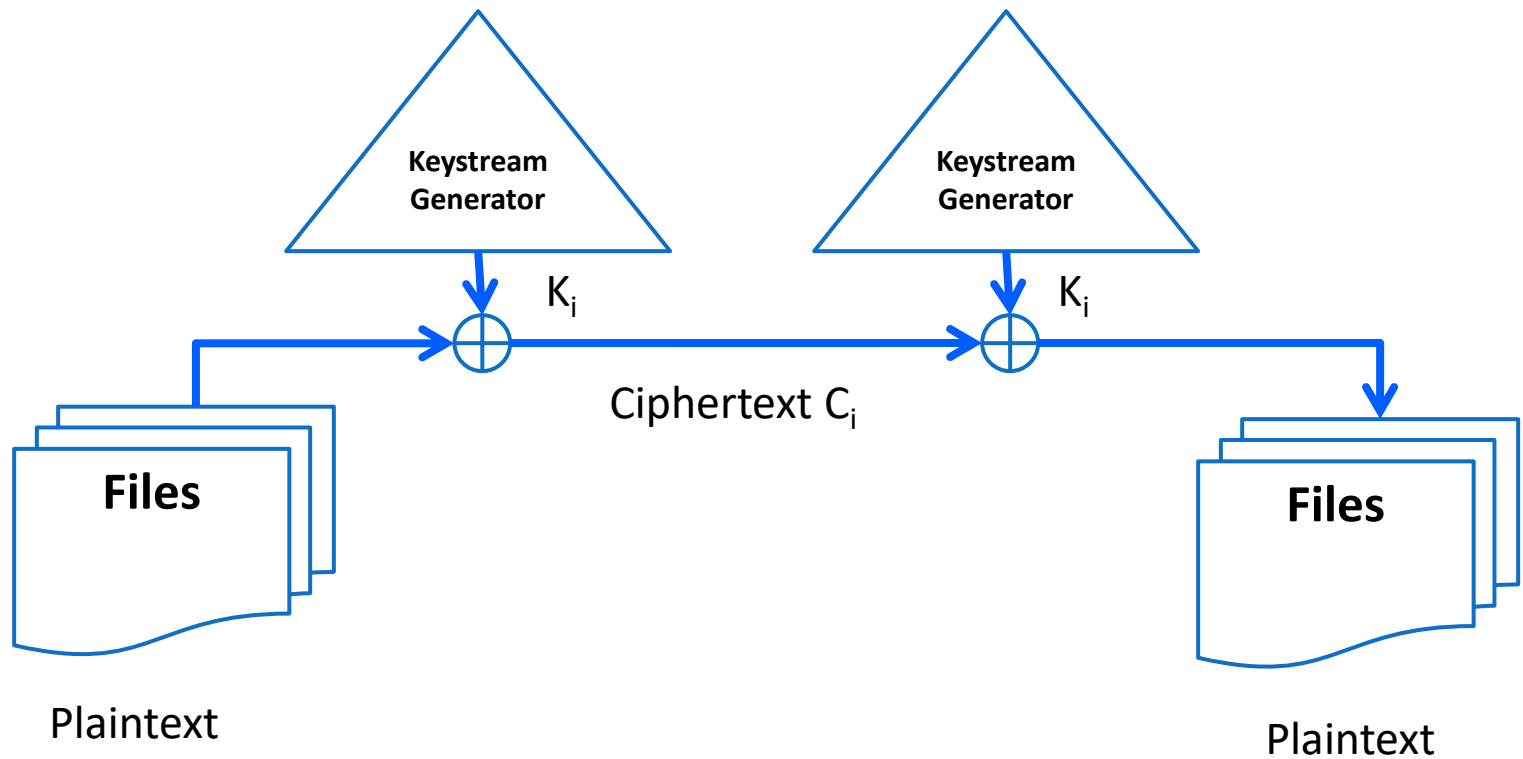
Stream Ciphers

Stream Ciphers

- Convert plaintext to ciphertext 1 bit/byte at a time
- Are based on XOR to encrypt the data(Vernam example)
- Security depends entirely on the insides of keystream generator.



Stream Ciphers



Sequential ciphering



Stream Ciphers Properties

- some design considerations are:
 - long period with no repetitions
 - statistically random
 - depends on large enough key
 - large linear complexity
- properly designed, can be as secure as a block cipher with same size key
- but usually simpler & faster



- each keystream bit is a function of a fixed number of previous ciphertext bits
- ciphertext auto key (CTAK)
- The decryption keystream generator will automatically synchronize with the encryption keystream generator after receiving n ciphertext bits.



Stream Ciphers

Self-synchronizing stream ciphers:

1. each message begins with a random header n bits long;
2. the header is encrypted, transmitted, and then decrypted
3. the decryption will be incorrect, but after those n bits both keystream generators will be synchronized.



Stream Ciphers

Synchronous stream cipher:

- the keystream is generated independent of the message stream
- Key Auto-Key (KAK)
- the two keystream generators (at encryption and decryption) are synchronized and generate the same output -> are deterministic -> are periodic
- Except for OTP, all keystream generators are periodic



- www.ism.ase.ro



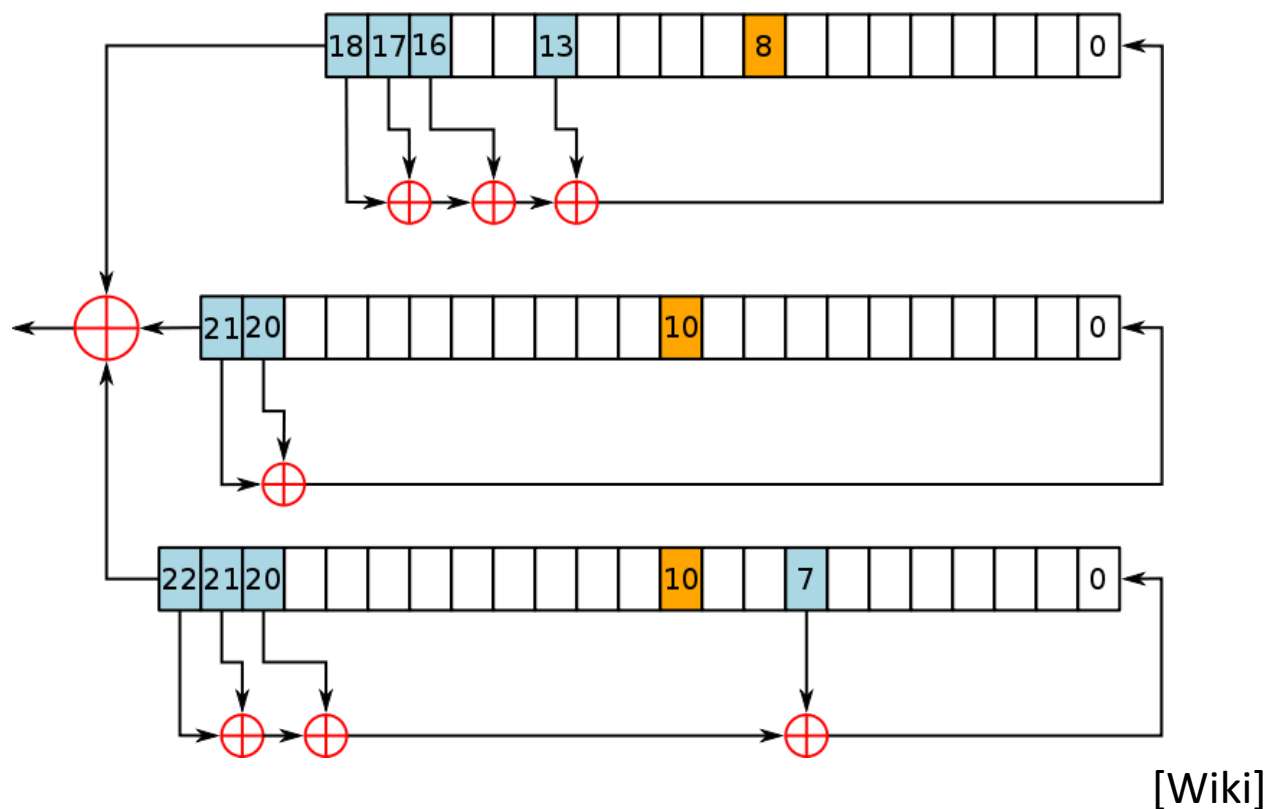
Stream Ciphers– A5

- Stream cipher used to encrypt GSM – the link between from the phone to base station
- Is a French design cipher
- Consists in 3 LFSRs (their length is 19, 22 and 23)
- It is very efficient and pass all known statistical test



Stream Ciphers– A5

- <http://en.wikipedia.org/wiki/A5/1>





- a proprietary cipher owned by RSA DSI
- another Ron Rivest design, simple but effective
- variable key size, byte-oriented stream cipher
- widely used (web SSL/TLS, wireless WEP)
- key forms random permutation of all 8-bit values
- uses that permutation to scramble input info processed a byte at a time



RC4 Key Schedule

- starts with an array S of numbers: $0..255$
- use key to well and truly shuffle
- S forms **internal state** of the cipher

```
for i = 0 to 255 do
    S[i] = i
    T[i] = K[i mod keylen])
j = 0
for i = 0 to 255 do
    j = (j + S[i] + T[i]) (mod 256)
    swap (S[i], S[j])
```



RC4 Encryption

- encryption continues shuffling array values
- sum of shuffled pair selects "stream key" value from permutation
- XOR $S[t]$ with next byte of message to en/decrypt

$i = j = 0$

for each message byte M_i

$i = (i + 1) \pmod{256}$

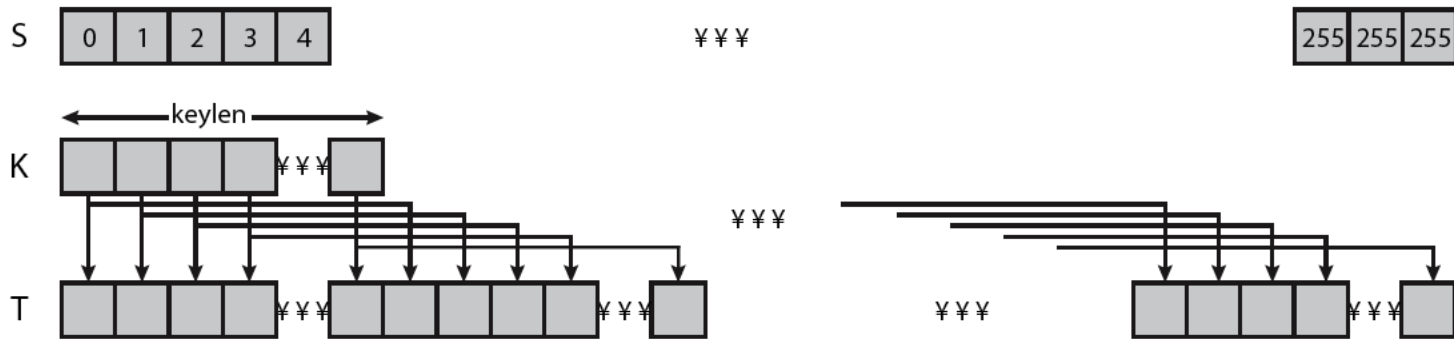
$j = (j + S[i]) \pmod{256}$

swap($S[i], S[j]$)

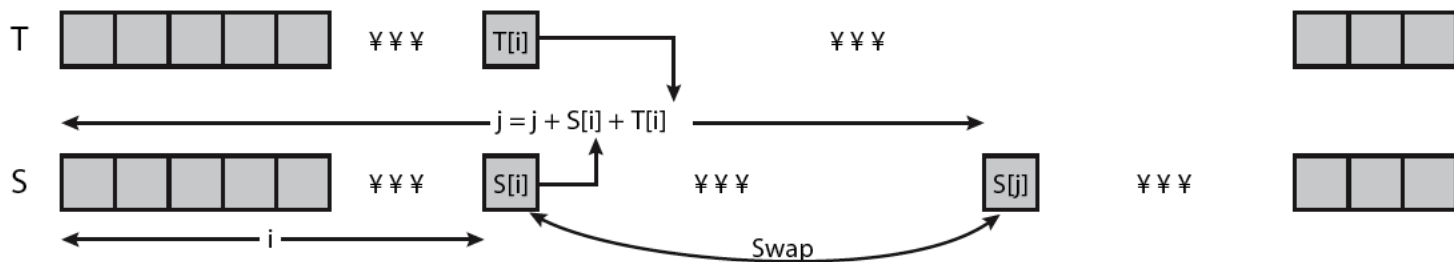
$t = (S[i] + S[j]) \pmod{256}$

$C_i = M_i \text{ XOR } S[t]$

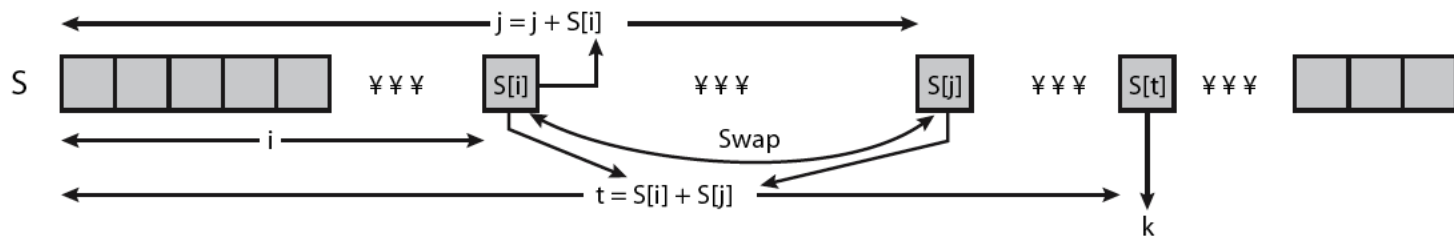
RC4 Overview



(a) Initial state of S and T



(b) Initial permutation of S



(c) Stream Generation



RC4 Security

- claimed secure against known attacks
 - have some analyses, none practical
- result is very non-linear
- since RC4 is a stream cipher, must **never reuse a key**
- have a concern with WEP, but due to key handling rather than RC4 itself

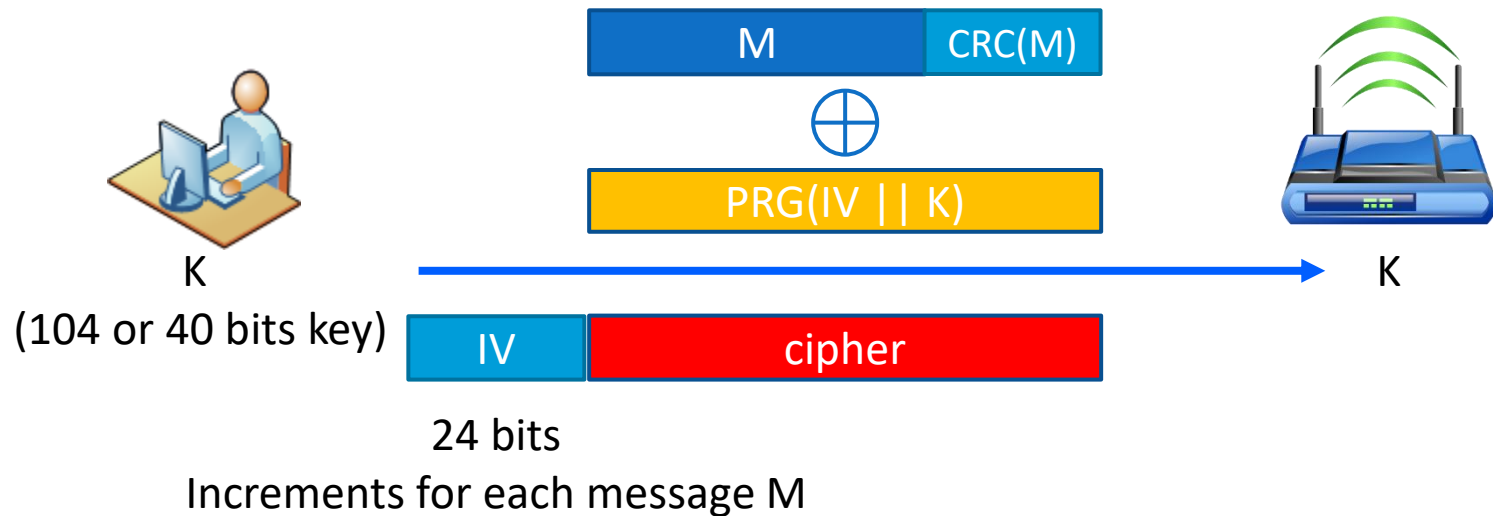


WEP problem

- 802.11 standard introduced in 1999 - Wired Equivalent Privacy (**WEP**)
- Used a key of 10 or 26 hexadecimal digits
- Standard 64-bit WEP uses a 40 bit key (also known as WEP-40), which is concatenated with a **24-bit initialization vector (IV)** to form the RC4 key
- User' 5 chars passwords reduced more the key strength
- because RC4 is a stream cipher, the same traffic key must never be used twice



WEP problem





WEP problem

- In 2001 they were able to recover the key in 10^6 frames
- Today 40000 frames are sufficient
- On some cards the IV is reset to 0 after a power cycle
- Because IV is generated incrementally, it repeats after 2^{24} values (between 5 and 7 hours on a busy network)
- The keys are related (differ only in 24 bits)



WPA2 problem

- [Wi-Fi Protected Access II \(WPA2\)](#) is a security algorithm for IEEE 802.11i wireless networks standard introduced in 2004
- replaces [Wired Equivalent Privacy \(WEP\)](#), which was shown to have security vulnerabilities
- Wi-Fi Protected Access (WPA) has been a temporary solution to WEP
- uses the [Advanced Encryption Standard](#) (AES) block cipher, whereas WEP and WPA used the [RC4](#) stream cipher



- On October 16, 2017, researchers revealed the KRACK (Key Reinstallation Attacks) exploit
- <https://www.krackattacks.com/>
- Allows attackers eavesdrop on traffic between computers and wireless access points
- The attack DOES NOT recover the password of the Wi-Fi network

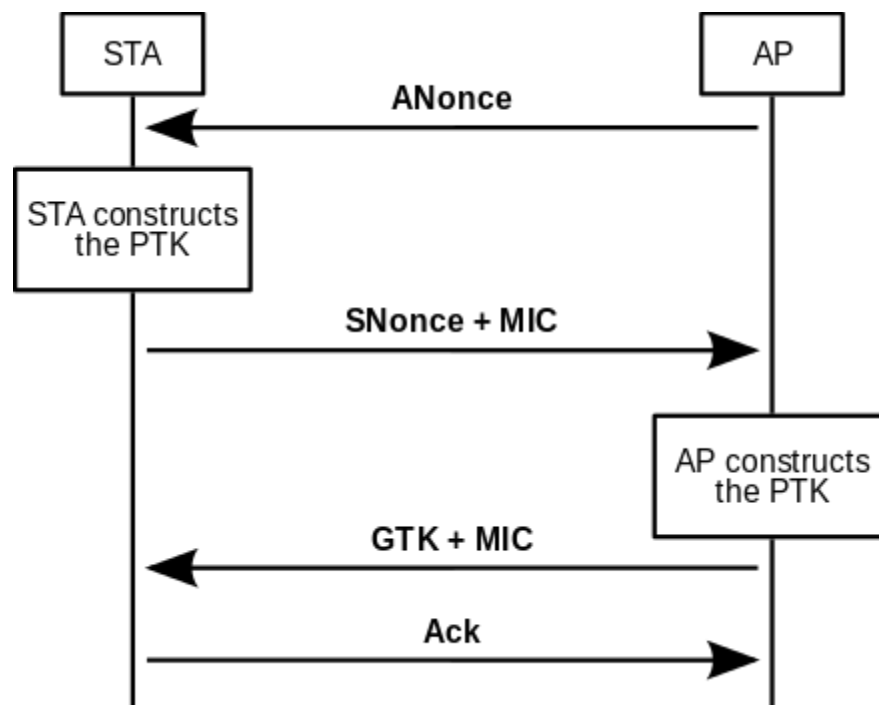


WPA2 problem

- The attacker can
 - inject and manipulate data (to inject ransomware or other malware into websites that you visit)
 - steal sensitive information such as credit card numbers, passwords, chat messages, emails, photos, and so on
 - read information that was previously assumed to be safely encrypted



WPA2 problem



[The four-way handshake in 802.11i](#)



WPA2 problem

- The 4-way handshake protocol
 - It allows clients and APs to check their pre-shared key
 - Generates a fresh encryption key that will be used to encrypt all subsequent traffic
-



- The attack is against the 4-way handshake of the WPA2 protocol by replaying cryptographic handshake messages
- The client will reinstall an already-in-use key and will reset its incremental transmit packet number (i.e. nonce) and receive packet number (i.e. replay counter)
- For Android (above 6.0) and Linux the client can be tricked to install an all-zero encryption key



- As with all stream ciphers you should NOT reuse the same key again
- ***the same encryption key is used with nonce values that have already been used in the past***
- The client will reuse keystream when encrypting packets and for known content it will be easy for an attacker to determine it



Complex ciphers

DES - Data Encryption Standard

AES – Advanced Encryption Standard



Complex ciphers

64 bits
(->1997)

- **Lucifer** (except 128 bits)
- **DES** – Data Encryption Standard
- **IDEA** – International Data Encryption Standard
- **FEAL** – Japanese Fast Data Encryption Algorithm
- **LOKI** – Australian symmetrical cipher
- **RC2** – Rivest Cipher

128 bits
(1997 ->)

- **AES** – Advanced Encryption Standard (Rijndael)
- **Twofish**
- **Serpent**
- **RC6**
- **MARS**
- **Blowfish**



- The 1st standard for data cryptographic protection
- Studied by IBM starting in 1970 for NBS (National Bureau of Standards)
- Published as FIPS PUBS 46 (Federal Information Processing Standards Publications)
- In 1977 is named DES and used until 1998 when it was hacked/cracked
- standard ANSI X3.92 and named DEA (Data Encryption Algorithm)



DES

(Data Encryption Standard)

- Symmetric block cipher
- Encrypts data in 64-bit blocks
- Key lengths of 64 bits – 56 bits random generated (or from password) and 8 bits for detecting transmission errors (each bits represents the odd parity of key's 8 octets)



DES Design Controversy

- although DES standard is public
- was considerable controversy over design
 - in choice of 56-bit key (vs Lucifer 128-bit)
 - and because design criteria were classified
- subsequent events and public analysis show in fact design was appropriate
- use of DES has flourished
 - especially in financial applications
 - still standardised for legacy application use



- Combines two encryption techniques: confusion and diffusion, a substitution followed by a permutation)
- A processing round: Feistel network with permutation between 2 blocks (32 bits) of initial message block and a substitution through f function that will become a nonreversible ff function due to Feistel network.



DES

(Data Encryption Standard)

- Made of 16 rounds
- Each round uses a different 48 key bits selected from an initial 56 key bits





DES

(Data Encryption Standard)

64 bits – block message



58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

IP
initial permutation



L_0



R_0



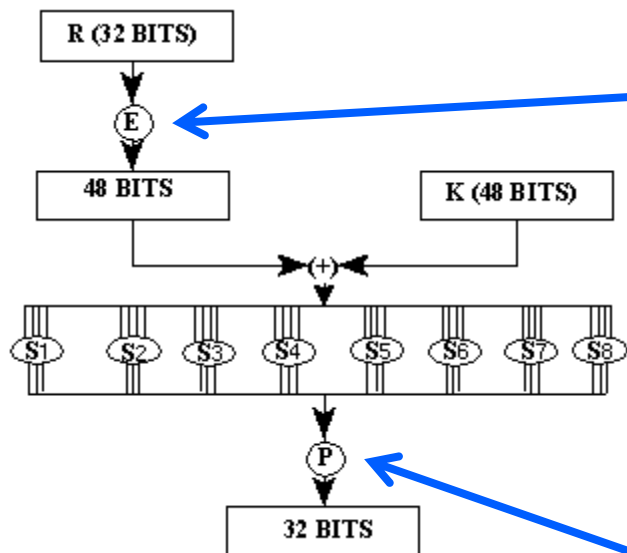
DES

(Data Encryption Standard)

- $L_i = R_{i-1}$
- $R_i = L_{i-1} \text{ xor } f(R_{i-1}, K_i)$
 - i is the round's number
 - K_n is each round's key ($1 \leq n \leq 16$)
 - $K_n = KS(n, KEY)$, where KS is the scheduling function of each round's keys (key scheduling)

DES

(Data Encryption Standard)



Generic f function $f(R,K)$

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

E BIT-SELECTION TABLE

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

P transformation

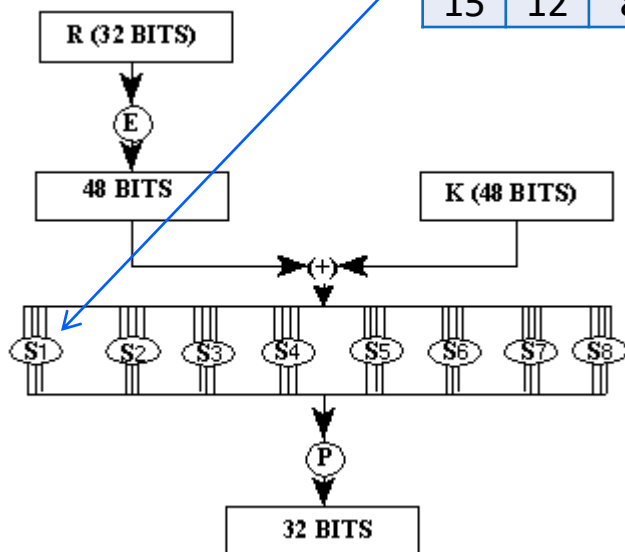


DES

(Data Encryption Standard)

V_{ij} - 4 bits values

14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13



B - 6 bits - $b_1b_2b_3b_4b_5b_6$

S1

L - 4 bits - $l_1l_2l_3l_4$

$L = S(B)$

$L = v[i][j],$

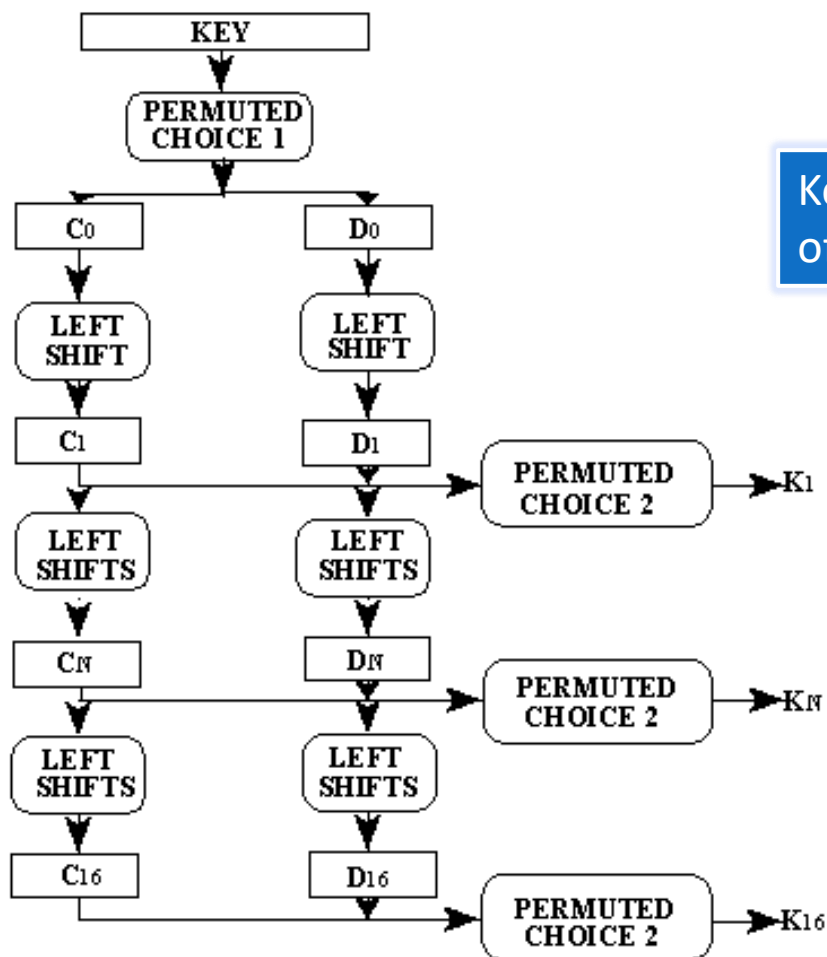
$i = (b_1b_6)_{10} \rightarrow \{0, 1, 2, 3\}$

$j = (b_2b_3b_4b_5)_{10} \rightarrow \{0, \dots, 15\}$

Boxes S DES.pfd

DES

(Data Encryption Standard)



Key scheduling function KS
of each round's keys

16 keys of 48 bits



DES

(Data Encryption Standard)

56 bits – initial key (+8 bits parity)



C_0	57	49	41	33	25	17	9
	1	58	50	42	34	26	18
	10	2	59	51	43	35	27
	19	11	3	60	52	44	36
D_0	63	55	47	39	31	23	15
	7	62	54	46	38	30	22
	14	6	61	53	45	37	29
	21	13	5	28	20	12	4

PC-1 Permuted Choice-1



DES

(Data Encryption Standard)

Iteration			Number of bits Shift-area cyclic on left
i =	1	<<<	1
i =	2	<<<	1
i =	3	<<<	2
i =	4	<<<	2
i =	5	<<<	2
i =	6	<<<	2
i =	7	<<<	2
i =	8	<<<	2
i =	9	<<<	1
i =	10	<<<	2
i =	11	<<<	2
i =	12	<<<	2
i =	13	<<<	2
i =	14	<<<	2
i =	15	<<<	2
i =	16	<<<	1

Shift-area cyclic on left ((a,b,c,d) shift-area cyclic on left => (b,c,d,a))



DES

(Data Encryption Standard)

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

PC-2 Permuted Choice-2



DES

(Data Encryption Standard)

Final round (16): $L_{16}=R_{16}$, $R_{16}=L_{16} \oplus f(R_{15}, K_{16})$



40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25



IP^{-1}
Inverse initial permutation

64 bits – cipher block



DES

(Data Encryption Standard)

Reverse cipher (decryption):

- use the same algorithm but with the keys K_i applied in reverse, from K_{16} to K_1 .
1. It is applied IP permutation, which undo the last step IP^{-1} , from the encryption operation.
 2. It is generated in reverse order : $R_{i-1}=L_i$, $L_{i-1}=R_i f(L_i, K_i)$; it starts from R_{16} and L_{16} generating in the end R_0 and L_0
 3. 64 bits block is submitted to a reverse permutation

IP^{-1}



AES – Rijndael

(Advanced Encryption Standard)

Criteria decided by NIST :

- A system of symmetric encryption based on 128 bits blocks
- Keys of 128, 192 and 256 bits length
- Does not contain weak keys
- Efficient on Intel platforms as well as other software or hardware platforms
- Able to be implemented on 32 bits processors and smart-cards (8 bits processors)
- Faster than DES and offering a higher security than 3DES



AES Requirements

- private key symmetric block cipher
- 128-bit data, 128/192/256-bit keys
- stronger & faster than Triple-DES
- active life of 20-30 years (+ archival use)
- provide full specification & design details
- both C & Java implementations
- NIST have released all submissions & unclassified analyses



AES Evaluation Criteria

- initial criteria:
 - security – effort for practical cryptanalysis
 - cost – in terms of computational efficiency
 - algorithm & implementation characteristics
- final criteria
 - general security
 - ease of software & hardware implementation
 - implementation attacks
 - flexibility (in en/decrypt, keying, other factors)



AES Shortlist

- after testing and evaluation, shortlist in Aug-99:
 - MARS (IBM) - complex, fast, high security margin
 - RC6 (USA) - v. simple, v. fast, low security margin
 - Rijndael (Belgium) - clean, fast, good security margin
 - Serpent (Euro) - slow, clean, v. high security margin
 - Twofish (USA) - complex, v. fast, high security margin
- then subject to further analysis & comment
- saw contrast between algorithms with
 - few complex rounds verses many simple rounds
 - which refined existing ciphers verses new proposals



- finalist and winner of AES contest launched by NIST 1997
- creation finalized by Belgians Joan Daemen and Vincent Rijman
- became standard from 2000 (FIPS PUB 197)
- uses 128, 192 or 256 bits keys
- It is an symmetric cryptographic algorithm
- processes blocks of 128, 192 or 256 bits



AES – Rijndael

(Advanced Encryption Standard)

- Some operations are defined at byte level, and bytes are represented in limited field $GF(2^8)$ - Galois Field (256)
- An integer value y , $0 \leq y \leq 255$ can be polynomial represented:
$$y = b_7 x^7 + b_6 x^6 + b_5 x^5 + b_4 x^4 + b_3 x^3 + b_2 x^2 + b_1 x + b_0$$
- The 2 operations from $GF(2^8)$: sum and product



AES – Rijndael

(Advanced Encryption Standard)

- Addition is equivalent with XOR (OR exclusive) on bits or the addition modulo 2;
- The set $\{0...255\}$ together with XOR operation forms an abelian group (the operation is internal, associative, commutative, the neutral element is '00', there is the reverse element—the element itself is his reverse)



- In polynomial representation, multiplication – in $GF(2^8)$ corresponds to the multiplication of 2 polynoms modulo with an irreducible of 8 level
- for Rijndael, the 8 rank irreducible polynom is named $m(x)$ and has the form: $m(x) = x^8 + x^4 + x^3 + x + 1$, meaning $11B_{16}$ or 283_{10}



AES – Rijndael

(Advanced Encryption Standard)

- In programming the multiplication of 2 numbers in $GF(2^8)$ is made as the exponential sum of 2 logarithms (**discrete logarithms or logarithms in discrete field**) based on a prime number in $GF(2^8)$
- If the prime number is $x + 1$ then $a * b = c = (x+1)^* (x+1)^* \dots * (x+1)$ multiplied by $\log_3(a) + \log_3(b)$ times



AES – Rijndael

(Advanced Encryption Standard)

- it operates with registries or data blocks of 32 bits (4 bytes)
- for abstractisation of the mathematical polynomial are defined ***operations with coefficients in $GF(2^8)$*** - a vector of 4 octets corresponds to a polynomial expression of lesser level than 4 with coefficients of byte type (8 bits)



AES – Rijndael

(Advanced Encryption Standard)

- The addition of coefficient polynomial expression is made by the mere addition of the coefficients (the coefficients are seen as polynomial expression, a coefficient has 8 bits –a byte) meaning exclusive or (xor) between coefficients.
- At multiplication, for Rijndael the irreducible polynomial is $M(x) = x^4+1$



AES – Rijndael

(Advanced Encryption Standard)

- $a(x) * b(x) = (a(x) * b(x)) \pmod{M(x)} = a(x) b(x) = c(x) \pmod{M(x)} = d(x)$
- $d(x) = d_3x^3 + d_2x^2 + d_1x + d_0$ where

$$d_0 = a_0 * b_0 \oplus a_3 * b_1 \oplus a_2 * b_2 \oplus a_1 * b_3$$

$$d_1 = a_1 * b_0 \oplus a_0 * b_1 \oplus a_3 * b_2 \oplus a_2 * b_3$$

$$d_2 = a_2 * b_0 \oplus a_1 * b_1 \oplus a_0 * b_2 \oplus a_3 * b_3$$

$$d_3 = a_3 * b_0 \oplus a_2 * b_1 \oplus a_1 * b_2 \oplus a_0 * b_3$$



$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$



- To be resistant against all known attacks
- To be implemented on a whole series of platforms
- To prove high computational speed
- Designing and implementing to be as simple as possible

- To be resistant against all known attacks
- To be implemented on a whole series of platforms
- To prove high computational speed
- Designing and implementing to be as simple as possible



AES – Rijndael

(Advanced Encryption Standard)

The architecture of Rijndael algorithm

- The round function IS NOT implemented by a Feistel network (structure) (DES, Twofish, Serpent)
- *The round function* – round transformation is composed by three different transformations, discrete, uniform and reversible called layers
- *State* – bits array taken into the algorithm or the bits array as an intermediary result of the encryption



Layers:

- The linear mixing layer
 - Ensures a great diffusion of the bits along the multiple rounds of the algorithm
 - Realized by the functions *ShiftRow* and *MixColumn*



AES – Rijndael

(Advanced Encryption Standard)

Layers:

- The non-linear layer
 - More parallel S boxes that produces bits combination in a non-linear way
 - Implemented by *ByteSub*



AES – Rijndael

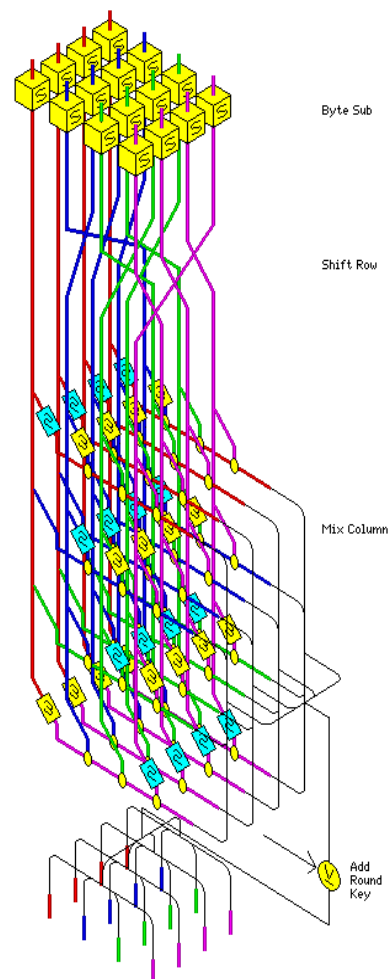
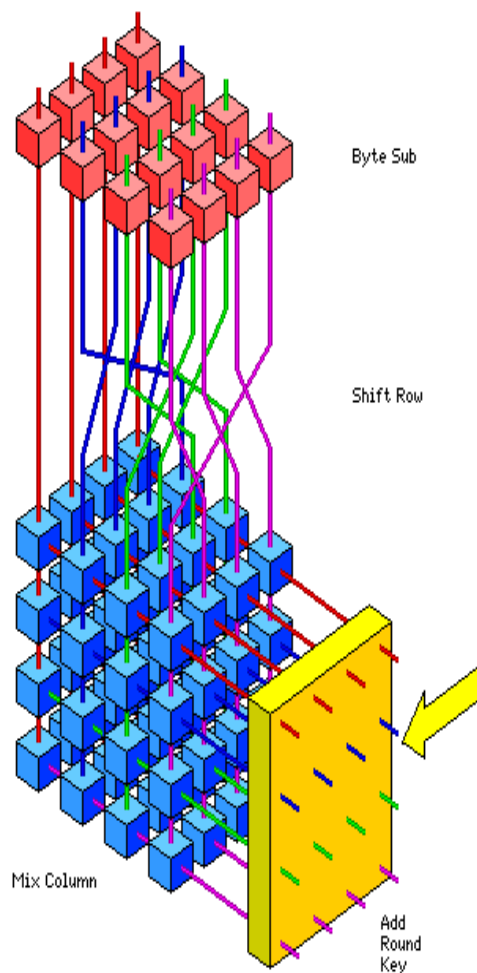
(Advanced Encryption Standard)

Layers:

- The key addition layer:
 - Executes XOR on bits between the key from a round (generated by user's original key) and bits from *State*
 - Implemented through function *AddRoundKey*

AES – Rijndael

(Advanced Encryption Standard)





AES – Rijndael

(Advanced Encryption Standard)

```
Rijndael (State, CipherKey) {  
    KeyExpansion (CipherKey, ExpandedKey) ;  
    AddRoundKey (State, ExpandedKey) ;  
    for ( i=1 ; i<Nr ; i++ )  
        Round (State, ExpandedKey + Nb*i) ;  
    FinalRound (State, ExpandedKey + Nb*Nr) ;  
}
```



AES – Rijndael

(Advanced Encryption Standard)

```
Round(State, RoundKey) {  
    ByteSub(State);  
    ShiftRow(State);  
    MixColumn(State);  
    AddRoundKey(State, RoundKey);  
}  
  
FinalRound(State, RoundKey) {  
    ByteSub(State);  
    ShiftRow(State);  
    AddRoundKey(State, RoundKey);  
}  
  
AddRoundKey(State, ExpandedKey) {  
    State = (State ^ ExpandedKey);  
}
```



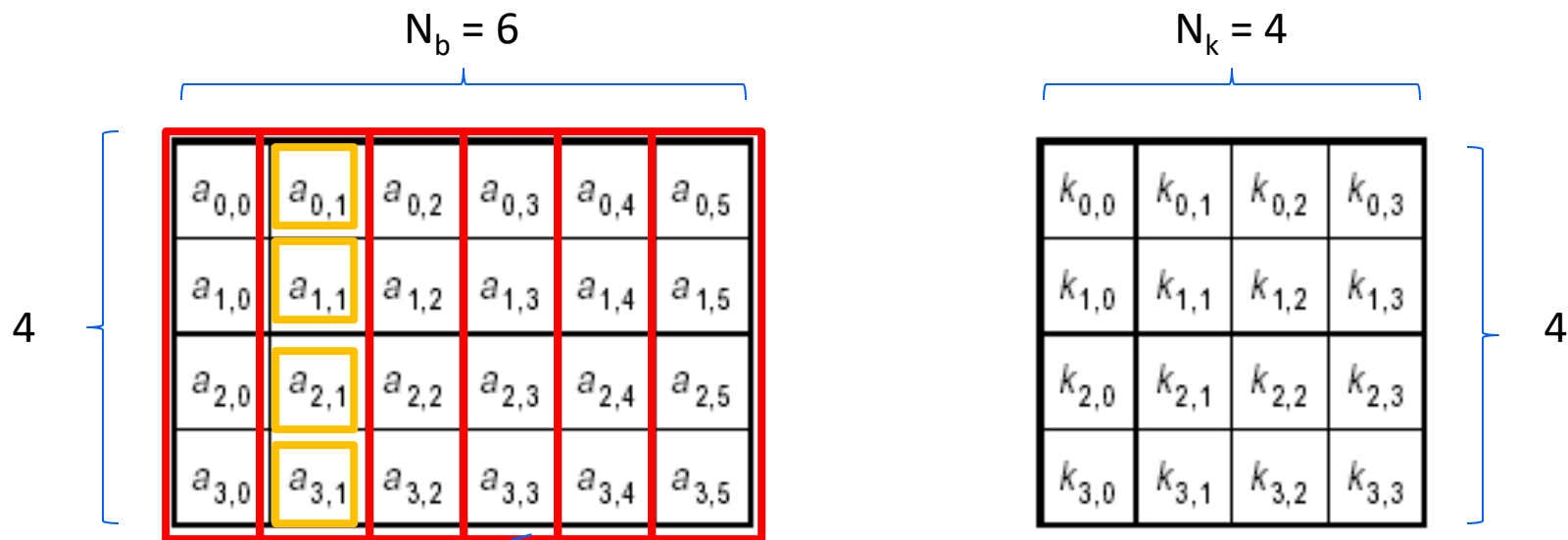

AES – Rijndael

(Advanced Encryption Standard)

- Iterative cipher with variable length of the clear message block and of the initial key (128, 192 or 256 de bits)
- *The State* – is a byte matrix that has $N_k=4$ lines and a number of columns called $N_b=4 / 6 / 8$ according to the length of the State 128 / 192 / 256 bits
- The initial key is represented in the same way



AES – Rijndael (Advanced Encryption Standard)



$$4 * N_b * 8 \text{ biți} = 192 \text{ biți}$$

State

$$N_k * 4 * 8 \text{ bits} = 128 \text{ bits}$$

Key

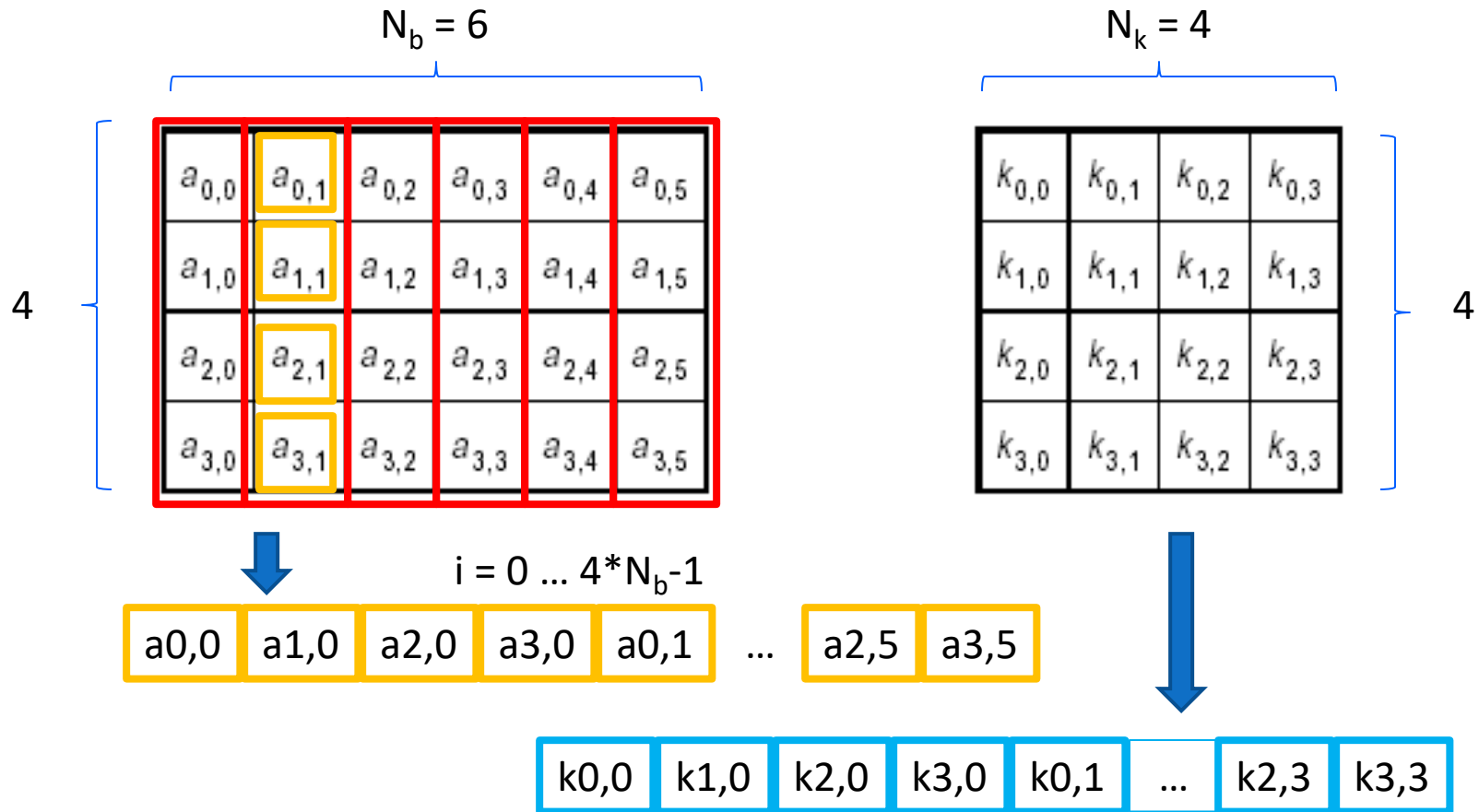
vector with 4 octets and coefficients

vector with 6 elements

every coefficient is a simple polynom – from $GF(2^8)$



AES – Rijndael (Advanced Encryption Standard)





AES – Rijndael

(Advanced Encryption Standard)

Number of rounds $N_r =$

N_r	$N_b = 4$	$N_b = 6$	$N_b = 8$
$N_k = 4$	10	12	14
$N_k = 6$	12	12	14
$N_k = 8$	14	14	14



AES – Rijndael

(Advanced Encryption Standard)

The function (transformation) ByteSub

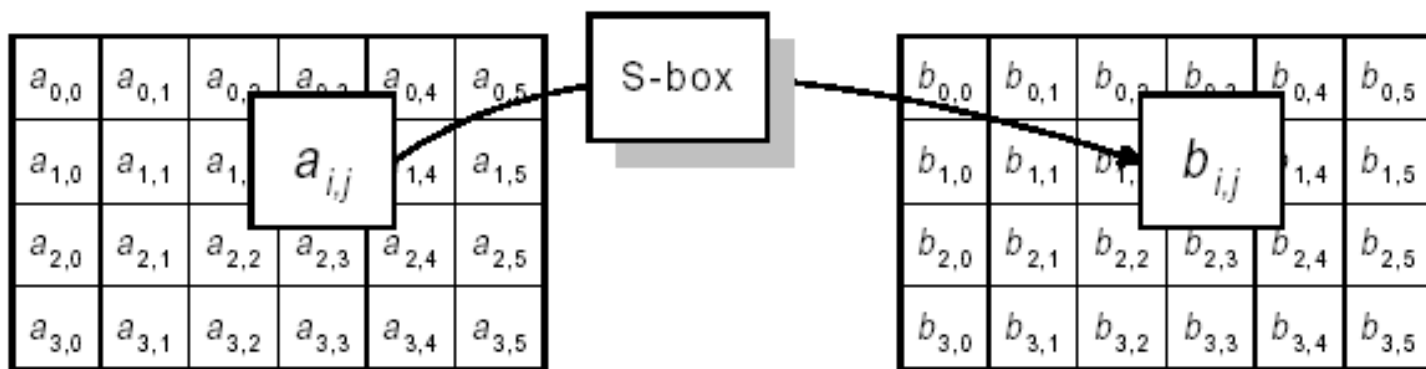
- Non-linearly substitution (is a S-box)
- Is executed in 2 steps:
 1. The byte is taken and is obtained the invert multiplication for the B byte
 2. Is applied an “affine” transformation given by:

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$



AES – Rijndael (Advanced Encryption Standard)

Function (transformation) ByteSub





AES – Rijndael

(Advanced Encryption Standard)

Function (transformation) ShiftRow

- Applies a cyclic rotation to the right the State
- line 0 is not moved; line 1 moved by C1 bytes; line 2 is moved by C2 bytes and line 3 by C3 bytes

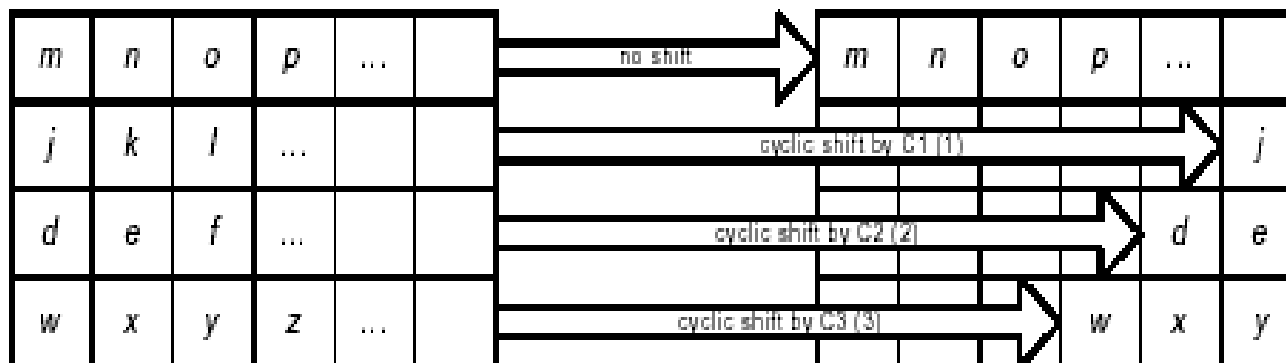
Nb	C1	C2	C3
4	1	2	3
6	1	2	3
8	1	3	4



AES – Rijndael

(Advanced Encryption Standard)

Function (transformation) ShiftRow





AES – Rijndael

(Advanced Encryption Standard)

Function (transformation) MixColumn

- The columns are considered words of 4 bytes (polynomial expression with coefficients over $GF(2^8)$)
- Each column in the State is multiplied modulo (x^4+1) with a polynomial expression $c(x)$ given by $c(x) = '03' x^3 + '01' x^2 + '01' x + '02'$
- The polynomial expression $c(x)$ is relatively prime with x^4+1



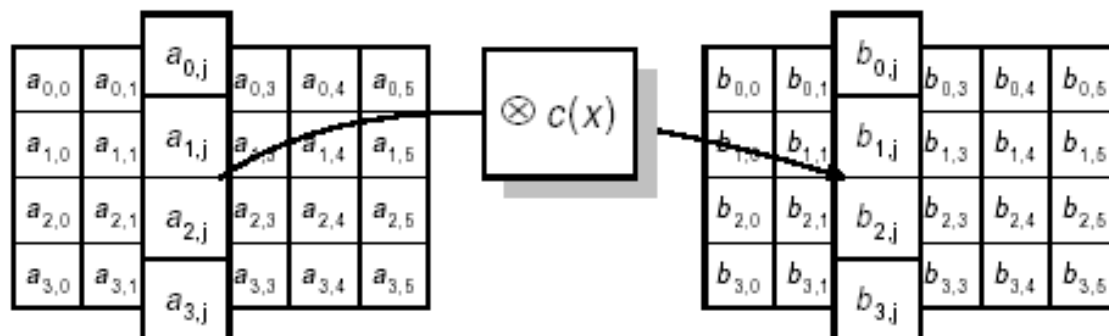
AES – Rijndael

(Advanced Encryption Standard)

Function (transformation) MixColumn

- This transformation can be written in a matrix form as well ($b(x) = c(x) \text{ AND } a(x)$)

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}$$





AES – Rijndael

(Advanced Encryption Standard)

Function (transformation) AddRoundKey

- State block is made XOR with the block of current key in a round

$$\begin{array}{|c|c|c|c|c|c|} \hline s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} & s_{0,4} & s_{0,5} \\ \hline s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} & s_{1,4} & s_{1,5} \\ \hline s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} & s_{2,4} & s_{2,5} \\ \hline s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} & s_{3,4} & s_{3,5} \\ \hline \end{array} \oplus \begin{array}{|c|c|c|c|c|c|} \hline k_{0,0} & k_{0,1} & k_{0,2} & k_{0,3} & k_{0,4} & k_{0,5} \\ \hline k_{1,0} & k_{1,1} & k_{1,2} & k_{1,3} & k_{1,4} & k_{1,5} \\ \hline k_{2,0} & k_{2,1} & k_{2,2} & k_{2,3} & k_{2,4} & k_{2,5} \\ \hline k_{3,0} & k_{3,1} & k_{3,2} & k_{3,3} & k_{3,4} & k_{3,5} \\ \hline \end{array} = \begin{array}{|c|c|c|c|c|c|} \hline b_{0,0} & b_{0,1} & b_{0,2} & b_{0,3} & b_{0,4} & b_{0,5} \\ \hline b_{1,0} & b_{1,1} & b_{1,2} & b_{1,3} & b_{1,4} & b_{1,5} \\ \hline b_{2,0} & b_{2,1} & b_{2,2} & b_{2,3} & b_{2,4} & b_{2,5} \\ \hline b_{3,0} & b_{3,1} & b_{3,2} & b_{3,3} & b_{3,4} & b_{3,5} \\ \hline \end{array}$$



AES – Rijndael

(Advanced Encryption Standard)

Key schedule:

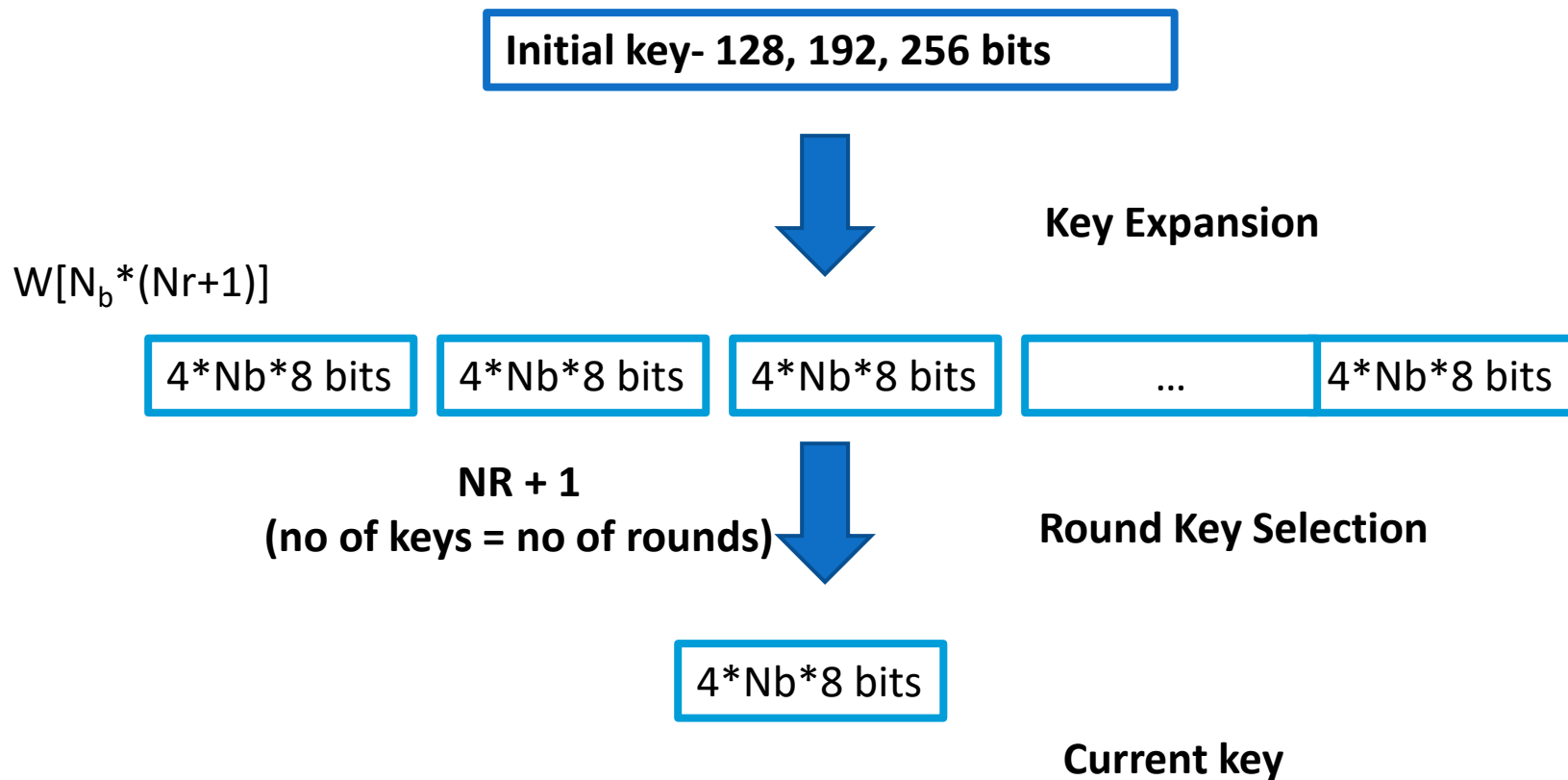
- **Key Expansion**
- **Round Key Selection**

Characteristics:

- Total number of Round Key bits = $N_k * 4 * (\text{No of rounds} + 1)$
- The key is expended in Expanded Key
- every *Round Key* is obtained from groups of Nb words of 4 bytes



AES – Rijndael (Advanced Encryption Standard)





AES – Rijndael

(Advanced Encryption Standard)

KeyExpansion process

- The expended key is a vector whose elements are 4 bytes words called $W[Nb*(Nr+1)]$
- The first Nk words of the expended key are represented by the original key memorized in $Key[4*Nk]$
- The KeyExpansion function is dependent on Nk

AES – Rijndael



AES – Rijndael

(Advanced Encryption Standard)

SubByte(W) returns a word of 4 bytes taking each byte from the input word through a Rijndael S-box

RotByte(W) – returns a word for which the bytes are cyclically rotated so that if the word is composed of bytes (a,b,c,d) the result is (b,c,d,a)

Rcon – N_k independent constants



AES – Rijndael

(Advanced Encryption Standard)

- $Rcon[i] = (RC[i], '00', '00', '00')$
- $RC[i]$ representing an element from $GF(2^8)$ that has the value $x^{(i-1)}$
- $RC[1] = 1; RC[2] = x; RC[3] = x^2$ so $RC[i] = x * RC[i-1] = x^{(i-1)}$

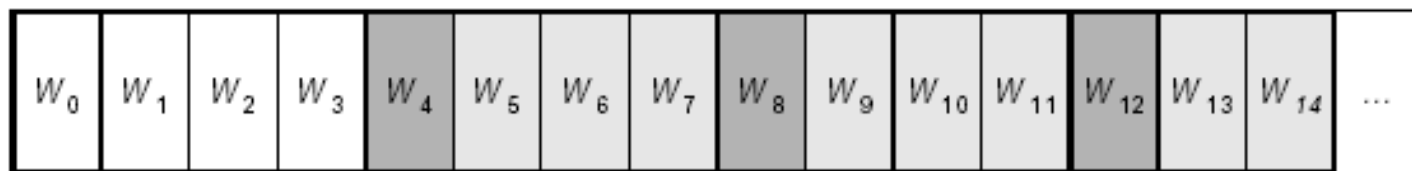


AES – Rijndael

(Advanced Encryption Standard)

RoundKeySelection process

- The key of each round I is included by W between the positions $W[Nb*i]$ and $W[Nb*(i+1)]$





AES – Rijndael

(Advanced Encryption Standard)

Revert cipher:

```
InvRijndael (State, CipherKey) {  
    KeyExpansion (CipherKey, ExpandedKey) ;  
    InvFinalRound (State, ExpandedKey + Nb*Nr) ;  
    for (i=1; i<Nr; i++)  
        InvRound (State, ExpandedKey + Nb*i) ;  
    AddRoundKey (State, ExpandedKey) ;  
}
```



AES – Rijndael

(Advanced Encryption Standard)

```
InvRound(State, RoundKey) {  
    AddRoundKey(State, RoundKey);  
    InvMixColumn(State);  
    InvShiftRow(State);  
    InvByteSub(State);  
}  
  
InvFinalRound(State, RoundKey) {  
    AddRoundKey(State, RoundKey);  
    InvShiftRow(State);  
    InvByteSub(State);  
}  
  
AddRoundKey(State, ExpandedKey) {  
    State = (State ^ ExpandedKey);  
}
```



AES – Rijndael

(Advanced Encryption Standard)

- **InvByteSub** is obtained by applying the reverse table of transformation
- **InvShiftRow** supposed the rotation of last 3 lines in the matrix which memorizes the State like this:
 - line 0 is not rotated
 - line 1 with $Nb-C1$
 - line 2 with $Nb-C2$
 - line 3 with $Nb-C3$ bytes
 - i.e. byte j in line l is moved to the position $(j+Nb-Ci) \bmod Nb$ in the linear representation of the matrix in memory



AES – Rijndael

(Advanced Encryption Standard)

- **InvMixColumn**, supposes that each column in the State is multiplied with $c(x)$ reverse polynomial, given by $d(x)$:

$$('03'x^3 + '01'x^2 + '01'x + '02') \text{ AND } d(x) = '01' \Rightarrow \\ d(x) = '0B' x^3 + '0D' x^2 + '09' x + '0E'$$



Implementation Aspects

- can efficiently implement on 8-bit CPU
 - byte substitution works on bytes using a table of 256 entries
 - shift rows is simple byte shift
 - add round key works on byte XOR's
 - mix columns requires matrix multiply in $GF(2^8)$ which works on byte values, can be simplified to use table lookups & byte XOR's



Implementation Aspects

- can efficiently implement on 32-bit CPU
 - redefine steps to use 32-bit words
 - can precompute 4 tables of 256-words
 - then each column in each round can be computed using 4 table lookups + 4 XORs
 - at a cost of 4Kb to store tables
- designers believe this very efficient implementation was a key factor in its selection as the AES cipher



Multiple encryptions systems

- Ways to combine block algorithms to get new algorithms
- Increase security by others ways then designing a new algorithm
- Combining techniques
 - *Multiple encryption* – using an algorithm to encrypt the same plaintext multiple times with multiple keys
 - *Cascading* - using different algorithms



Multiple encryptions systems

Double encryption:

- Encryption with 2 keys:
 - Encrypts with 2 different keys
 - $C = E_{K1}(E_{K2}(P)); P = D_{K1}(D_{K2}(C));$
- Encryption on Davies-Price method:
 - CBC variant
 - $C_i = E_{K1}(P_i \oplus E_{K2}(C_{i-1})); P_i = D_{K1}(C_i) \oplus D_{K2}(C_{i-1});$



- Triple encryption with 2 keys

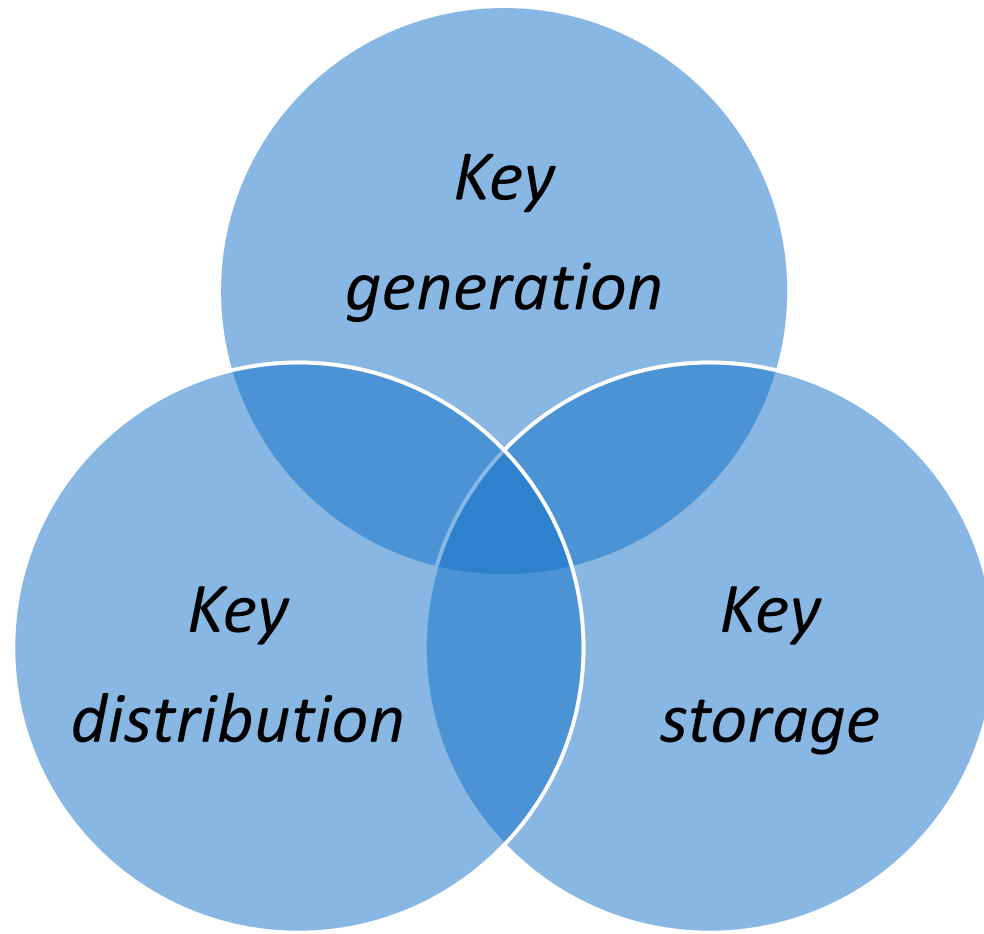
- $C = E_{K_1}(D_{K_2}(E_{K_1}(P)))$; $P = D_{K_1}(E_{K_2}(D_{K_2}(C)))$;
- EDE – encrypt-decrypt-encrypt

- Triple encryption with 3 keys

- $$- C = E_{K3}(D_{K2}(E_{K1}(P))); P = D_{K1}(E_{K2}(D_{K3}(C)));$$



The security of symmetric encryption





What is the key-length ?

- it depends on what is the value of the data
- it depends on the use span of the cryptographic algorithm

Do not reduce the keyspace

- a 64 bit key with only lowercase-letters -> 2^{40} possible keys



Key generation

- random keys – for a 64 bits key, every possible 64 bit value must be equally likely
- pass phrases – take entire phrases and convert them in keys using hash functions
- ANSI X9.17 standard for random key generation



Key storage

Lifetime of keys:

- the longer a key is used, the greater the chance that it will be compromised
- the longer a key is used, the greater the loss if the key is compromised
- the longer a key is used, the greater the temptation for someone to break it
- cryptanalysis gives better results with more ciphertext encrypted with the same key



Asymmetric encryption systems (with public keys)

RSA

Merkle-Hellman



Asymmetric encryption systems

- 1976 – idea of public key encryption system, Diffie and Hellman (key distribution design)
- Probably most significant advance in the 3000 year history of cryptography
- Use distinct keys of encryption and decryption (but dependent on each other)
- It is impossible to extract a key from the other key
- One of the keys is made public, being available for everyone wishing to send an encrypted message.
- The receiver owning the 2nd key, can decrypt and use the message
- The public key technique is also used in digital (electronic) signature



Asymmetric encryption systems

Objective:

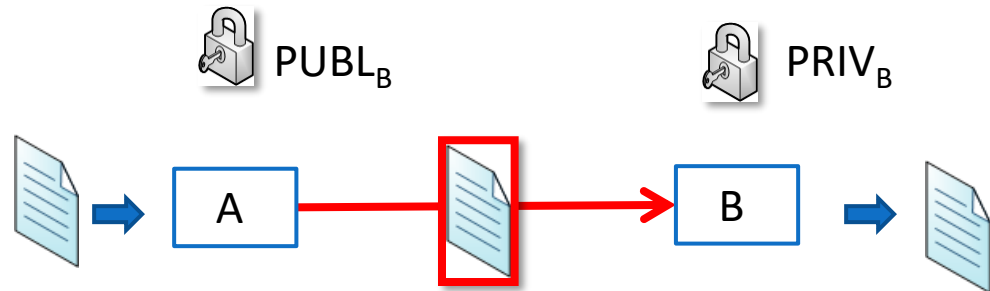
Correct vulnerabilities of symmetric encryption algorithms

- **key distribution**
- **non-repudiation**
- **authentication**



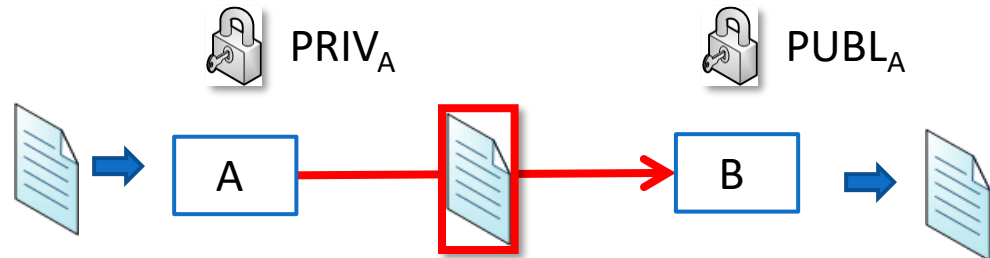
Asymmetric encryption systems

Confidentiality

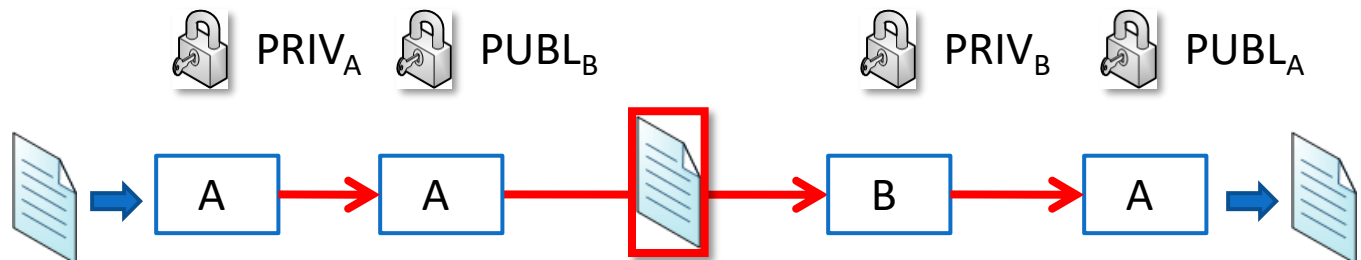


Digital signature course

Authentication (digital signature)



Authentication & Confidentiality



ENCRYPT

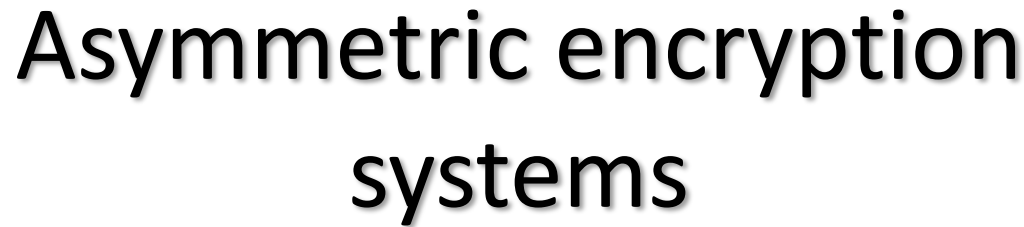
DECRYPT



Asymmetric encryption systems

Concepts used:

- Complex problems (NP class)
- Prime numbers
- factorization
- Discrete logarithms(multiplication in finite field $GF(2^8)$ from Rijndael
- Knapsack problem
- Lattice computation
- elliptic curve cryptography (ECC)



[https://en.wikipedia.org/wiki/Trapdoor_function]

- if f is a trapdoor function, then there exists some secret information t , such that given $f(x)$ and t , it is easy to compute x
- Concept promoted in mid-1970 by Diffie, Hellman, and Merkle ([Diffie & Hellman \(1976\)](#))



Asymmetric encryption systems

Algorithms:

- **RSA** (Rivest-Shamir-Adleman), **Rabin** – factorization problem
- **Diffie-Hellman**, **El Gamal**, **DSS**, **LUC**, **XTR** – based on discrete logarithms
- **BrandStorm**, **PIEPRZYK** – based on equations in finite fields, work with polynomial rings
- **Miller and Kobitz** – based on elliptic curves



- **Rivest-Chor, MH** – Merkle-Hellman, variants:
 - iterative variant
 - with additive trapdoor
 - with multiplicative trapdoor
- **GS** (Graham-Shamir), **SH** (Shamir) – based on Knapsack problem
- **NTRU** – based on lattice operations
(http://en.wikipedia.org/wiki/Lattice_order)
- **Elliptic curves**
(https://en.wikipedia.org/wiki/Elliptic_curve)



- Based on knapsack problem (NP complete problem with a $O(2^{n/2})$ complexity)
- *Simple knapsack* – a special class of problems that can be solved in linear time (polynomial)
 - C is a positive integer
 - $A=(a_1, a_2, \dots, a_n)$ is a vector of positive integers
 - **elements a_i ($i=1, \dots, n$) are in dominant relationship: $a_i > \sum_{j=1, i} (a_j)$**
 - Find an A subset whose sum to be C



Merkle-Hellman with additive trapdoor

- MH algorithm with additive trapdoor converted *simple knapsack* in *trapdoor knapsack*
 - It is selected a vector *simple knapsack* $A'=(a'_1, a'_2, \dots, a'_m)$ which allows a simple output of the problem, $C'=A'*M$ (M is the binary solution)
 - It is selected an integer n such that:
$$n \geq 2a'_m \geq \text{SUM}(a_j)_{i=1,m}$$
 - It is selected another integer w ($w < n$), such that $\text{gcd}(n, w) = 1$



- $$C' = (w^{-1}C) \bmod n = (w^{-1}A^*M) \bmod n = (w^{-1}(wA'))^*M \bmod n = A'^*M \bmod n = A'^*M$$



Merkle-Hellman with additive trapdoor

- The encryption E_A (public) (ensures the confidentiality) uses the public key represented by “heavy knapsack” A .

$$C = E_A(M) = A * M$$

- The decryption D_A uses the secret key $(A', n, w-1)$, calculated based on the function “knapsack simple”:

$$D_A(C) = \text{SimpleKnapsack}(w^{-1}C \bmod n, A', M) = M$$



El Gamal

- Presented for the 1st time in 1985 by Taher ElGamal
- Derived from Diffie and Hellman's keys distribution schema
- Funds its cryptographic strengths on difficulty to calculate logarithms in large Galois fields, discrete logarithm problem



El Gamal

Key generation:

- It is generated a large prime number and an α generator for Z_p group of modulo p integers
- It is selected a random number a , with $1 \leq a \leq p-2$ and it is calculated $\alpha^a \bmod p$
- Public key is (p, α, α^a)
- Private key is a



- It is obtained the public key(p, α, α^a)
- It is considered message m represented as integer value in $\{0, 1, \dots, p-1\}$
- It is chosen a random integer value k , with $1 \leq k \leq p - 2$.
- It is calculated $\gamma = \alpha^k \bmod p$ si $\delta = m \cdot (\alpha^a)^k \bmod p$
- Cipher is $c = (\gamma, \delta)$



- Private key is **a**
- Cipher is **c** = $(\gamma, \delta) = (\alpha^k \bmod p, m \cdot (\alpha^a)^k \bmod p)$
- Using key **a** it is calculated $\gamma^{p-1-a} \bmod p$ because $\gamma^{p-1-a} = \gamma^{-a} = \alpha^{-ak}$
- **m** is obtained calculating $(\gamma^{-a}) \cdot \delta \bmod p$
- It is possible because $\gamma^{-a} \cdot \delta \equiv \alpha^{-ak} m \alpha^{ak} \equiv m \pmod{p}$.



RSA

(Rivest-Shamir-Adleman)

- Created by 3 researchers from MIT (Massachusetts Institute of Technology)
- “de facto” standard in digital signature field and of encryption with public keys
- The safest method to secure and authenticate commercially available



RSA

(Rivest-Shamir-Adleman)

- Is based on the present quite impossibility to factorize very large integer numbers
- The encryption/decryption functions are of exponential type, where the exponent is the key and the calculation is made in the ring of rest modulo n classes
- Can be used for electronic signature as well as encryption/decryption



- 405



RSA

(Rivest-Shamir-Adleman)

The cryptographic system's parameters are:

- **p** and **q** are 2 very big prime numbers (secret, eventually known only by the owner, of minimum 1024 bits).
- The module **n**, made public, is **$n = p * q$** .
- The Euler indicator **$\phi(n) = (p-1) * (q-1)$** , impossible to be determined by an attacker, because its prime factors of **n** (**p** and **q**) are not known.
- The secret key, **PRIV**, chosen as being a big integer number relatively prime with **$\phi(n)$** , preferable in the period **$[\max(p,q)+1, n-1]$** .
- The public key, **PUB**, an integer calculated by a version of the algorithm of Euclid, as being inverse of **PRIV** modulo **$\phi(n)$** ; **$PUB = \text{inv}(\text{PRIV}, \phi(n))$** .
- **M** the document in electronic form (the file), the message, the object
- **H(M)**, the digest of the document, calculated with a hash dispersion function



RSA

(Rivest-Shamir-Adleman)

RSA configuration:

1. Are generated 2 big prime numbers p, q ;
2. It is calculated $n = p * q$ and $\phi(n) = (p - 1)(q - 1)$;
3. It is chosen a random number b , ($1 < b < \phi(n)$) with $\gcd(b, \phi(n)) = 1$
4. It is calculated $a = b^{-1} \bmod (\phi(n))$
5. Make public n and a . Keep private b .



RSA

(Rivest-Shamir-Adleman)

RSA encryption:

1. The intention is to send message M (simplification 1 byte)
2. It is calculated $C = M^b \bmod n$
3. It is send C



RSA

(Rivest-Shamir-Adleman)

RSA decryption:

1. It is received message C (simplification 1 byte)
2. It is calculated $M = C^a \bmod n$

Decryption is possible because

$$\begin{aligned} M &= C^a \bmod n = (M^b)^a \bmod n = M^{ba} \bmod n = \\ &= M^{b^{b-1 \bmod(n)}} \bmod n = M^1 \bmod n = M \end{aligned}$$



RSA Example - Key Setup

1. Select primes: $p=17$ & $q=11$
2. Compute $n = pq = 17 \times 11 = 187$
3. Compute $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$
4. Select e : $\gcd(e, 160) = 1$; choose $e=7$
5. Determine d : $de=1 \pmod{160}$ and $d < 160$
Value is $d=23$ since $23 \times 7 = 161 = 10 \times 160 + 1$
6. Publish public key **PU** = { 23, 187 }
7. Keep secret private key **PR** = { 7, 187 }



RSA Example - En/Decryption

- sample RSA encryption/decryption is:
- given message $M = 88$ (nb. $88 < 187$)
- encryption:

$$C = 88^7 \bmod 187 = 11$$

- decryption:

$$M = 11^{23} \bmod 187 = 88$$



RSA

(Rivest-Shamir-Adleman)

Analysis:

- function $E_k(x) = x^b \bmod n$ is not reversible because of its complexity
- Secret trapdoor owned by the receiver for decryption is factorization $n = p * q \rightarrow$ determines $\phi(n) \rightarrow$ determines a
- p and q are big numbers



Analysis:

- The effort of encryption/decryption depends on number of bits, k , of n ($k = \log_2(n) + 1$)
- Sum of 2 numbers of k bits - $O(k)$;
- Product of 2 numbers of k bits - $O(k^2)$
- Number $x * y \bmod n$ - $O(k^2)$
- Modular exponentiation $x^y \bmod n$ - $O(k^2)$



RSA

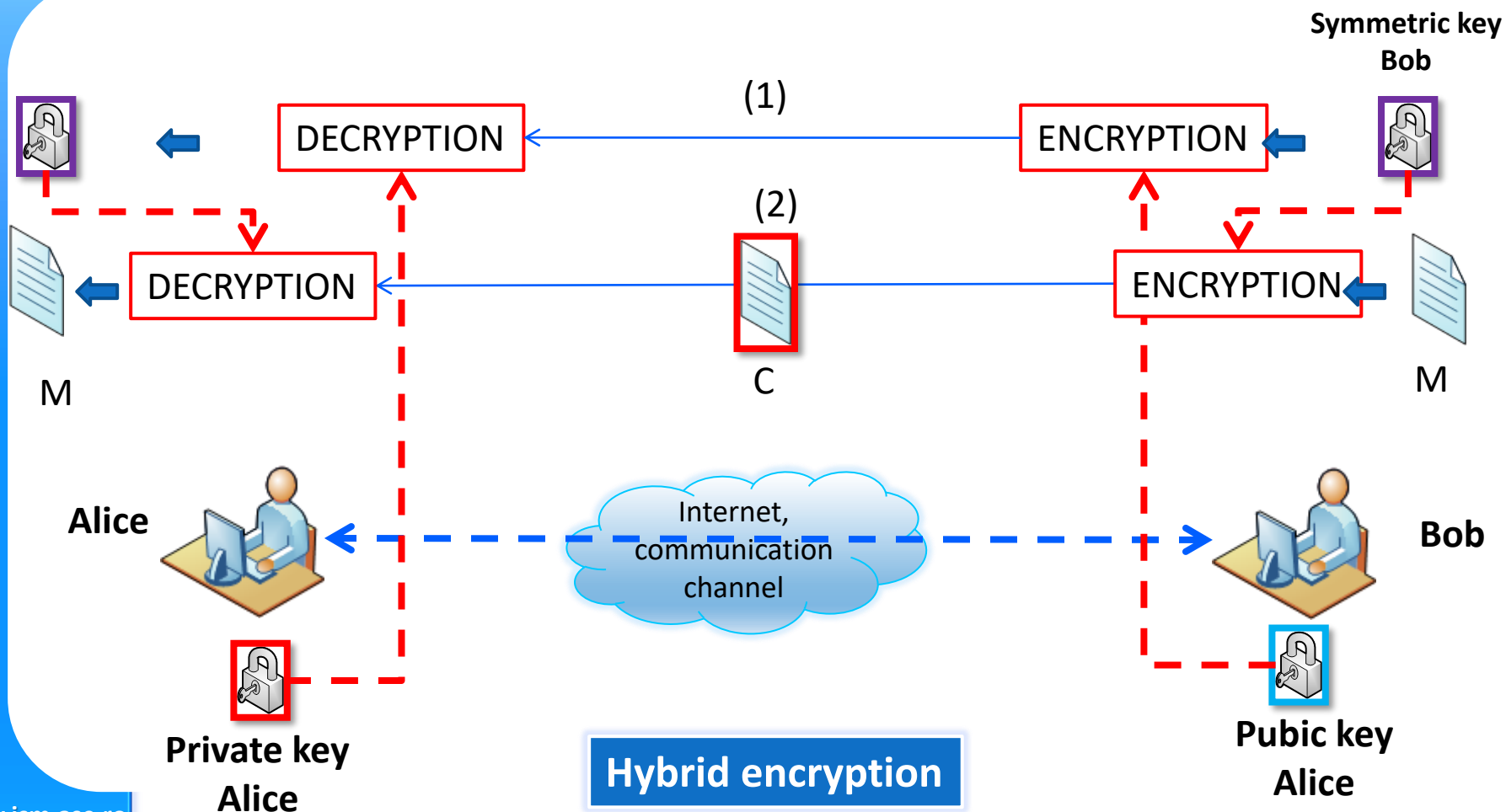
(Rivest-Shamir-Adleman)

Analysis:

- A RSA is approximately 1000 times slower than DES for a hardware implementation and approximately 100 times for a software implementation
- For efficiency reasons hybrid encryption is used

RSA

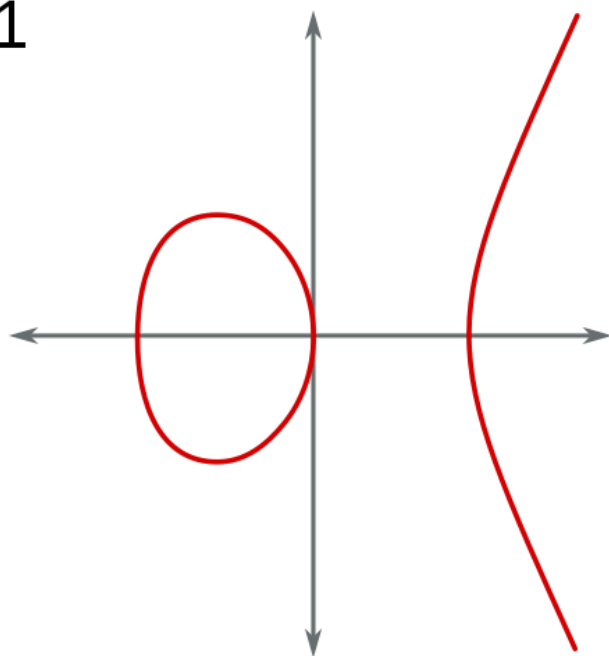
(Rivest-Shamir-Adleman)





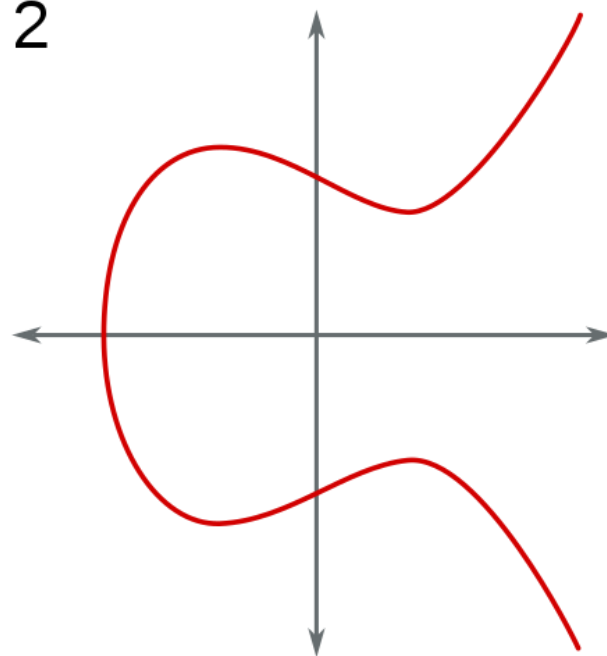
Elliptic Curves

1



$$y^2 = x^3 - x$$

2



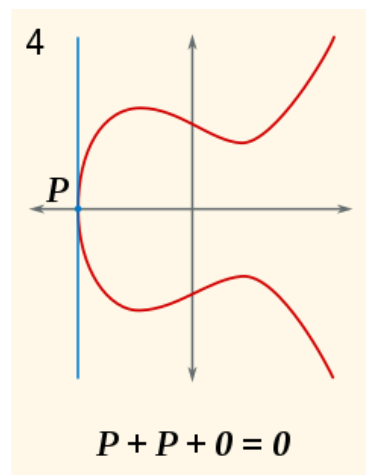
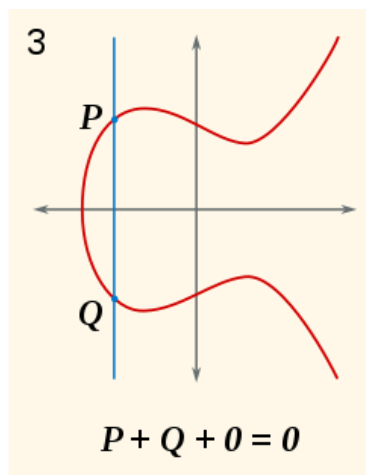
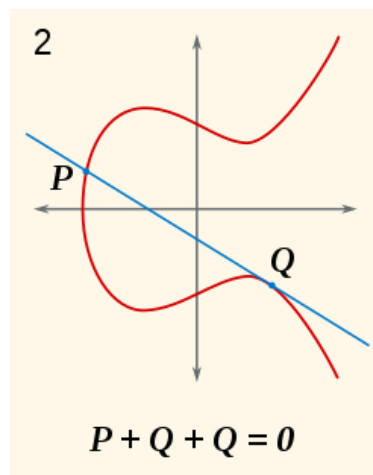
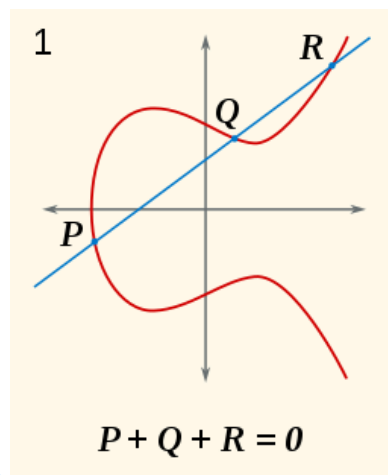
$$y^2 = x^3 - x + 1$$

https://en.wikipedia.org/wiki/Elliptic_curve



Elliptic Curves

- The curve is symmetrical about the x-axis; any point P , has an opposite point, $-P$
- If P and Q are two points on the curve, then we can uniquely describe a third point, $P + Q$, in the following way. First, draw the line that intersects P and Q . This will generally intersect the cubic at a third point, R . We then take $P + Q$ to be $-R$, the point opposite R





Elliptic Curves Cryptography

- Used for key agreement, digital signatures, pseudo-random generators
- Used mostly to distribute symmetric keys
- Is a public key encryption technique that can be used to create faster, smaller cryptographic keys
- ECC generates keys using the properties of the elliptic curve equation (RSA uses product of very large prime numbers)



Elliptic Curves Cryptography

- The use of elliptic curves in cryptography was suggested independently by [Neal Koblitz and Victor S. Miller](#) in 1985
- Elliptic curve cryptography algorithms entered wide use around 2004
- <http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-8951-CryptoAuth-RSA-ECC-Comparison-Embedded-Systems-WhitePaper.pdf>



Elliptic Curves Cryptography

- Multiplying a point on the curve by a number will produce another point on the curve
- It is very difficult to find what number was used, even if you know the original point and the result.
- Equations based on elliptic curves are relatively easy to perform, and extremely difficult to reverse



Elliptic Curves Cryptography

- [Lenstra, Kleinjung and Thome](#) defined in 2013 the concept of "Global Security." - compute how much energy is needed to break a cryptographic algorithm and compare that with how much water that energy could boil.
- Breaking a 228-bit RSA key requires less energy than it takes to boil a teaspoon of water. Comparatively, breaking a 228-bit elliptic curve key requires enough energy to boil all the water on earth. For this level of security with RSA, you'd need a key with 2,380-bits



<https://www.ssl2buy.com/wiki/rsa-vs-ecc-which-is-better-algorithm-for-security>



Elliptic Curves

- The [Elliptic Curve Diffie–Hellman](#) (ECDH) key agreement scheme is based on the [Diffie–Hellman](#) scheme,
- The Elliptic Curve [Integrated Encryption Scheme](#) (ECIES), also known as Elliptic Curve Encryption Scheme,
- The [Elliptic Curve Digital Signature Algorithm](#) (ECDSA) is based on the [Digital Signature Algorithm](#),
- The [Edwards-curve Digital Signature Algorithm](#) (EdDSA) is based on [Schnorr signature](#) and uses [twisted Edwards curves](#),
- The [ECMQV](#) key agreement scheme is based on the [MQV](#) key agreement scheme



Controversy

- [NIST Special Publication 800-90](#) (.pdf) on DRBGs, or "Deterministic Random Bit Generators."
 - Describes 4 PRNGs, one based on ECC
 - [Dual EC DRBG](#) (**Dual Elliptic Curve Deterministic Random Bit Generator**) – contains a lot of constants which are not explained ([CRYPTO 2007 paper](#))
 - If you know the secret numbers, you can predict the output of the random-number generator after collecting [just 32 bytes of its output](#). By monitoring the [TLS](#) internet encryption connection you can break the protocol security
- https://www.schneier.com/essays/archives/2007/11/did_nsa_put_a_secret.html



Symmetric vs. Asymmetric

Symmetric algorithms advantages:

- High encryption power
- Easy to implement
- Relatively short keys (128,192, 256 bits)
- Can be composed into new cryptographic systems (multiple, sequential – 3DES)
- Tested into practice
- Used to generate pseudo-random numbers or hash functions



Symmetric vs. Asymmetric

Symmetric algorithms disadvantages :

- Secret key known by both parts
- Long message's length is a vulnerability
- In distributed systems, key management is an issue
- Key transmission is a security problem
- Frequent key modification leads to rerun transmission process



Symmetric vs. Asymmetric

Asymmetric algorithms advantages :

- Simple solution to send data through unsure channels
- Easy to implement
- Secret key found in one place
- Define the base for distributed applications: electronic signature, authentication, electronic payments
- Key pair (public and private) can be modified without too much effort

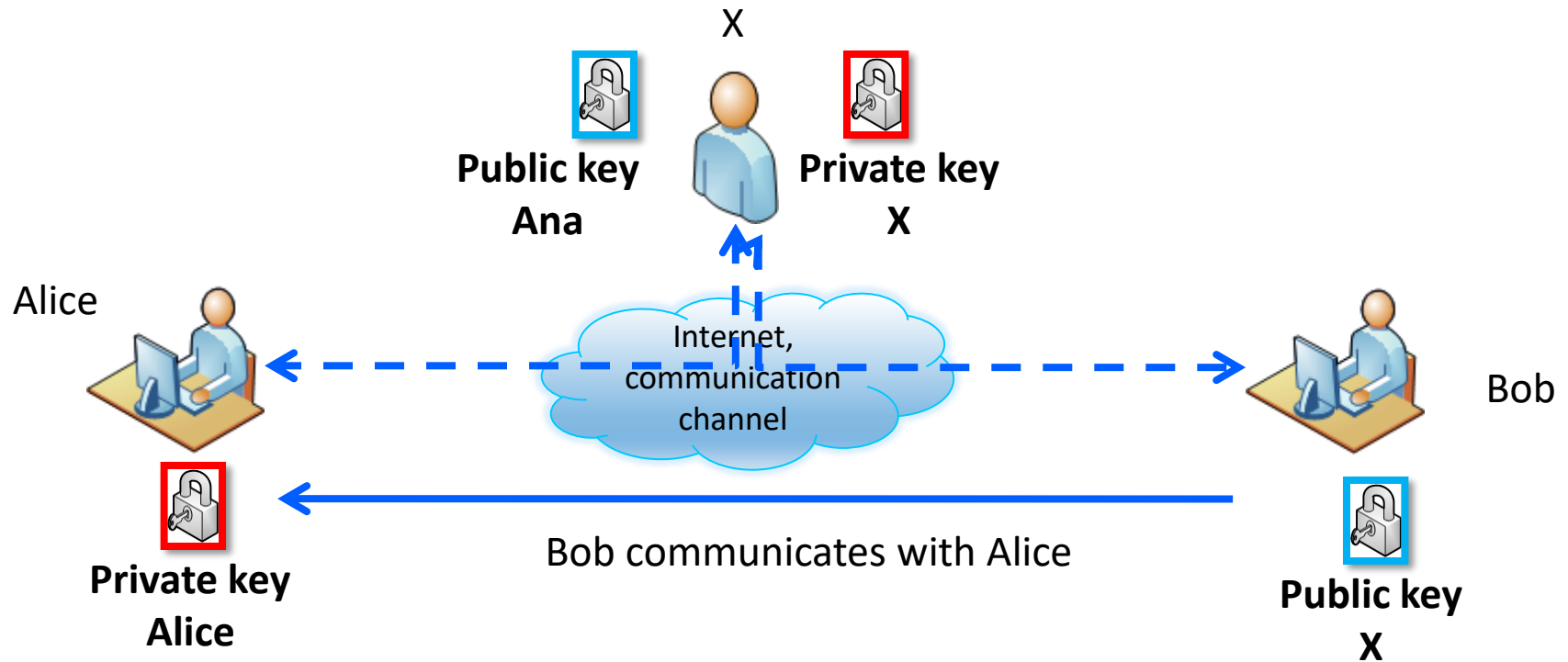


- Low encryption power
- Need big length keys
- Does not guarantee absolute security
- Level of security depends on implementation
- **Anyone can make encryptions=> can find certain weak points that allow message decryption**
- **No authentication, very vulnerable to man-in-the-middle type attacks**



Symmetric vs. Asymmetric

Man-in-the-middle attack on no authentication asymmetric system





Cryptanalysis

Encrypt analysis is the art of decrypting messages (files, data structures, communications) encrypted without knowing the decryption key

The encryption method or algorithm **MUST NOT** be secret, only the key. Cryptographic security means key protection.

The security of a cryptosystem should rest in the key



Cryptanalysis

Objectives:

- recover the key, not just the plaintext message for a ciphertext

Strategies:

- **Brute-force attack:** try every possible key
- **Cryptanalytic attack:** finds the key based on knowledge on the algorithm, plaintext and cipher text



- www.ism.ase.ro



Cryptanalysis

- **Ciphertext-only attack**
 - The analysis of statistics frequencies (certain characters appear with medium frequency in different languages)
 - Exhaustive search of the keys part of the encryption
- **Known-plaintext attack:**
 - The attacker knows decrypted text for certain cryptogram parts – brute-force attack
 - Linear encrypt-analysis against block ciphers (Matsui, 1994)



Cryptanalysis

- **Chosen plaintext attack**
 - The attacker can choose plaintext and obtain the ciphertext
- **Chosen ciphertext attack:**
 - The attacker can choose ciphertext and obtain the plaintext
- **Chosen text attack:**
 - The attacker can choose plaintext/ciphertext to encrypt/decrypt



Cryptanalysis

Brute-force attack:

- Trying every possible key
- It is needed a small amount of ciphertext and the corresponding plaintext
- The attack complexity = 2^n , where n is the key length in bits
- Based on a birthday-paradox, there is a 50% probability to find the solution in $2^{n/2}$ tries



Cryptanalysis

Brute-force attack with a **million/second** key search computer [3]:

- for a 56 bits key – 2285 years
- for a 64 bit key – 585000 years
- for a 128 bit key – 10^{25} years

An efficient brute-force attack is implemented in a parallel architecture (Ex The Chinese lottery [3])



Cryptanalysis

Key Size (bits)	Number of Alternative Keys	Time required at 1 decryption/ μ s	Time required at 10^6 decryptions/ μ s
32	$2^{32} = 4.3 \times 10^9$	$2^{31} \mu\text{s} = 35.8 \text{ minutes}$	2.15 milliseconds
56	$2^{56} = 7.2 \times 10^{16}$	$2^{55} \mu\text{s} = 1142 \text{ years}$	10.01 hours
128	$2^{128} = 3.4 \times 10^{38}$	$2^{127} \mu\text{s} = 5.4 \times 10^{24} \text{ years}$	$5.4 \times 10^{18} \text{ years}$
168	$2^{168} = 3.7 \times 10^{50}$	$2^{167} \mu\text{s} = 5.9 \times 10^{36} \text{ years}$	$5.9 \times 10^{30} \text{ years}$
26 characters (permutation)	$26! = 4 \times 10^{26}$	$2 \times 10^{26} \mu\text{s} = 6.4 \times 10^{12} \text{ years}$	$6.4 \times 10^6 \text{ years}$

Source [7]



Cryptanalysis

Character Type Difference
Combining ASCII, Lowercase, Uppercase, and Numeric

"Password"
Cracked just under the time
it would take lightning to strike 2-3 times

"P@ssw0rD"
Will be cracked in the same amount of time
it took to carve Mt. Rushmore, or 14 years.

Don't be naive.
Remember
Entropy ?



Dictionary attack:

- the attacker uses a dictionary of common keys
- based on common names, places, famous people, cartoons characters, locations and names from movies, books, mythology, sports,
- works better against a file of keys
- based on a preliminary analysis of the target



Security of Monoalphabetic Cipher

- each plaintext letter maps to a different random ciphertext letter
- a total of $26! = 4 \times 10^{26}$ keys
- with so many keys, might think is secure - **!!!WRONG!!!**
- problem is language characteristics



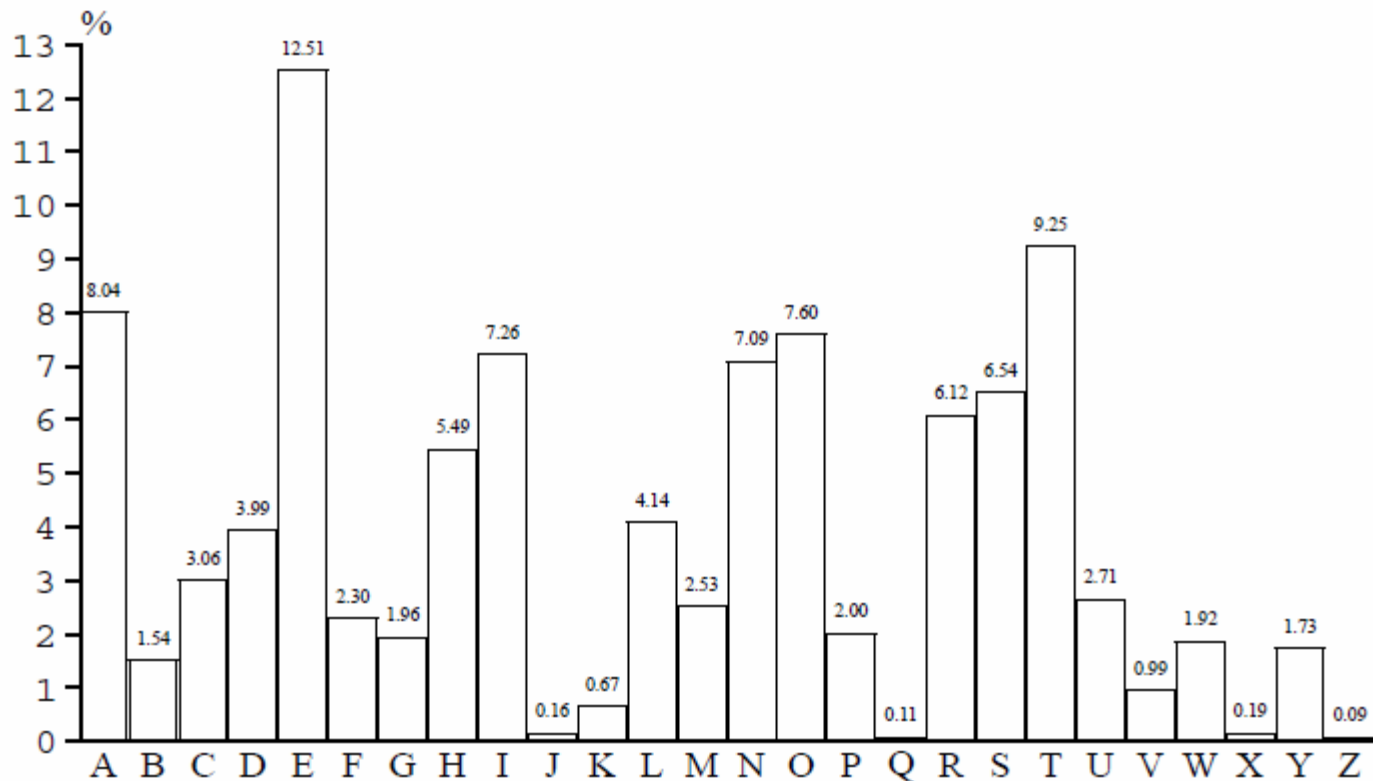
Cryptanalysis

Language Redundancy and Cryptanalysis:

- human languages are **redundant**
- You can read this "nfrmtcs scrty mstr"
- letters are not equally commonly used
- in English E is by far the most common letter, followed by T,R,N,I,O,A,S
- based on tables of single, double & triple letter frequencies for various languages



Letter frequency statistic attacks

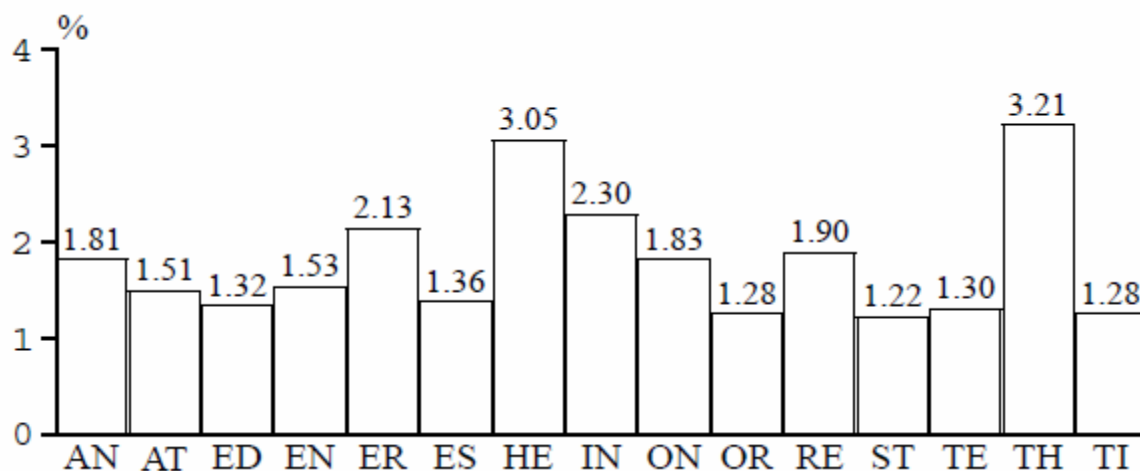


Handbook of Applied Cryptography

Letters' frequency in English language



Letter frequency statistic attacks



Handbook of Applied Cryptography

Frequency of 15 most used digrams in English language



Letter frequency statistic attacks

- key concept - monoalphabetic substitution ciphers do not change relative letter frequencies
- discovered by Arabian scientists in 9th century
- calculate letter frequencies for ciphertext
- compare counts/plots against known values
- if caesar cipher look for common peaks/troughs
 - peaks at: A-E-I triple, NO pair, RST triple
 - troughs at: JK, X-Z
- for monoalphabetic must identify each letter
 - tables of common double/triple letters help



Letter frequency statistic attacks

- monoalphabetic substitution cipher was broken by Arabic scientists - Abu al-Kindi's "*A Manuscript on Deciphering Cryptographic Messages*", published in the 9th century but only rediscovered in 1987 in Istanbul;
- The cryptanalyst looks for a mapping between the observed pattern in the ciphertext, and the known source language letter frequencies



Example of Cryptanalysis

- given ciphertext:

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFP
ESXUDBMETSXAI ZVUEPHZHMDZSHZOW
SFPAPPDTSVPQUZWYMXUZUHSXEPYEP
OPDZSZUFPOMBZWPFUPZHMDJUDTMOH
MQ



Example of Cryptanalysis

- You get the plaintext:

it was disclosed yesterday
that several informal but
direct contacts have been
made with political
representatives of the
viet cong in moscow



Caesar Cipher Cryptanalysis

- only have 26 possible ciphers
 - A maps to A,B,..Z
- a **brute force search**
- given ciphertext, just try all shifts of letters
- You need to recognize when have plaintext
- eg. break ciphertext "GCUA VQ DTGCM"



Cryptanalysis

- *Man-in-the-middle* attack or *fire brigade bucket* attack
- Correlation between secret key and cryptosystem exit is the main source of information for a cryptanalyst.
- Attacks by speculating Faults in cryptosystem
 - Mistakes made in cryptosystem design lead the cryptanalyst to find the secrete key



Cryptanalysis

- Attacks via quantum and genetic channel



Security of Vigenère Ciphers

- have multiple ciphertext letters for each plaintext letter
- hence letter frequencies are obscured
- but not totally lost
- start with letter frequencies
 - see if look monoalphabetic or not
- if not, then need to determine number of alphabets, since then can attach each



Kasiski Method

- method developed by Babbage / Kasiski
- repetitions in ciphertext give clues to period
- so find same plaintext an exact period apart
- which results in the same ciphertext
- of course, could also be random fluke
- eg repeated “VTW” in previous example
- suggests size of 3 or 9
- then attack each monoalphabetic cipher individually using same techniques as before



Strength of DES – Key Size

- 56-bit keys have $2^{56} = 7.2 \times 10^{16}$ values
- brute force search looks hard
- recent advances have shown is possible
 - in 1997 on Internet in a few months
 - in 1998 on dedicated h/w (EFF) in a few days
 - in 1999 above combined in 22hrs!
- still must be able to recognize plaintext
- must now consider alternatives to DES



Strength of DES – Analytic Attacks

- now have several analytic attacks on DES
- these utilise some deep structure of the cipher
 - by gathering information about encryptions
 - can eventually recover some/all of the sub-key bits
 - if necessary then exhaustively search for the rest
- generally these are statistical attacks
- include
 - differential cryptanalysis
 - linear cryptanalysis
 - related key attacks



Strength of DES – Timing Attacks

- attacks actual implementation of cipher
- use knowledge of consequences of implementation to derive information about some/all subkey bits
- specifically use fact that calculations can take varying times depending on the value of the inputs to it
- particularly problematic on smartcards



Differential Cryptanalysis

- one of the most significant recent (public) advances in cryptanalysis
- known by NSA in 70's cf DES design
- Murphy, Biham & Shamir published in 90's
- powerful method to analyse block ciphers
- used to analyse most current block ciphers with varying degrees of success
- DES reasonably resistant to it, cf Lucifer



Differential Cryptanalysis

- a statistical attack against Feistel ciphers
- uses cipher structure not previously used
- design of S-P networks has output of function f influenced by both input & key
- hence cannot trace values back through cipher without knowing value of the key
- differential cryptanalysis compares two related pairs of encryptions



Differential Cryptanalysis Compares Pairs of Encryptions

- with a known difference in the input
- searching for a known difference in output
- when same subkeys are used

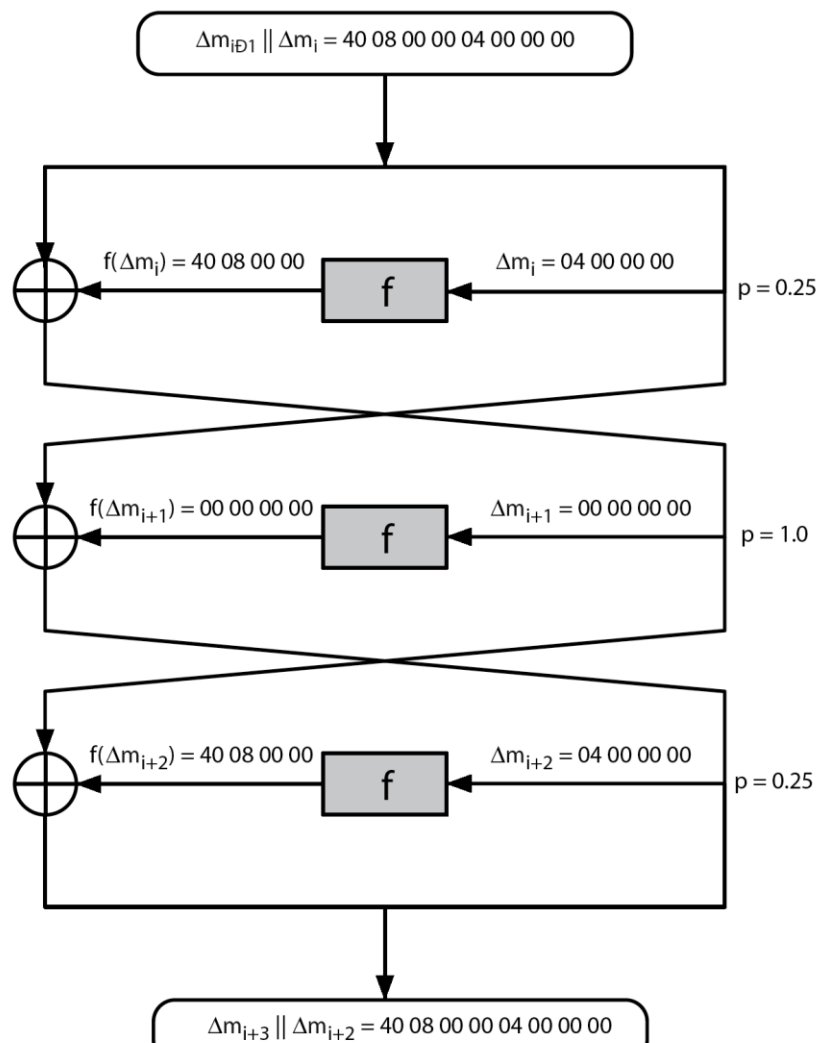
$$\begin{aligned}\Delta m_{i+1} &= m_{i+1} \oplus m'_{i+1} \\ &= [m_i \oplus f(m_i, K_i)] \oplus [m'_i \oplus f(m'_i, K_i)] \\ &= \Delta m_i \oplus [f(m_i, K_i) \oplus f(m'_i, K_i)]\end{aligned}$$



Differential Cryptanalysis

- have some input difference giving some output difference with probability p
- if find instances of some higher probability input / output difference pairs occurring
- can infer subkey that was used in round
- then must iterate process over many rounds (with decreasing probabilities)

Differential Cryptanalysis





Differential Cryptanalysis

- perform attack by repeatedly encrypting plaintext pairs with known input XOR until obtain desired output XOR
- when found
 - if intermediate rounds match required XOR have a **right pair**
 - if not then have a **wrong pair**, relative ratio is S/N for attack
- can then deduce keys values for the rounds
 - right pairs suggest same key bits
 - wrong pairs give random values
- for large numbers of rounds, probability is so low that more pairs are required than exist with 64-bit inputs
- Biham and Shamir have shown how a 13-round iterated characteristic can break the full 16-round DES



- another recent development
- also a statistical method
- must be iterated over rounds, with decreasing probabilities
- developed by Matsui et al in early 90's
- based on finding linear approximations
- can attack DES with 2^{43} known plaintexts, easier but still in practise infeasible



Linear Cryptanalysis

- find linear approximations with prob $p \neq \frac{1}{2}$

$$P[i_1, i_2, \dots, i_a] \oplus C[j_1, j_2, \dots, j_b] = K[k_1, k_2, \dots, k_c]$$

where i_a, j_b, k_c are bit locations in P, C, K

- gives linear equation for key bits
- get one key bit using max likelihood alg
- using a large number of trial encryptions
- effectiveness given by: $|p - \frac{1}{2}|$



Future developments

- Privately outsourcing computation
 - Use encrypted data without accessing it
 - Homomorphic encryption
- Quantum cryptography